

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**



Họ và tên sinh viên

Nguyễn Văn Khánh

**HỆ THỐNG QUẢN LÝ DINH DƯỠNG CÁ NHÂN
HÓA**

(Personalized Nutrition Management System)

ĐỒ ÁN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Chuyên ngành: Khoa học máy tính

Hà Nội – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**



Họ và tên sinh viên

Nguyễn Văn Khánh

HỆ THỐNG QUẢN LÝ DINH DƯỠNG CÁ NHÂN HÓA

(Personalized Nutrition Management System)

ĐỒ ÁN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Chuyên ngành: Khoa học máy tính

Giáo viên hướng dẫn: TS. Phạm Hồng Phong

Sinh viên thực hiện: Nguyễn Văn Khánh

Mã số sinh viên: 0241967

Lớp: 67CS2

Hà Nội – 2026

LỜI NÓI ĐẦU

Trong quá trình học tập tại Trường Đại học Xây dựng Hà Nội, em đã được trang bị các kiến thức nền tảng và chuyên ngành về lập trình, cơ sở dữ liệu, phân tích thiết kế hệ thống, phát triển ứng dụng web và các phương pháp xây dựng phần mềm. Đồ án tốt nghiệp là cơ hội để em tổng hợp, vận dụng và kiểm chứng những kiến thức đã học vào một bài toán cụ thể. Trên cơ sở đó, đề tài **“Xây dựng hệ thống quản lý dinh dưỡng cá nhân”** được thực hiện nhằm xây dựng một hệ thống hỗ trợ người dùng theo dõi nhật ký dinh dưỡng, quản lý chỉ số cơ thể, phân tích dữ liệu ăn uống và sinh báo cáo theo thời gian.

Trong quá trình thực hiện đồ án, em đã tìm hiểu các kiến thức liên quan đến quản lý dinh dưỡng cá nhân, thiết kế cơ sở dữ liệu, xây dựng RESTful API, phát triển giao diện người dùng bằng React và triển khai backend bằng Node.js, Express. Bên cạnh các chức năng quản lý dữ liệu cơ bản, hệ thống còn được xây dựng thêm một số thành phần xử lý dữ liệu dinh dưỡng như tính toán chỉ số nền tảng, điều chỉnh TDEE theo dữ liệu thực tế, chấm điểm món ăn, sinh cảnh báo, lập kế hoạch bữa ăn và quét thông tin sản phẩm. Quá trình thực hiện đề tài giúp em củng cố thêm kiến thức về tổ chức mã nguồn, thiết kế hệ thống theo mô hình client-server, xử lý dữ liệu người dùng và kiểm thử chức năng trong phạm vi một hệ thống web phục vụ đồ án tốt nghiệp.

Em xin gửi lời cảm ơn chân thành đến **[Học hàm, học vị và họ tên giảng viên hướng dẫn]**, người đã tận tình hướng dẫn, góp ý và định hướng cho em trong suốt quá trình thực hiện đồ án tốt nghiệp. Những nhận xét và chỉ dẫn của thầy/cô là cơ sở quan trọng giúp em hoàn thiện nội dung nghiên cứu, điều chỉnh cách trình bày báo cáo và cải thiện chất lượng triển khai hệ thống.

Em cũng xin gửi lời cảm ơn đến các thầy cô trong **[Tên khoa/viện]**, Trường Đại học Xây dựng Hà Nội, đã truyền đạt cho em những kiến thức chuyên môn cần thiết trong suốt quá trình học tập. Những kiến thức này là nền tảng giúp em có thể tiếp cận, phân tích và triển khai đề tài tốt nghiệp một cách hệ thống hơn.

Mặc dù đã cố gắng hoàn thiện báo cáo và hệ thống trong phạm vi khả năng, đồ án vẫn khó tránh khỏi những thiếu sót về nội dung, kỹ thuật triển khai và cách trình bày. Em kính mong nhận được sự góp ý của các thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày tháng năm 2026

Sinh viên thực hiện

Nguyễn Văn Khánh

CAM ĐOAN

Tôi xin cam đoan đề án tốt nghiệp với đề tài **“Xây dựng hệ thống quản lý dinh dưỡng cá nhân”** là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của **[Học hàm, học vị và họ tên giảng viên hướng dẫn]**.

Các nội dung trình bày trong đề án, bao gồm phần phân tích, thiết kế, xây dựng hệ thống, thực nghiệm và đánh giá kết quả, được thực hiện dựa trên quá trình nghiên cứu, triển khai và kiểm thử của bản thân. Các số liệu, kết quả thực nghiệm, bảng biểu và hình ảnh sử dụng trong báo cáo là trung thực, phản ánh đúng phạm vi hệ thống đã xây dựng.

Các tài liệu, công thức, khái niệm, công nghệ và thông tin tham khảo từ nguồn bên ngoài đều được trích dẫn và ghi rõ trong phần tài liệu tham khảo của đề án. Tôi không sao chép nội dung từ các công trình khác mà không ghi nguồn, không sử dụng kết quả nghiên cứu của người khác làm kết quả của mình và hoàn toàn chịu trách nhiệm về tính trung thực của nội dung báo cáo.

Tôi xin chịu trách nhiệm trước Nhà trường, Khoa/Bộ môn và Hội đồng chấm đề án tốt nghiệp về lời cam đoan này.

Hà Nội, ngày tháng năm 2026

Sinh viên thực hiện

Nguyễn Văn Khánh

MỤC LỤC

Lời nói đầu	i
Cam đoan	ii
Mục lục	iii
	Trang
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	4
1.1. Tổng quan bài toán quản lý dinh dưỡng cá nhân	4
1.1.1.Đặc điểm của bài toán	4
1.1.2.Các yêu cầu đặt ra đối với hệ thống	5
1.2. Mục tiêu và yêu cầu của hệ thống	6
1.2.1.Yêu cầu phi chức năng	6
1.3. Công nghệ và nền tảng phát triển	6
1.3.1.React	7
1.3.2.Node.js và Express	7
1.3.3.MySQL và Sequelize ORM	8
1.3.4.Các thư viện và dịch vụ hỗ trợ	8
1.3.5.Đánh giá việc lựa chọn công nghệ	10
1.4. Kiến trúc tổng thể của hệ thống	10
1.4.1.Mô hình kiến trúc Client - Server	11
1.4.2.Kiến trúc Frontend	11
1.4.3.Kiến trúc Backend	12
1.4.4.Kiến trúc cơ sở dữ liệu	12
1.4.5.Đánh giá kiến trúc hệ thống	13
1.5. Kết luận chương	13
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	15
2.1. Phân tích yêu cầu	15
2.1.1.Yêu cầu chức năng	15
2.1.2.Yêu cầu phi chức năng	17
2.1.3.Tổng kết yêu cầu	18
2.2. Phân tích nghiệp vụ	18
2.2.1.Nghiệp vụ đăng nhập và khởi tạo hồ sơ sức khỏe	20

2.2.2.Nghiệp vụ ghi nhật ký ăn uống và nước uống	20
2.2.3.Nghiệp vụ sử dụng Hybrid Nutrition Scanner	21
2.2.4.Nghiệp vụ xây dựng thực đơn	21
2.2.5.Tổng kết nghiệp vụ	22
2.3. Thiết kế chức năng	22
2.3.1.Tác nhân và hệ thống liên quan	22
2.3.2.Biểu đồ Use Case tổng thể	23
2.3.3.Danh sách Use Case chính	23
2.3.4.Mô tả một số Use Case tiêu biểu	25
2.3.5.Quan hệ giữa các Use Case	27
2.3.6.Tổng kết thiết kế chức năng	28
2.4. Thiết kế cơ sở dữ liệu	28
2.4.1.Nguyên tắc thiết kế cơ sở dữ liệu	28
2.4.2.Phân nhóm bảng dữ liệu	29
2.4.3.Quan hệ giữa các bảng chính	31
2.4.4.Thiết kế lưu trữ dữ liệu lịch sử	33
2.4.5.Thiết kế dữ liệu phục vụ Scanner	33
2.4.6.Thiết kế hỗ trợ truy vấn và tổng hợp dữ liệu	33
2.4.7.Tổng kết thiết kế cơ sở dữ liệu	34
2.5. Thiết kế API	34
2.5.1.Nguyên tắc thiết kế API	34
2.5.2.Cấu trúc phản hồi API	35
2.5.3.Thiết kế xác thực API	36
2.5.4.Các nhóm API chính	37
2.5.5.Một số endpoint tiêu biểu	38
2.5.6.Xử lý lỗi và mã trạng thái	40
2.5.7.Tổng kết thiết kế API	41
2.6. Thiết kế giao diện	41
2.6.1.Nguyên tắc thiết kế giao diện	42
2.6.2.Cấu trúc điều hướng và bố cục ứng dụng	42
2.6.3.Thiết kế giao diện Dashboard	43
2.6.4.Thiết kế giao diện nhật ký ăn uống và luồng thêm thực phẩm	44
2.6.5.Thiết kế giao diện Meal Planner	44
2.6.6.Tổng hợp các màn hình chức năng còn lại	45
2.6.7.Tổng kết thiết kế giao diện	46
2.7. Kết luận chương	46

CHƯƠNG 3. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	48
3.1. Triển khai Backend	48
3.1.1.Cấu trúc khởi tạo và middleware	49
3.1.2.Tổ chức route và RESTful API	50
3.1.3.Controller, service và xử lý nghiệp vụ	51
3.1.4.Kết nối cơ sở dữ liệu và tổ chức model	51
3.1.5.Xác thực và tổng hợp xử lý phía Backend	52
3.2. Triển khai Frontend	53
3.2.1.Tổ chức ứng dụng React	54
3.2.2.Điều hướng và kiểm soát truy cập giao diện	54
3.2.3.Quản lý trạng thái xác thực và dữ liệu server	55
3.2.4.Xây dựng giao diện theo component	56
3.2.5.Liên kết Frontend với Backend	57
3.2.6.Nutrition Core	57
3.2.7.Adaptive TDEE	60
3.2.8.Food Scoring	64
3.2.9.Health Insights	68
3.2.10Meal Planner	72
3.3. Hybrid Nutrition Scanner	77
3.4. Bảo mật và xử lý lỗi	81
3.5. Kết luận chương	83
 CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ	 85
4.1. Môi trường thực nghiệm	85
4.2. Kiểm thử chức năng	87
4.3. Thực nghiệm các thuật toán chính	91
4.4. Đánh giá kết quả	94
4.5. Hạn chế và hướng phát triển	94
4.6. Kết luận chương	96
 KẾT LUẬN	 98

MỞ ĐẦU

Lý do chọn đề tài

Quản lý chế độ dinh dưỡng là một trong những yếu tố quan trọng trong việc duy trì sức khỏe và phòng ngừa các bệnh liên quan đến lối sống. Để xây dựng một chế độ ăn phù hợp, người sử dụng cần theo dõi nhiều thông tin như lượng năng lượng tiêu thụ, thành phần dinh dưỡng của thực phẩm, cân nặng, mức độ hoạt động thể chất và sự thay đổi của các chỉ số sức khỏe theo thời gian. Việc ghi chép và tổng hợp các thông tin này bằng phương pháp thủ công thường mất nhiều thời gian, đồng thời khó đảm bảo tính chính xác khi khối lượng dữ liệu ngày càng tăng.

Các ứng dụng theo dõi dinh dưỡng hiện nay thường tập trung vào chức năng ghi nhận lượng calo hoặc lưu trữ nhật ký ăn uống. Tuy nhiên, việc đánh giá chất lượng của chế độ ăn không chỉ phụ thuộc vào tổng năng lượng tiêu thụ mà còn liên quan đến sự cân đối giữa các nhóm chất dinh dưỡng, mục tiêu sức khỏe của từng cá nhân và sự thay đổi của cơ thể trong quá trình sử dụng. Ngoài bài toán đánh giá dữ liệu dinh dưỡng, quá trình nhập liệu thực phẩm cũng ảnh hưởng đáng kể đến trải nghiệm của người sử dụng, đặc biệt đối với các sản phẩm đóng gói hoặc thực phẩm chưa có trong cơ sở dữ liệu.

Xuất phát từ những vấn đề trên, đề tài "**Xây dựng hệ thống quản lý dinh dưỡng cá nhân**" được thực hiện nhằm xây dựng một hệ thống hỗ trợ quản lý thông tin sức khỏe, theo dõi nhật ký dinh dưỡng và phân tích dữ liệu dựa trên các thuật toán được triển khai trong hệ thống. Ngoài các chức năng quản lý dữ liệu, đề tài tập trung nghiên cứu kiến trúc hệ thống, thiết kế cơ sở dữ liệu và xây dựng các thuật toán phục vụ tính toán nhu cầu năng lượng, đánh giá chất lượng thực phẩm, phân tích dữ liệu dinh dưỡng và hỗ trợ nhập liệu bằng công nghệ nhận dạng. Điểm nghiên cứu của đề tài tập trung vào việc thiết kế và triển khai các thuật toán phục vụ phân tích dữ liệu dinh dưỡng, góp phần nâng cao khả năng hỗ trợ theo dõi và đánh giá tình trạng dinh dưỡng của hệ thống.

Mục tiêu nghiên cứu

Đề tài hướng đến việc xây dựng một hệ thống quản lý dinh dưỡng cá nhân hoạt động trên nền tảng web, cho phép người dùng quản lý thông tin sức khỏe và theo dõi chế độ ăn uống hằng ngày thông qua một cơ sở dữ liệu tập trung.

Các mục tiêu cụ thể của đề tài bao gồm:

- Xây dựng hệ thống quản lý tài khoản và hồ sơ sức khỏe của người dùng.
- Xây dựng chức năng ghi nhận nhật ký ăn uống, theo dõi cân nặng, lượng nước uống và hoạt động luyện tập.
- Xây dựng chức năng tính toán chỉ số BMI, BMR, TDEE và mục tiêu dinh dưỡng dựa trên thông tin cá nhân.
- Triển khai thuật toán Adaptive TDEE để hiệu chỉnh nhu cầu năng lượng theo dữ

liệu thực tế của người dùng.

- Triển khai thuật toán đánh giá mật độ dinh dưỡng của thực phẩm và thuật toán phân tích tình trạng dinh dưỡng trong ngày.
- Tích hợp chức năng quét mã vạch và nhận dạng bảng thành phần dinh dưỡng nhằm hỗ trợ nhập liệu đối với thực phẩm đóng gói.
- Xây dựng chức năng tổng hợp dữ liệu và xuất báo cáo dưới định dạng PDF.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm các phương pháp xây dựng hệ thống quản lý dinh dưỡng trên nền tảng web, kiến trúc phần mềm theo mô hình Client - Server, thiết kế cơ sở dữ liệu quan hệ và các thuật toán phục vụ tính toán, phân tích dữ liệu dinh dưỡng.

Ngoài ra, đề tài cũng nghiên cứu việc tích hợp các dịch vụ hỗ trợ tra cứu thông tin thực phẩm theo mã vạch và nhận dạng bảng thành phần dinh dưỡng từ hình ảnh nhằm hỗ trợ quá trình nhập liệu.

Phạm vi nghiên cứu

Đề tài tập trung xây dựng hệ thống quản lý dinh dưỡng dành cho người dùng cá nhân với các chức năng quản lý hồ sơ sức khỏe, nhật ký ăn uống, theo dõi cân nặng, theo dõi hoạt động luyện tập, xây dựng thực đơn, phân tích dữ liệu dinh dưỡng, quét thực phẩm và xuất báo cáo.

Hệ thống được phát triển dưới dạng ứng dụng web sử dụng React ở phía giao diện người dùng, Node.js và Express ở phía máy chủ, MySQL làm hệ quản trị cơ sở dữ liệu và Sequelize ORM để quản lý truy cập dữ liệu.

Trong phạm vi của đồ án, đề tài chưa xem xét việc xây dựng ứng dụng trên nền tảng di động, tích hợp với các thiết bị đeo thông minh hoặc phát triển các chức năng dành cho chuyên gia dinh dưỡng và cơ sở y tế.

Phương pháp nghiên cứu

Đề tài được thực hiện thông qua các phương pháp nghiên cứu sau:

- Nghiên cứu các tài liệu liên quan đến quản lý dinh dưỡng, các công thức tính toán chỉ số cơ thể và phương pháp đánh giá dữ liệu dinh dưỡng.
- Nghiên cứu các công nghệ phát triển ứng dụng web và kiến trúc phần mềm phù hợp với bài toán quản lý dữ liệu.
- Phân tích yêu cầu của hệ thống, thiết kế kiến trúc, cơ sở dữ liệu, API và các luồng xử lý nghiệp vụ.
- Triển khai hệ thống theo từng module chức năng và kiểm thử trong quá trình phát triển.

- Đánh giá kết quả thực nghiệm của các chức năng và thuật toán được triển khai trong hệ thống.

Bố cục báo cáo

Ngoài phần Mở đầu, Kết luận, Tài liệu tham khảo và Phụ lục, nội dung chính của báo cáo được trình bày trong bốn chương:

Chương 1. Tổng quan đề tài

Trình bày bài toán quản lý dinh dưỡng cá nhân, mục tiêu nghiên cứu, yêu cầu của hệ thống, các công nghệ được sử dụng và kiến trúc tổng thể của hệ thống.

Chương 2. Phân tích và thiết kế hệ thống

Trình bày kết quả phân tích yêu cầu, thiết kế chức năng, thiết kế cơ sở dữ liệu, thiết kế API và thiết kế giao diện của hệ thống.

Chương 3. Xây dựng và triển khai hệ thống

Trình bày quá trình hiện thực các thành phần của hệ thống, bao gồm Backend, Frontend, các thuật toán tính toán và phân tích dữ liệu dinh dưỡng, cùng các chức năng hỗ trợ nhập liệu và xử lý dữ liệu.

Chương 4. Thực nghiệm và đánh giá

Trình bày môi trường thực nghiệm, kết quả kiểm thử các chức năng và thuật toán, đánh giá kết quả đạt được, các hạn chế của hệ thống và định hướng phát triển trong tương lai.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

Chương này trình bày tổng quan về bài toán quản lý dinh dưỡng cá nhân, các yêu cầu đặt ra đối với hệ thống, mục tiêu xây dựng, công nghệ được sử dụng và kiến trúc tổng thể của hệ thống. Đây là cơ sở để phân tích, thiết kế và triển khai các thành phần của hệ thống trong các chương tiếp theo.

1.1. Tổng quan bài toán quản lý dinh dưỡng cá nhân

Quản lý dinh dưỡng cá nhân là quá trình thu thập, lưu trữ và phân tích các thông tin liên quan đến chế độ ăn uống, hoạt động thể chất và các chỉ số sức khỏe của từng người dùng nhằm hỗ trợ xây dựng chế độ dinh dưỡng phù hợp với mục tiêu cá nhân. Dữ liệu được theo dõi thường bao gồm thông tin cơ thể, lượng năng lượng tiêu thụ, thành phần dinh dưỡng của thực phẩm, cân nặng, mức độ vận động và sự thay đổi của các chỉ số theo thời gian.

Khối lượng dữ liệu cần theo dõi trong quá trình quản lý dinh dưỡng thường tăng lên theo thời gian và có sự thay đổi liên tục theo từng ngày sử dụng. So với nhiều hệ thống quản lý thông tin thông thường, dữ liệu trong bài toán quản lý dinh dưỡng có mối liên hệ chặt chẽ giữa nhiều nhóm thông tin như hồ sơ sức khỏe, nhật ký ăn uống, hoạt động luyện tập và lịch sử cân nặng. Những dữ liệu này vừa được lưu trữ để theo dõi, vừa được sử dụng làm đầu vào cho các phép tính và quá trình phân tích của hệ thống.

Trong phạm vi đề tài, bài toán không chỉ dừng ở việc lưu trữ nhật ký ăn uống mà còn bao gồm việc tính toán các chỉ số sức khỏe, đánh giá chất lượng thực phẩm, phân tích mức độ đáp ứng nhu cầu dinh dưỡng và hỗ trợ người dùng theo dõi quá trình thay đổi của cơ thể. Các chức năng này được xây dựng trên cơ sở kết hợp dữ liệu đầu vào với các thuật toán tính toán và phân tích được triển khai trong hệ thống.

1.1.1. Đặc điểm của bài toán

Bài toán quản lý dinh dưỡng cá nhân có một số đặc điểm khác với nhiều hệ thống quản lý dữ liệu thông thường.

Thứ nhất, dữ liệu đầu vào được hình thành từ nhiều nguồn khác nhau. Ngoài các thông tin do người dùng khai báo như giới tính, tuổi, chiều cao và cân nặng, hệ thống còn tiếp nhận dữ liệu từ nhật ký ăn uống, hoạt động luyện tập, lịch sử cân nặng và thông tin thực phẩm. Các nguồn dữ liệu này có mối liên hệ với nhau và được sử dụng làm đầu vào cho quá trình tính toán và phân tích.

Thứ hai, nhiều chỉ số trong hệ thống không được nhập trực tiếp mà được tính toán từ dữ liệu đã lưu trữ. Ví dụ, chỉ số BMI, BMR, TDEE, mục tiêu năng lượng và nhu cầu các chất dinh dưỡng được xác định dựa trên thông tin hồ sơ sức khỏe của người dùng. Khi dữ liệu đầu vào thay đổi, các giá trị tính toán cũng cần được cập nhật tương ứng.

Thứ ba, dữ liệu được sử dụng đồng thời bởi nhiều chức năng khác nhau trong hệ

thông. Thông tin hồ sơ sức khỏe và nhật ký dinh dưỡng không chỉ phục vụ việc lưu trữ mà còn được sử dụng trong các thuật toán tính toán nhu cầu năng lượng, đánh giá chất lượng thực phẩm, phân tích tình trạng dinh dưỡng và theo dõi sự thay đổi của cơ thể theo thời gian.

Thứ tư, chất lượng của chế độ ăn không thể được đánh giá chỉ thông qua tổng lượng năng lượng tiêu thụ. Hai thực phẩm có cùng lượng năng lượng có thể khác nhau đáng kể về thành phần dinh dưỡng như protein, chất xơ, đường hoặc natri. Ví dụ, 100 kcal từ ức gà và 100 kcal từ nước ngọt đều cung cấp cùng mức năng lượng nhưng giá trị dinh dưỡng hoàn toàn khác nhau. Vì vậy, việc đánh giá cần dựa trên nhiều chỉ số thay vì chỉ sử dụng một giá trị duy nhất.

Cuối cùng, dữ liệu thực phẩm không phải lúc nào cũng đầy đủ trong cơ sở dữ liệu nội bộ. Đối với các sản phẩm đóng gói hoặc sản phẩm mới, hệ thống cần có cơ chế hỗ trợ nhập liệu thông qua tra cứu mã vạch hoặc nhận dạng bằng thành phần dinh dưỡng từ hình ảnh nhằm giảm khối lượng thao tác của người dùng.

Gợi ý chèn Hình 1.1. Tổng quan bài toán quản lý dinh dưỡng cá nhân

Hình minh họa các nguồn dữ liệu đầu vào (hồ sơ sức khỏe, nhật ký ăn uống, cân nặng, luyện tập, thực phẩm) và các kết quả đầu ra như chỉ số sức khỏe, đánh giá dinh dưỡng và báo cáo.

1.1.2. Các yêu cầu đặt ra đối với hệ thống

Từ đặc điểm của bài toán, hệ thống cần đáp ứng một số yêu cầu nhằm đảm bảo quá trình quản lý và phân tích dữ liệu được thực hiện thống nhất.

Trước hết, hệ thống cần lưu trữ đầy đủ các thông tin liên quan đến người dùng và duy trì lịch sử dữ liệu trong quá trình sử dụng. Việc lưu trữ dữ liệu theo thời gian là cơ sở để thực hiện các chức năng thống kê, theo dõi xu hướng và đánh giá sự thay đổi của các chỉ số sức khỏe.

Hệ thống cần thực hiện tự động các phép tính đối với những chỉ số được xác định từ dữ liệu đầu vào, hạn chế việc người dùng phải tính toán thủ công. Kết quả tính toán cần được cập nhật khi thông tin hồ sơ hoặc dữ liệu nhật ký thay đổi.

Đối với dữ liệu thực phẩm, hệ thống cần hỗ trợ nhiều phương thức nhập liệu nhằm tăng khả năng sử dụng trong thực tế. Ngoài việc tìm kiếm từ cơ sở dữ liệu, hệ thống cần hỗ trợ tra cứu theo mã vạch và nhận dạng bằng thành phần dinh dưỡng đối với các sản phẩm chưa có dữ liệu.

Bên cạnh chức năng lưu trữ, hệ thống cần có khả năng tổng hợp và phân tích dữ liệu để hỗ trợ người dùng theo dõi quá trình thực hiện mục tiêu dinh dưỡng. Các kết quả phân tích cần được trình bày dưới dạng trực quan thông qua các chỉ số tổng hợp, biểu đồ và báo cáo.

Bảng 1.1: Các yêu cầu chính của hệ thống

Nhóm yêu cầu	Nội dung
Quản lý dữ liệu	Lưu trữ hồ sơ, nhật ký ăn uống, cân nặng và luyện tập
Tính toán	BMI, BMR, TDEE và mục tiêu dinh dưỡng
Phân tích	Đánh giá chất lượng thực phẩm và phân tích tình trạng dinh dưỡng
Hỗ trợ nhập liệu	Tra cứu mã vạch và nhận dạng bảng thành phần dinh dưỡng
Báo cáo	Dashboard và xuất báo cáo PDF

1.2. Mục tiêu và yêu cầu của hệ thống

Từ các yêu cầu của bài toán quản lý dinh dưỡng cá nhân, hệ thống được xây dựng theo hướng kết hợp giữa quản lý dữ liệu và xử lý nghiệp vụ. Ngoài chức năng lưu trữ thông tin, hệ thống còn thực hiện các phép tính, phân tích và tổng hợp dữ liệu nhằm hỗ trợ người dùng theo dõi quá trình thực hiện mục tiêu dinh dưỡng.

Các chức năng được triển khai trên kiến trúc Client–Server và được tổ chức thành các module tương ứng với từng nhóm nghiệp vụ như quản lý hồ sơ sức khỏe, quản lý nhật ký ăn uống, theo dõi cân nặng, theo dõi luyện tập, phân tích dữ liệu và hỗ trợ nhập liệu. Cách tổ chức này giúp giảm sự phụ thuộc giữa các thành phần, đồng thời thuận lợi cho quá trình phát triển, bảo trì và mở rộng hệ thống.

1.2.1. Yêu cầu phi chức năng

Kiến trúc hệ thống cần hỗ trợ mở rộng, thuận lợi cho việc bổ sung các chức năng mới và bảo trì trong quá trình phát triển.

Để đáp ứng các yêu cầu trên, hệ thống triển khai các thuật toán và cơ chế xử lý dữ liệu phục vụ tính toán nhu cầu năng lượng, điều chỉnh TDEE thích ứng, đánh giá mật độ dinh dưỡng của thực phẩm, phân tích tình trạng dinh dưỡng và hỗ trợ nhập liệu bằng quét mã vạch kết hợp nhận dạng bảng thành phần dinh dưỡng từ hình ảnh. Nội dung chi tiết của các thành phần này được trình bày trong Chương 3.

1.3. Công nghệ và nền tảng phát triển

Để đáp ứng yêu cầu xây dựng một hệ thống quản lý dinh dưỡng có khả năng xử lý dữ liệu theo thời gian, triển khai nhiều thuật toán và hỗ trợ mở rộng chức năng, đề tài lựa chọn kiến trúc Client–Server, trong đó giao diện người dùng, xử lý nghiệp vụ và cơ sở dữ liệu được tách biệt thành các thành phần độc lập. Các thành phần giao tiếp với nhau thông qua RESTful API, tạo điều kiện thuận lợi cho việc phát triển, kiểm thử và bảo trì hệ thống.

Hệ thống được xây dựng hoàn toàn trên nền tảng JavaScript ở cả phía máy khách và máy chủ. Việc sử dụng cùng một ngôn ngữ lập trình giúp giảm sự khác biệt giữa Frontend và Backend, đồng thời thuận lợi cho việc tổ chức mã nguồn và chia sẻ các thành phần xử lý.

Gợi ý chèn Hình 1.3. Các công nghệ sử dụng trong hệ thống

Hình minh họa: React → RESTful API → Express → Service → Sequelize ORM → MySQL. Bên cạnh Backend thể hiện các dịch vụ JWT Authentication, Open Food Facts API và Gemini AI Vision.

1.3.1. React

Giao diện người dùng của hệ thống được phát triển bằng React theo mô hình Single Page Application (SPA). Trong mô hình này, việc chuyển đổi giữa các chức năng được thực hiện trên cùng một trang web, chỉ những thành phần có thay đổi mới được cập nhật thay vì tải lại toàn bộ giao diện. Cách tiếp cận này giúp giảm thời gian phản hồi và cải thiện trải nghiệm sử dụng.

Mã nguồn Frontend được tổ chức theo các component độc lập, mỗi component đảm nhiệm một nhóm chức năng hoặc một phần giao diện cụ thể. Việc chia nhỏ giao diện thành các component giúp tăng khả năng tái sử dụng mã nguồn, giảm sự phụ thuộc giữa các thành phần và thuận lợi cho việc mở rộng hệ thống.

Trong hệ thống, React được sử dụng để xây dựng các giao diện như Dashboard, nhật ký ăn uống, theo dõi cân nặng, quản lý hoạt động luyện tập, xây dựng thực đơn, hồ sơ sức khỏe và chức năng quét thực phẩm.

React Router được sử dụng để quản lý điều hướng giữa các trang, kết hợp với cơ chế Protected Route nhằm kiểm soát quyền truy cập đối với các chức năng yêu cầu xác thực. Bên cạnh đó, TanStack Query (React Query) được sử dụng để quản lý dữ liệu lấy từ Backend, hỗ trợ lưu bộ nhớ đệm, đồng bộ dữ liệu sau khi cập nhật và giảm số lượng yêu cầu gửi đến máy chủ đối với các dữ liệu ít thay đổi. Cơ chế này đặc biệt phù hợp với các chức năng thường xuyên cập nhật như Dashboard, nhật ký ăn uống và theo dõi cân nặng.

1.3.2. Node.js và Express

Backend của hệ thống được xây dựng trên nền tảng Node.js kết hợp với Express.js. Node.js cung cấp môi trường thực thi JavaScript phía máy chủ, hỗ trợ mô hình xử lý bất đồng bộ thông qua cơ chế non-blocking I/O. Đặc điểm này phù hợp với hệ thống NutriTrack do backend cần xử lý nhiều loại tác vụ khác nhau như truy vấn cơ sở dữ liệu, gọi Open Food Facts API, xử lý ảnh thông qua Gemini AI Vision, tạo báo cáo PDF và trả dữ liệu cho giao diện người dùng thông qua API.

Express.js được sử dụng để xây dựng các RESTful API và tổ chức luồng xử lý request từ phía client. Trong hệ thống, các request được định tuyến qua route tương ứng, sau đó

chuyển đến controller để kiểm tra dữ liệu đầu vào, xác thực người dùng và gọi các service xử lý nghiệp vụ. Các thao tác với cơ sở dữ liệu được thực hiện thông qua Sequelize ORM, giúp ánh xạ dữ liệu giữa các model trong mã nguồn và các bảng trong MySQL.

Mã nguồn backend được tổ chức theo mô hình MVC kết hợp với tầng service. Trong đó, route chịu trách nhiệm định tuyến, controller tiếp nhận và điều phối yêu cầu, service chứa logic nghiệp vụ, còn model đại diện cho cấu trúc dữ liệu và quan hệ trong cơ sở dữ liệu. Cách phân tách này giúp giảm sự phụ thuộc giữa các thành phần, đồng thời thuận lợi cho quá trình kiểm thử, bảo trì và mở rộng chức năng.

Các nhóm chức năng như quản lý hồ sơ sức khỏe, nhật ký ăn uống, tính toán chỉ số dinh dưỡng, Adaptive TDEE, Food Scoring, Health Insights, xây dựng thực đơn, quét thực phẩm và xuất báo cáo được tách thành các module xử lý riêng trong backend. Nhờ đó, mỗi nhóm nghiệp vụ có phạm vi xử lý tương đối độc lập, trong khi vẫn có thể dùng chung các thành phần nền tảng như xác thực người dùng, chuẩn hóa phản hồi API, middleware xử lý lỗi và tầng truy cập cơ sở dữ liệu.

1.3.3. MySQL và Sequelize ORM

Hệ thống sử dụng MySQL làm hệ quản trị cơ sở dữ liệu để lưu trữ toàn bộ dữ liệu phát sinh trong quá trình sử dụng. Các nhóm dữ liệu chính bao gồm thông tin người dùng, hồ sơ sức khỏe, danh mục thực phẩm, nhật ký ăn uống, cân nặng, hoạt động luyện tập, dữ liệu quét sản phẩm và kết quả phân tích.

Để đơn giản hóa quá trình truy cập dữ liệu, đề tài sử dụng Sequelize ORM làm lớp trung gian giữa ứng dụng và cơ sở dữ liệu. Thay vì thao tác trực tiếp bằng các câu lệnh SQL, các bảng dữ liệu được biểu diễn dưới dạng các Model trong chương trình. Sequelize hỗ trợ thực hiện các thao tác thêm, sửa, xóa và truy vấn dữ liệu thông qua các phương thức được cung cấp sẵn.

Ngoài ra, Sequelize được sử dụng để thiết lập quan hệ giữa các bảng dữ liệu, quản lý Migration và Seeder, đồng thời hỗ trợ định nghĩa Association giữa các Model nhằm phục vụ các truy vấn dữ liệu có liên kết giữa nhiều bảng. Điều này góp phần đảm bảo tính nhất quán của cơ sở dữ liệu và thuận lợi cho việc mở rộng hệ thống.

Gợi ý chèn Hình 1.4. Kiến trúc truy cập dữ liệu của hệ thống

1.3.4. Các thư viện và dịch vụ hỗ trợ

Ngoài các công nghệ chính, hệ thống sử dụng một số thư viện và dịch vụ nhằm hỗ trợ triển khai các chức năng chuyên biệt.

JSON Web Token (JWT) được sử dụng để xác thực người dùng. Sau khi đăng nhập thành công, hệ thống phát hành **Access Token** để xác thực các yêu cầu gửi đến API và **Refresh Token** được lưu dưới dạng HttpOnly Cookie nhằm cập lại Access Token khi hết hạn mà không yêu cầu người dùng đăng nhập lại. Cơ chế này góp phần tăng tính an toàn

và cải thiện trải nghiệm sử dụng.

Axios được sử dụng tại Frontend để gửi các yêu cầu HTTP đến Backend và nhận dữ liệu trả về dưới định dạng JSON.

TanStack Query (React Query) được sử dụng để quản lý dữ liệu phía máy khách, hỗ trợ lưu bộ nhớ đệm, đồng bộ dữ liệu giữa các thành phần và tự động cập nhật lại dữ liệu sau các thao tác thêm, sửa hoặc xóa.

Đối với chức năng quét thực phẩm đóng gói, hệ thống tích hợp **Open Food Facts API** để tra cứu thông tin sản phẩm theo mã vạch khi dữ liệu chưa tồn tại trong cơ sở dữ liệu nội bộ. Dữ liệu sau khi được chuẩn hóa sẽ được lưu vào cơ sở dữ liệu của hệ thống nhằm giảm số lần truy vấn đối với các lần sử dụng tiếp theo.

Trong trường hợp sản phẩm không thể tra cứu bằng mã vạch hoặc chưa có dữ liệu trên Open Food Facts, hệ thống sử dụng **Gemini AI Vision** để nhận dạng bảng thành phần dinh dưỡng từ hình ảnh. Kết quả nhận dạng được xử lý và chuẩn hóa trước khi lưu vào cơ sở dữ liệu, từ đó phục vụ các chức năng tính toán và phân tích dinh dưỡng của hệ thống.

Ngoài ra, hệ thống còn sử dụng các thư viện hỗ trợ xây dựng biểu đồ thống kê, xuất báo cáo PDF, gửi thư điện tử và xử lý tập tin theo yêu cầu của từng chức năng.

Gợi ý chèn Bảng 1.2. Danh sách công nghệ sử dụng trong hệ thống

Công nghệ

Vai trò trong hệ thống

React

Xây dựng giao diện người dùng

React Router

Điều hướng và bảo vệ các trang

TanStack Query

Quản lý dữ liệu phía máy khách

Axios

Gửi yêu cầu HTTP

Node.js

Môi trường thực thi phía máy chủ

Express

Xây dựng RESTful API

Sequelize ORM

Tương tác với cơ sở dữ liệu

MySQL

Lưu trữ dữ liệu

JWT

Xác thực bằng Access Token và Refresh Token

Open Food Facts API

Tra cứu thông tin thực phẩm theo mã vạch

Gemini AI Vision

Nhận dạng bảng thành phần dinh dưỡng từ hình ảnh

1.3.5. Đánh giá việc lựa chọn công nghệ

Các công nghệ được lựa chọn đều phù hợp với kiến trúc Client–Server và đáp ứng yêu cầu của hệ thống quản lý dinh dưỡng. Việc tách biệt giữa giao diện người dùng, xử lý nghiệp vụ và lưu trữ dữ liệu giúp hệ thống thuận lợi trong quá trình phát triển, kiểm thử và bảo trì. Đồng thời, việc tích hợp các dịch vụ như Open Food Facts API, Gemini AI Vision và cơ chế xác thực bằng Access Token kết hợp Refresh Token góp phần mở rộng khả năng xử lý dữ liệu và nâng cao tính thực tiễn của hệ thống.

1.4. Kiến trúc tổng thể của hệ thống

Hệ thống quản lý dinh dưỡng được xây dựng theo kiến trúc **Client - Server** nhiều tầng, trong đó giao diện người dùng, xử lý nghiệp vụ, xác thực, truy cập dữ liệu và các dịch vụ hỗ trợ được tổ chức thành các thành phần độc lập. Các thành phần giao tiếp với nhau thông qua **RESTful API**, sử dụng định dạng **JSON** để trao đổi dữ liệu.

Kiến trúc này cho phép Frontend và Backend được phát triển độc lập, đồng thời tạo điều kiện thuận lợi cho việc mở rộng hệ thống, tích hợp các dịch vụ bên ngoài và bảo trì trong quá trình phát triển.

Trong mô hình này, người dùng thao tác trên giao diện được xây dựng bằng React thông qua trình duyệt web. Các yêu cầu như đăng nhập, cập nhật hồ sơ sức khỏe, ghi nhận nhật ký ăn uống, theo dõi cân nặng, xây dựng thực đơn hoặc quét thực phẩm được gửi đến Backend thông qua Axios dưới dạng các yêu cầu HTTP. Backend tiếp nhận yêu cầu, thực hiện xác thực người dùng, xử lý nghiệp vụ, truy xuất cơ sở dữ liệu hoặc gọi các dịch vụ bên ngoài khi cần thiết trước khi trả kết quả về cho Frontend dưới dạng dữ liệu JSON.

Gợi ý chèn Hình 1.5. Kiến trúc tổng thể của hệ thống

Hình minh họa: Người dùng → React Frontend → Axios → RESTful API → Router → Middleware → Controller → Service → Sequelize ORM → MySQL. Bên cạnh Backend thể hiện JWT Authentication, Open Food Facts API và Gemini AI Vision.

1.4.1. Mô hình kiến trúc Client - Server

Kiến trúc Client - Server được lựa chọn nhằm tách biệt phần giao diện người dùng và phần xử lý nghiệp vụ của hệ thống.

Phía Client chịu trách nhiệm hiển thị giao diện, tiếp nhận thao tác của người dùng và

gửi các yêu cầu đến Backend thông qua RESTful API bằng giao thức HTTP. Dữ liệu trao đổi giữa hai thành phần sử dụng định dạng JSON nhằm đảm bảo tính thống nhất trong quá trình truyền nhận và xử lý dữ liệu.

Phía Server chịu trách nhiệm xác thực người dùng, kiểm tra dữ liệu đầu vào, xử lý nghiệp vụ, truy cập cơ sở dữ liệu và tích hợp các dịch vụ bên ngoài. Sau khi hoàn thành xử lý, kết quả được trả về Frontend dưới dạng JSON để cập nhật giao diện mà không cần tải lại toàn bộ ứng dụng.

Việc phân tách giữa Client và Server giúp Frontend và Backend có thể được phát triển, kiểm thử và bảo trì độc lập, đồng thời tạo điều kiện thuận lợi cho việc mở rộng các chức năng của hệ thống trong tương lai.

1.4.2. Kiến trúc Frontend

Frontend được phát triển bằng React theo mô hình **Single Page Application (SPA)**. Giao diện được tổ chức thành các component độc lập và kết hợp thành các trang chức năng như Dashboard, hồ sơ sức khỏe, nhật ký ăn uống, theo dõi cân nặng, hoạt động luyện tập, Meal Planner, Scanner và báo cáo.

Việc tổ chức giao diện theo component giúp tăng khả năng tái sử dụng mã nguồn, giảm sự phụ thuộc giữa các thành phần và thuận lợi cho việc mở rộng hệ thống.

Điều hướng giữa các trang được quản lý bởi **React Router**. Các chức năng yêu cầu xác thực được bảo vệ thông qua **Protected Route**, kết hợp với **Auth Context** để quản lý trạng thái đăng nhập và thông tin người dùng trong suốt quá trình sử dụng hệ thống.

Dữ liệu hiển thị trên giao diện được quản lý bởi **TanStack Query (React Query)**. Thư viện này hỗ trợ lưu bộ nhớ đệm, đồng bộ dữ liệu giữa các thành phần và tự động cập nhật dữ liệu sau các thao tác thêm, sửa hoặc xóa. Cơ chế này giúp giảm số lượng yêu cầu gửi tới Backend đối với các dữ liệu ít thay đổi và duy trì tính nhất quán của dữ liệu trên giao diện.

Việc trao đổi dữ liệu giữa Frontend và Backend được thực hiện thông qua **Axios**, đóng vai trò gửi yêu cầu HTTP và nhận dữ liệu phản hồi từ các RESTful API.

Gợi ý chèn Hình 1.6. Kiến trúc Frontend

Hình minh họa: React Router → Pages → Components → Custom Hooks → Auth Context → TanStack Query → Axios.

1.4.3. Kiến trúc Backend

Backend được xây dựng trên nền tảng **Node.js** kết hợp với **Express Framework** và được tổ chức theo mô hình **MVC kết hợp với tầng Service**.

Mỗi yêu cầu từ Frontend được tiếp nhận thông qua **Router**, sau đó đi qua các **Middlewares** để thực hiện các chức năng như xác thực người dùng bằng JWT, kiểm tra dữ liệu đầu vào và xử lý các chức năng dùng chung trước khi chuyển đến Controller.

Controller chịu trách nhiệm tiếp nhận yêu cầu và điều phối luồng xử lý, trong khi toàn bộ nghiệp vụ của hệ thống được triển khai tại tầng **Service**. Các chức năng như tính toán BMI, BMR, TDEE, Adaptive TDEE, Food Scoring, Health Insights, Meal Planner, Scanner và xuất báo cáo đều được xử lý tại tầng này.

Sau khi hoàn thành xử lý nghiệp vụ, Service truy cập cơ sở dữ liệu thông qua các Model của Sequelize ORM hoặc tích hợp các dịch vụ bên ngoài như Open Food Facts API và Gemini AI Vision khi cần thiết. Kết quả sau cùng được trả về Controller để phản hồi cho Frontend dưới dạng JSON.

Việc tách riêng tầng Service giúp Controller chỉ đảm nhiệm vai trò điều phối yêu cầu, trong khi toàn bộ logic nghiệp vụ được quản lý tập trung, góp phần nâng cao khả năng bảo trì, kiểm thử và mở rộng hệ thống.

Gợi ý chèn Hình 1.7. Kiến trúc Backend theo mô hình MVC kết hợp Service

Hình minh họa: Request → Router → Middleware → Controller → Service → Model → MySQL → Response. Từ tầng Service kết nối tới Open Food Facts API và Gemini AI Vision.

1.4.4. Kiến trúc cơ sở dữ liệu

Hệ thống sử dụng **MySQL** làm hệ quản trị cơ sở dữ liệu để lưu trữ toàn bộ dữ liệu phát sinh trong quá trình sử dụng. Các bảng dữ liệu được xây dựng theo mô hình quan hệ và liên kết với nhau thông qua khóa chính, khóa ngoại nhằm đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

Cơ sở dữ liệu được chia thành các nhóm thông tin chính bao gồm:

- Dữ liệu người dùng và hồ sơ sức khỏe.
- Dữ liệu thực phẩm và sản phẩm.
- Nhật ký ăn uống.
- Lịch sử cân nặng.
- Hoạt động luyện tập.
- Dữ liệu phục vụ chức năng Scanner.
- Dữ liệu phục vụ Meal Planner và báo cáo.

Các bảng dữ liệu được ánh xạ thành các Model thông qua **Sequelize ORM**. Ngoài việc hỗ trợ các thao tác truy vấn dữ liệu, Sequelize còn được sử dụng để thiết lập Association giữa các Model, quản lý Migration và Seeder, qua đó đảm bảo sự đồng bộ giữa mã nguồn và cấu trúc cơ sở dữ liệu trong suốt quá trình phát triển.

Trong phạm vi chương này chỉ trình bày kiến trúc tổng quan của cơ sở dữ liệu. Thiết kế chi tiết các bảng dữ liệu, mối quan hệ giữa các thực thể và sơ đồ ERD sẽ được trình bày trong Chương 2.

Gợi ý chèn Hình 1.8. Kiến trúc tổng quan cơ sở dữ liệu

Chỉ trình bày các nhóm bảng chính. Sơ đồ ERD chi tiết được trình bày tại Chương 2 và Phụ lục.

1.4.5. Đánh giá kiến trúc hệ thống

Kiến trúc của hệ thống được xây dựng theo hướng phân tách rõ ràng giữa giao diện người dùng, xử lý nghiệp vụ và lưu trữ dữ liệu. Việc tổ chức Backend theo mô hình MVC kết hợp với tầng Service giúp tập trung toàn bộ logic nghiệp vụ tại một lớp xử lý riêng, trong khi Frontend được tổ chức theo các component độc lập và quản lý dữ liệu thông qua TanStack Query.

Kiến trúc này tạo điều kiện thuận lợi cho việc triển khai các chức năng như Adaptive TDEE, Food Scoring, Health Insights, Meal Planner và Hybrid Nutrition Scanner, đồng thời hỗ trợ mở rộng hệ thống hoặc tích hợp thêm các dịch vụ bên ngoài mà không ảnh hưởng đáng kể đến các thành phần còn lại.

1.5. Kết luận chương

Chương 1 đã trình bày tổng quan về bài toán quản lý dinh dưỡng cá nhân, bao gồm đặc điểm dữ liệu, các yêu cầu đặt ra đối với hệ thống, mục tiêu xây dựng và phạm vi chức năng cần triển khai. Nội dung chương cho thấy hệ thống không chỉ thực hiện lưu trữ nhật ký ăn uống mà còn cần kết hợp các chức năng tính toán, phân tích và hỗ trợ nhập liệu để phục vụ quá trình theo dõi dinh dưỡng của người dùng.

Các công nghệ chính được sử dụng trong hệ thống gồm React, Node.js, Express, MySQL và Sequelize ORM. Ngoài ra, hệ thống còn sử dụng một số thư viện, cơ chế xác thực và dịch vụ bên ngoài như JWT, TanStack Query, Open Food Facts API và Gemini AI Vision nhằm phục vụ các yêu cầu về xác thực người dùng, quản lý dữ liệu phía giao diện, tra cứu thực phẩm và nhận dạng bảng thành phần dinh dưỡng.

Về kiến trúc, hệ thống được xây dựng theo mô hình Client - Server. Frontend đảm nhiệm vai trò hiển thị giao diện và tiếp nhận thao tác của người dùng, Backend xử lý nghiệp vụ và cung cấp RESTful API, còn cơ sở dữ liệu MySQL lưu trữ dữ liệu phát sinh trong quá trình sử dụng. Các thành phần này được tổ chức theo hướng phân tách trách nhiệm, trong đó Backend được triển khai theo mô hình MVC kết hợp tầng Service và truy cập dữ liệu thông qua Sequelize ORM.

Trên cơ sở tổng quan đã trình bày, Chương 2 sẽ tập trung phân tích yêu cầu, mô tả nghiệp vụ, thiết kế chức năng, thiết kế cơ sở dữ liệu, API và giao diện của hệ thống.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích yêu cầu và thiết kế hệ thống quản lý dinh dưỡng cá nhân. Nội dung chương bao gồm phân tích yêu cầu chức năng, yêu cầu phi chức năng, mô tả nghiệp vụ, thiết kế chức năng, thiết kế cơ sở dữ liệu, thiết kế API và thiết kế giao diện. Đây là cơ sở để triển khai hệ thống trong Chương 3.

2.1. Phân tích yêu cầu

Phân tích yêu cầu là bước xác định các chức năng và ràng buộc mà hệ thống cần đáp ứng trước khi tiến hành thiết kế chi tiết. Đối với hệ thống quản lý dinh dưỡng cá nhân, yêu cầu của hệ thống không chỉ dừng ở việc lưu trữ dữ liệu người dùng mà còn bao gồm các xử lý tính toán, phân tích và hỗ trợ nhập liệu.

Dữ liệu trong hệ thống được hình thành từ nhiều nguồn như hồ sơ sức khỏe, nhật ký ăn uống, lượng nước uống, cân nặng, luyện tập và dữ liệu thực phẩm. Các nhóm dữ liệu này có quan hệ với nhau trong quá trình tính toán chỉ số cá nhân, tổng hợp dinh dưỡng, xây dựng thực đơn, phân tích tình trạng sức khỏe và xuất báo cáo. Vì vậy, yêu cầu hệ thống được chia thành hai nhóm chính: yêu cầu chức năng và yêu cầu phi chức năng.

2.1.1. Yêu cầu chức năng

Yêu cầu chức năng mô tả các chức năng mà hệ thống cần cung cấp cho người dùng trong quá trình sử dụng. Các yêu cầu này được xác định dựa trên mục tiêu của đề tài và các nghiệp vụ chính của hệ thống.

Trước hết, hệ thống cần hỗ trợ người dùng đăng ký, đăng nhập, đăng xuất và duy trì trạng thái xác thực. Các chức năng liên quan đến dữ liệu cá nhân cần được truy cập sau khi người dùng đăng nhập hợp lệ. Sau khi đăng nhập, hệ thống cần xác định người dùng hiện tại để truy xuất dữ liệu tương ứng với từng tài khoản.

Hệ thống cần cho phép người dùng khởi tạo và cập nhật hồ sơ sức khỏe. Các thông tin như giới tính, tuổi hoặc ngày sinh, chiều cao, cân nặng, mức độ vận động và mục tiêu cá nhân được sử dụng để tính toán các chỉ số như BMI, BMR, TDEE, mục tiêu năng lượng, tỷ lệ chất đa lượng và mục tiêu nước.

Đối với quá trình theo dõi hằng ngày, hệ thống cần hỗ trợ người dùng ghi nhật ký ăn uống theo từng bữa, ghi nhận lượng nước uống, cập nhật cân nặng và ghi nhận hoạt động luyện tập. Dữ liệu nhật ký ăn uống cần lưu lại giá trị dinh dưỡng tại thời điểm ghi nhận để tránh việc thay đổi dữ liệu thực phẩm gốc làm sai lệch lịch sử theo dõi.

Hệ thống cũng cần cung cấp chức năng tìm kiếm và quản lý dữ liệu thực phẩm. Dữ liệu thực phẩm là cơ sở để ghi nhật ký ăn uống, tính toán dinh dưỡng, đánh giá chất lượng thực phẩm và xây dựng thực đơn. Đối với thực phẩm đóng gói, hệ thống cần hỗ trợ nhập liệu bằng Hybrid Nutrition Scanner thông qua mã vạch hoặc ảnh bảng thành phần dinh dưỡng.

Bên cạnh các chức năng ghi nhận dữ liệu, hệ thống cần thực hiện các xử lý tính toán và phân tích như Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner. Các kết quả xử lý được sử dụng để hiển thị trên Dashboard, hỗ trợ xây dựng thực đơn và xuất báo cáo PDF theo khoảng thời gian.

Bảng 2.x. Tổng hợp yêu cầu chức năng của hệ thống

Mã yêu cầu

Nhóm chức năng

Mô tả yêu cầu

FR-01

Quản lý tài khoản

Đăng ký, đăng nhập, đăng xuất và duy trì trạng thái xác thực

FR-02

Hồ sơ sức khỏe

Khởi tạo và cập nhật thông tin cá nhân phục vụ tính toán dinh dưỡng

FR-03

Nhật ký ăn uống

Ghi nhận thực phẩm theo bữa và lưu giá trị dinh dưỡng tại thời điểm nhập liệu

FR-04

Nước uống

Ghi nhận lượng nước uống trong ngày và theo dõi tiến độ so với mục tiêu

FR-05

Cân nặng và luyện tập

Theo dõi cân nặng, hoạt động vận động và năng lượng tiêu hao

FR-06

Thực phẩm

Tìm kiếm, quản lý và bổ sung dữ liệu thực phẩm

FR-07

Tính toán và phân tích

Tính BMI, BMR, TDEE, Adaptive TDEE, Food Scoring và Health Insights

FR-08

Meal Planner

Xây dựng thực đơn gợi ý dựa trên mục tiêu dinh dưỡng

FR-09

Hybrid Nutrition Scanner

Tra cứu mã vạch và nhận dạng bảng thành phần dinh dưỡng từ hình ảnh

FR-10

Dashboard và báo cáo

Hiển thị dữ liệu tổng hợp, biểu đồ, nhận xét và xuất báo cáo PDF

2.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng mô tả các tiêu chí về chất lượng, bảo mật, khả năng sử dụng, hiệu năng và khả năng mở rộng của hệ thống.

Về bảo mật, hệ thống cần kiểm soát quyền truy cập đối với các API liên quan đến dữ liệu cá nhân. Người dùng sau khi đăng nhập sử dụng Access Token để truy cập các API được bảo vệ. Refresh Token được sử dụng để hỗ trợ làm mới Access Token khi cần, giúp duy trì phiên đăng nhập trong quá trình sử dụng.

Về tính nhất quán dữ liệu, các dữ liệu phát sinh như nhật ký ăn uống, nước uống, cân nặng, luyện tập và log Adaptive TDEE cần được gắn với từng người dùng cụ thể. Cơ sở dữ liệu cần sử dụng khóa chính, khóa ngoại và các ràng buộc cần thiết để bảo đảm quan hệ giữa các bảng. Dữ liệu từ nguồn bên ngoài hoặc từ AI Vision cần được chuẩn hóa trước khi lưu trữ.

Về khả năng sử dụng, giao diện cần tổ chức các chức năng theo luồng sử dụng rõ ràng. Các thao tác thường xuyên như thêm thực phẩm, ghi nhận nước uống, cập nhật cân nặng hoặc xem Dashboard cần được thực hiện với số bước hợp lý. Hệ thống cần hiển thị rõ trạng thái đang tải, thành công, lỗi hoặc không có dữ liệu.

Về hiệu năng, các chức năng thường xuyên sử dụng như Dashboard, nhật ký ăn uống, tìm kiếm thực phẩm và cập nhật dữ liệu hàng ngày cần có thời gian phản hồi phù hợp với quy mô dữ liệu của hệ thống. Với các chức năng phụ thuộc dịch vụ bên ngoài như Open Food Facts hoặc AI Vision, hệ thống cần có cơ chế xử lý khi không tìm thấy dữ liệu, dữ liệu không hợp lệ hoặc vượt quá giới hạn tần suất gọi.

Về khả năng mở rộng và bảo trì, hệ thống cần được tổ chức theo kiến trúc phân lớp, tách biệt Frontend, Backend và cơ sở dữ liệu. Backend cần phân tách Router, Middleware, Controller, Service và Model. Frontend cần tổ chức theo trang, component và custom hook để thuận lợi cho việc tái sử dụng và mở rộng chức năng.

Bảng 2.x. Tổng hợp yêu cầu phi chức năng của hệ thống

Mã yêu cầu

Nhóm yêu cầu

Nội dung

NFR-01

Bảo mật

Kiểm soát quyền truy cập API bằng Access Token và hỗ trợ duy trì phiên bằng Refresh Token

NFR-02

Nhất quán dữ liệu

Dữ liệu được gắn với người dùng, có ràng buộc quan hệ và lưu snapshot dinh dưỡng khi cần

NFR-03

Khả năng sử dụng

Giao diện rõ ràng, hỗ trợ thao tác nhập liệu, hiển thị trạng thái xử lý và thông báo lỗi

NFR-04

Hiệu năng

Đảm bảo thời gian phản hồi phù hợp cho các chức năng thường xuyên sử dụng

NFR-05

Khả năng mở rộng

Kiến trúc phân lớp, tách biệt Frontend, Backend, Service, Model và cơ sở dữ liệu

NFR-06

Tích hợp bên ngoài

Xử lý và chuẩn hóa dữ liệu từ Open Food Facts API và Gemini AI Vision

NFR-07

Bảo trì

Mã nguồn được tổ chức theo module, component và service để thuận lợi cho kiểm thử và mở rộng

2.1.3. Tổng kết yêu cầu

Các yêu cầu trên cho thấy hệ thống cần kết hợp giữa quản lý dữ liệu cá nhân, ghi nhận dữ liệu hằng ngày, xử lý tính toán và hỗ trợ phân tích dinh dưỡng. Bên cạnh các chức năng cơ bản như tài khoản, hồ sơ sức khỏe, nhật ký ăn uống, nước uống, cân nặng và luyện tập, hệ thống còn cần triển khai các chức năng đặc thù như Adaptive TDEE, Food Scoring, Health Insights, Meal Planner và Hybrid Nutrition Scanner.

Kết quả phân tích yêu cầu là cơ sở để xác định các luồng nghiệp vụ, Use Case, mô hình dữ liệu, API và giao diện trong các mục tiếp theo.

2.2. Phân tích nghiệp vụ

Phân tích nghiệp vụ nhằm mô tả các quy trình chính mà hệ thống cần hỗ trợ trong quá trình người dùng sử dụng. Đối với hệ thống quản lý dinh dưỡng cá nhân, các nghiệp vụ được hình thành xoay quanh quá trình người dùng đăng ký tài khoản, khởi tạo hồ sơ sức khỏe, ghi nhận dữ liệu hằng ngày, xem kết quả phân tích và sử dụng các chức năng hỗ trợ như Scanner, Meal Planner và báo cáo PDF.

Các nghiệp vụ trong hệ thống có mối liên hệ chặt chẽ với nhau thông qua dữ liệu người dùng. Hồ sơ sức khỏe là cơ sở để tính toán các chỉ số ban đầu như BMI, BMR, TDEE, mục tiêu năng lượng, mục tiêu nước và tỷ lệ chất đa lượng. Nhật ký ăn uống,

nước uống, cân nặng và luyện tập là các dữ liệu phát sinh trong quá trình sử dụng. Các dữ liệu này được hệ thống tổng hợp để hiển thị trên Dashboard, phục vụ báo cáo PDF và làm đầu vào cho một số thuật toán phân tích.

Trong phạm vi đề tài, các nghiệp vụ chính của hệ thống được tổng hợp trong bảng sau.

Bảng 2.x. Tổng hợp các luồng nghiệp vụ chính của hệ thống

Nghiệp vụ

Dữ liệu đầu vào

Xử lý chính

Kết quả đầu ra

Đăng ký, đăng nhập

Họ tên, email, mật khẩu

Kiểm tra tài khoản, xác thực thông tin, cấp Access Token và Refresh Token

Người dùng được xác thực và truy cập hệ thống

Khởi tạo hồ sơ sức khỏe

Giới tính, tuổi, chiều cao, cân nặng, mức độ vận động, mục tiêu

Lưu hồ sơ, tính BMI, BMR, TDEE, mục tiêu năng lượng, macro và nước

Hồ sơ sức khỏe được khởi tạo, người dùng được chuyển vào hệ thống chính

Ghi nhật ký ăn uống

Thực phẩm, khẩu phần, bữa ăn, ngày ghi nhận

Tính dinh dưỡng theo khẩu phần, lưu bản ghi nhật ký và dữ liệu snapshot

Nhật ký ăn uống và tổng dinh dưỡng trong ngày được cập nhật

Ghi nhận nước uống

Lượng nước uống, ngày ghi nhận

Lưu dữ liệu nước uống, tổng hợp theo ngày

Tiến độ nước uống được cập nhật trên Dashboard

Theo dõi cân nặng

Cân nặng, ngày ghi nhận

Lưu lịch sử cân nặng, cập nhật biểu đồ và xu hướng

Dữ liệu cân nặng được dùng cho theo dõi và Adaptive TDEE

Ghi nhận luyện tập

Hoạt động, thời lượng hoặc năng lượng tiêu hao

Lưu dữ liệu luyện tập, tổng hợp năng lượng tiêu hao

Dữ liệu vận động được hiển thị trên Dashboard và báo cáo

Hybrid Nutrition Scanner

Mã vạch hoặc ảnh bảng thành phần dinh dưỡng

Tra cứu dữ liệu nội bộ, gọi Open Food Facts hoặc AI Vision, chuẩn hóa kết quả
Thông tin sản phẩm được trả về để ghi nhật ký hoặc đóng góp dữ liệu
Xem Dashboard

Ngày xem, dữ liệu người dùng

Tổng hợp hồ sơ, nhật ký, nước uống, cân nặng, luyện tập và phân tích

Các chỉ số tổng hợp và nhận xét được hiển thị

Xây dựng thực đơn

Mục tiêu dinh dưỡng, bữa ăn, mẫu bữa, dữ liệu thực phẩm

Tính toán khẩu phần thực phẩm theo mục tiêu

Thực đơn gợi ý được tạo và có thể dùng cho nhật ký ăn uống

Xuất báo cáo PDF

Khoảng thời gian cần báo cáo

Tổng hợp dữ liệu hồ sơ, nhật ký, cân nặng, luyện tập, nước uống và phân tích

File PDF báo cáo được tạo

2.2.1. Nghiệp vụ đăng nhập và khởi tạo hồ sơ sức khỏe

Nghiệp vụ đăng nhập là bước đầu để người dùng truy cập hệ thống. Người dùng nhập email và mật khẩu trên giao diện. Frontend gửi dữ liệu tới Backend để kiểm tra thông tin tài khoản. Nếu dữ liệu hợp lệ, hệ thống cấp Access Token để xác thực các yêu cầu API và thiết lập Refresh Token dưới dạng HttpOnly Cookie nhằm hỗ trợ làm mới phiên đăng nhập khi Access Token hết hạn.

Sau khi đăng nhập thành công, hệ thống kiểm tra trạng thái khởi tạo hồ sơ của người dùng. Nếu người dùng chưa hoàn tất hồ sơ sức khỏe ban đầu, giao diện sẽ điều hướng tới trang Onboarding. Tại đây, người dùng khai báo các thông tin như giới tính, tuổi hoặc ngày sinh, chiều cao, cân nặng, mức độ vận động và mục tiêu cá nhân.

Dữ liệu hồ sơ được Backend kiểm tra và lưu vào cơ sở dữ liệu. Từ các thông tin này, hệ thống tính toán các chỉ số nền tảng như BMI, BMR, TDEE, mục tiêu năng lượng, tỷ lệ chất đa lượng và mục tiêu nước. Sau khi hoàn tất Onboarding, người dùng được chuyển tới các màn hình chức năng chính như Dashboard, nhật ký ăn uống, Meal Planner, cân nặng, luyện tập và hồ sơ cá nhân.

2.2.2. Nghiệp vụ ghi nhật ký ăn uống và nước uống

Ghi nhật ký ăn uống là một trong các nghiệp vụ trung tâm của hệ thống. Người dùng lựa chọn thực phẩm, nhập khẩu phần sử dụng, chọn bữa ăn và xác nhận thêm vào nhật ký. Backend tính toán lượng dinh dưỡng thực tế dựa trên khẩu phần đã nhập, sau đó lưu dữ liệu vào cơ sở dữ liệu.

Mỗi bản ghi nhật ký cần lưu lại thông tin thực phẩm, khẩu phần, bữa ăn, ngày ghi nhận và các giá trị dinh dưỡng tại thời điểm nhập liệu. Việc lưu dữ liệu dinh dưỡng

snapshot giúp lịch sử ăn uống không bị thay đổi khi thông tin thực phẩm gốc được chỉnh sửa sau đó. Dữ liệu nhật ký ăn uống được sử dụng để tổng hợp năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri và một số vi chất phục vụ Dashboard, báo cáo và các thuật toán phân tích.

Bên cạnh nhật ký ăn uống, hệ thống hỗ trợ người dùng ghi nhận lượng nước uống trong ngày. Người dùng nhập lượng nước đã uống, hệ thống lưu dữ liệu và cập nhật tổng lượng nước theo ngày. Kết quả được so sánh với mục tiêu nước khuyến nghị để hiển thị tiến độ trên Dashboard.

2.2.3. Nghiệp vụ sử dụng Hybrid Nutrition Scanner

Hybrid Nutrition Scanner được thiết kế nhằm hỗ trợ nhập liệu đối với thực phẩm đóng gói. Chức năng này giúp giảm thao tác nhập thủ công khi người dùng muốn ghi nhận sản phẩm chưa có sẵn trong cơ sở dữ liệu thực phẩm.

Đối với luồng quét mã vạch, người dùng quét hoặc nhập mã vạch sản phẩm. Hệ thống ưu tiên tra cứu trong cơ sở dữ liệu nội bộ, bao gồm dữ liệu đã xác thực và dữ liệu chưa xác thực. Nếu không tìm thấy dữ liệu phù hợp, Backend tiếp tục tra cứu từ Open Food Facts API. Dữ liệu nhận được cần được chuẩn hóa trước khi trả về giao diện hoặc lưu vào cơ sở dữ liệu.

Đối với luồng nhận dạng bằng thành phần dinh dưỡng, người dùng gửi ảnh bằng thông tin sản phẩm. Hệ thống sử dụng AI Vision để trích xuất các giá trị như năng lượng, protein, carbohydrate, chất béo, đường và natri. Kết quả nhận dạng cần được kiểm tra và chuẩn hóa trước khi sử dụng. Sau khi có kết quả, người dùng có thể dùng dữ liệu sản phẩm để ghi vào nhật ký ăn uống hoặc đóng góp vào cơ sở dữ liệu sản phẩm.

2.2.4. Nghiệp vụ xây dựng thực đơn

Nghiệp vụ xây dựng thực đơn hỗ trợ người dùng tạo gợi ý bữa ăn dựa trên mục tiêu dinh dưỡng. Dữ liệu đầu vào gồm mục tiêu năng lượng, mục tiêu chất đa lượng, bữa ăn cần xây dựng, mẫu bữa ăn và danh sách thực phẩm phù hợp.

Người dùng có thể chọn bữa ăn, chọn mẫu thực đơn, ghim nguyên liệu hoặc thay thế nguyên liệu trong quá trình tạo gợi ý. Backend sử dụng thuật toán Meal Planner để tính toán khối lượng thực phẩm theo mục tiêu dinh dưỡng. Kết quả trả về là thực đơn gợi ý gồm các nguyên liệu, khối lượng và giá trị dinh dưỡng tương ứng.

Sau khi hệ thống sinh kết quả, người dùng có thể kiểm tra thực đơn, thay thế nguyên liệu nếu cần và sử dụng kết quả để thêm vào nhật ký ăn uống. Do nghiệp vụ này liên quan trực tiếp đến thuật toán tính toán khẩu phần thực phẩm, phần triển khai chi tiết sẽ được trình bày trong Chương 3.

2.2.5. Tổng kết nghiệp vụ

Các nghiệp vụ của hệ thống được tổ chức xoay quanh dữ liệu người dùng và dữ liệu dinh dưỡng theo thời gian. Hồ sơ sức khỏe cung cấp dữ liệu nền tảng cho các phép tính cá nhân. Nhật ký ăn uống, nước uống, cân nặng và luyện tập tạo thành dữ liệu phát sinh trong quá trình sử dụng. Hybrid Nutrition Scanner hỗ trợ bổ sung dữ liệu thực phẩm, trong khi Meal Planner sử dụng mục tiêu dinh dưỡng và dữ liệu thực phẩm để tạo gợi ý thực đơn.

Kết quả phân tích nghiệp vụ là cơ sở để xác định các Use Case, các nhóm dữ liệu chính, các API cần thiết và các màn hình giao diện trong các mục thiết kế tiếp theo.

2.3. Thiết kế chức năng

Thiết kế chức năng được thực hiện nhằm xác định các tác nhân, các nhóm chức năng chính và mối quan hệ giữa người dùng với hệ thống. Dựa trên kết quả phân tích yêu cầu và phân tích nghiệp vụ ở các mục trước, hệ thống quản lý dinh dưỡng cá nhân được thiết kế với tác nhân nghiệp vụ chính là người dùng cá nhân.

Trong phạm vi đồ án, hệ thống không phân tách nhiều vai trò như quản trị viên, chuyên gia dinh dưỡng hoặc bác sĩ. Các chức năng được xây dựng chủ yếu phục vụ người dùng cuối trong quá trình quản lý hồ sơ sức khỏe, ghi nhận dữ liệu hằng ngày, theo dõi các chỉ số cá nhân và xem kết quả phân tích.

2.3.1. Tác nhân và hệ thống liên quan

Tác nhân chính của hệ thống là **Người dùng**. Người dùng tương tác trực tiếp với hệ thống thông qua giao diện web để thực hiện các nghiệp vụ như đăng ký, đăng nhập, khởi tạo hoặc cập nhật hồ sơ sức khỏe, ghi nhật ký ăn uống, ghi nhận nước uống, theo dõi cân nặng, ghi nhận luyện tập, sử dụng Scanner, xem Dashboard, xây dựng thực đơn và xuất báo cáo.

Ngoài tác nhân người dùng, một số chức năng của hệ thống có sử dụng các hệ thống bên ngoài như Open Food Facts API và Gemini AI Vision. Các hệ thống này không phải là tác nhân nghiệp vụ chính vì người dùng không tương tác trực tiếp với chúng. Chúng được Backend gọi trong quá trình xử lý nghiệp vụ nhằm hỗ trợ tra cứu và nhận dạng dữ liệu thực phẩm.

Bảng 2.x. Tác nhân và hệ thống liên quan

Thành phần

Loại

Vai trò

Người dùng

Tác nhân chính

Sử dụng các chức năng quản lý dinh dưỡng cá nhân

Open Food Facts API

Hệ thống bên ngoài

Cung cấp dữ liệu sản phẩm theo mã vạch khi cơ sở dữ liệu nội bộ chưa có dữ liệu phù hợp

Gemini AI Vision

Hệ thống bên ngoài

Hỗ trợ nhận dạng bảng thành phần dinh dưỡng từ hình ảnh

2.3.2. Biểu đồ Use Case tổng thể

Biểu đồ Use Case tổng thể thể hiện các chức năng chính mà người dùng có thể thực hiện trong hệ thống. Các Use Case được chia thành các nhóm chức năng tương ứng với các nghiệp vụ đã phân tích, bao gồm xác thực người dùng, quản lý hồ sơ sức khỏe, ghi nhận dữ liệu hàng ngày, theo dõi chỉ số cá nhân, sử dụng Scanner, xây dựng thực đơn, xem Dashboard và xuất báo cáo.

Gợi ý chèn Hình 2.x. Biểu đồ Use Case tổng thể của hệ thống

Biểu đồ nên thể hiện tác nhân Người dùng ở bên ngoài biên hệ thống. Bên trong hệ thống gồm các nhóm Use Case chính. Open Food Facts API và Gemini AI Vision có thể được đặt ngoài hệ thống và liên kết với nhóm Use Case Hybrid Nutrition Scanner.

2.3.3. Danh sách Use Case chính

Bảng dưới đây tổng hợp các Use Case chính của hệ thống. Các Use Case này được lựa chọn dựa trên những chức năng có vai trò trực tiếp trong quá trình sử dụng của người dùng và được triển khai trong hệ thống.

Bảng 2.x. Danh sách Use Case chính

Mã Use Case

Tên Use Case

Tác nhân

Mô tả

UC01

Đăng ký tài khoản

Người dùng

Tạo tài khoản mới để sử dụng hệ thống

UC02

Đăng nhập

Người dùng

Xác thực tài khoản và truy cập hệ thống

UC03

Khởi tạo và cập nhật hồ sơ sức khỏe

Người dùng

Khai báo hoặc chỉnh sửa thông tin phục vụ tính toán dinh dưỡng

UC04

Ghi nhật ký ăn uống

Người dùng

Thêm thực phẩm đã sử dụng vào nhật ký theo từng bữa

UC05

Ghi nhận nước uống

Người dùng

Ghi nhận lượng nước uống trong ngày và theo dõi tiến độ so với mục tiêu

UC06

Tìm kiếm thực phẩm

Người dùng

Tìm kiếm thực phẩm trong cơ sở dữ liệu để ghi nhật ký hoặc xây dựng thực đơn

UC07

Theo dõi cân nặng

Người dùng

Ghi nhận cân nặng theo thời gian

UC08

Ghi nhận luyện tập

Người dùng

Lưu thông tin hoạt động vận động và năng lượng tiêu hao

UC09

Xem Dashboard

Người dùng

Xem các chỉ số tổng hợp, biểu đồ và nhận xét phân tích

UC10

Xây dựng thực đơn

Người dùng

Nhận gợi ý thực đơn dựa trên mục tiêu dinh dưỡng

UC11

Quét mã vạch sản phẩm

Người dùng

Tra cứu thông tin sản phẩm đóng gói theo mã vạch

UC12

Nhận dạng bảng thành phần dinh dưỡng

Người dùng

Trích xuất thông tin dinh dưỡng từ ảnh bảng thành phần

UC13

Xuất báo cáo PDF

Người dùng

Tạo báo cáo tổng hợp dữ liệu dinh dưỡng theo khoảng thời gian

2.3.4. Mô tả một số Use Case tiêu biểu

Trong phần nội dung chính của báo cáo, chỉ một số Use Case tiêu biểu được mô tả để làm rõ cách hệ thống xử lý các nghiệp vụ quan trọng. Các Use Case còn lại được trình bày chi tiết hơn trong Phụ lục A.

a. Use Case đăng nhập

Use Case đăng nhập cho phép người dùng xác thực tài khoản trước khi truy cập các chức năng cần quyền sử dụng. Người dùng nhập email và mật khẩu trên giao diện đăng nhập. Frontend gửi dữ liệu tới Backend để kiểm tra thông tin tài khoản.

Nếu thông tin hợp lệ, Backend trả về Access Token để Frontend sử dụng trong các yêu cầu API cần xác thực, đồng thời thiết lập Refresh Token dưới dạng HttpOnly Cookie nhằm hỗ trợ làm mới Access Token khi hết hạn. Sau khi đăng nhập thành công, Frontend kiểm tra trạng thái khởi tạo hồ sơ của người dùng. Nếu người dùng chưa hoàn tất hồ sơ sức khỏe ban đầu, hệ thống điều hướng tới trang Onboarding; ngược lại, người dùng được truy cập các chức năng chính.

Nếu thông tin đăng nhập không hợp lệ, hệ thống trả về thông báo lỗi để người dùng kiểm tra lại dữ liệu đã nhập.

Bảng 2.x. Đặc tả rút gọn Use Case đăng nhập

Thành phần

Nội dung

Tên Use Case

Đăng nhập

Tác nhân

Người dùng

Tiền điều kiện

Người dùng đã có tài khoản

Luồng chính

Nhập thông tin đăng nhập, gửi yêu cầu, xác thực tài khoản, cấp Access Token và Refresh Token, điều hướng theo trạng thái hồ sơ

Luồng thay thế

Email hoặc mật khẩu không hợp lệ, hệ thống trả thông báo lỗi

Hậu điều kiện

Người dùng được xác thực và có thể truy cập các chức năng phù hợp

b. Use Case ghi nhật ký ăn uống

Use Case ghi nhật ký ăn uống cho phép người dùng thêm thực phẩm đã sử dụng vào nhật ký theo từng bữa trong ngày. Người dùng tìm kiếm thực phẩm, nhập khẩu phần và chọn bữa ăn tương ứng. Sau khi xác nhận, hệ thống tính toán lượng dinh dưỡng dựa trên khẩu phần đã nhập và lưu dữ liệu vào nhật ký.

Dữ liệu nhật ký ăn uống được sử dụng để tổng hợp năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri và một số vi chất. Kết quả tổng hợp được hiển thị trên Dashboard và được sử dụng trong các thuật toán phân tích tình trạng dinh dưỡng.

Một điểm quan trọng trong Use Case này là hệ thống cần lưu lại giá trị dinh dưỡng tại thời điểm ghi nhật ký. Cách xử lý này giúp dữ liệu lịch sử không bị thay đổi khi thông tin thực phẩm gốc được chỉnh sửa sau đó.

Bảng 2.x. Đặc tả rút gọn Use Case ghi nhật ký ăn uống

Thành phần

Nội dung

Tên Use Case

Ghi nhật ký ăn uống

Tác nhân

Người dùng

Tiền điều kiện

Người dùng đã đăng nhập và có dữ liệu thực phẩm phù hợp

Luồng chính

Chọn thực phẩm, nhập khẩu phần, chọn bữa ăn, xác nhận thêm vào nhật ký, hệ thống tính toán và lưu dữ liệu

Luồng thay thế

Không tìm thấy thực phẩm phù hợp, người dùng tìm kiếm lại hoặc sử dụng Scanner

Hậu điều kiện

Nhật ký ăn uống được cập nhật và tổng dinh dưỡng trong ngày được tính lại

c. Use Case sử dụng Hybrid Nutrition Scanner

Use Case sử dụng Hybrid Nutrition Scanner hỗ trợ người dùng nhập dữ liệu đối với thực phẩm đóng gói. Người dùng có thể quét mã vạch sản phẩm hoặc chụp ảnh bảng thành phần dinh dưỡng.

Với luồng quét mã vạch, hệ thống ưu tiên tìm kiếm trong cơ sở dữ liệu nội bộ, bao gồm dữ liệu đã xác thực và dữ liệu chưa xác thực. Nếu không có dữ liệu phù hợp, Backend tra cứu từ Open Food Facts API và chuẩn hóa kết quả trước khi trả về giao diện hoặc lưu

vào cơ sở dữ liệu.

Với luồng nhận dạng bằng thành phần dinh dưỡng, người dùng gửi ảnh bằng thông tin sản phẩm. Hệ thống sử dụng Gemini AI Vision để trích xuất dữ liệu dinh dưỡng, sau đó kiểm tra và chuẩn hóa kết quả trước khi sử dụng.

Use Case này giúp giảm thao tác nhập liệu thủ công trong trường hợp người dùng cần ghi nhận thực phẩm đóng gói chưa có sẵn trong cơ sở dữ liệu nội bộ.

Bảng 2.x. Đặc tả rút gọn Use Case sử dụng Hybrid Nutrition Scanner

Thành phần

Nội dung

Tên Use Case

Sử dụng Hybrid Nutrition Scanner

Tác nhân

Người dùng

Hệ thống liên quan

Open Food Facts API, Gemini AI Vision

Tiền điều kiện

Người dùng đã đăng nhập

Luồng chính

Quét mã vạch hoặc gửi ảnh, hệ thống tìm kiếm dữ liệu nội bộ, gọi dịch vụ bên ngoài nếu cần, chuẩn hóa kết quả và trả về giao diện

Luồng thay thế

Không tìm thấy sản phẩm hoặc ảnh không đủ dữ liệu, hệ thống thông báo để người dùng nhập lại hoặc bổ sung thủ công

Hậu điều kiện

Dữ liệu sản phẩm được trả về, lưu trữ hoặc sử dụng để ghi nhật ký

2.3.5. Quan hệ giữa các Use Case

Một số Use Case trong hệ thống có quan hệ phụ thuộc với nhau. Người dùng cần đăng nhập trước khi khởi tạo hoặc cập nhật hồ sơ sức khỏe, ghi nhật ký ăn uống, ghi nhận nước uống, theo dõi cân nặng, ghi nhận luyện tập, sử dụng Scanner, xem Dashboard, xây dựng thực đơn hoặc xuất báo cáo.

Hồ sơ sức khỏe cần được khởi tạo trước khi hệ thống có thể tính toán các chỉ số dinh dưỡng cá nhân như BMI, BMR, TDEE, mục tiêu năng lượng, tỷ lệ chất đa lượng và lượng nước khuyến nghị. Các dữ liệu này được sử dụng trong Dashboard, Health Insights, Meal Planner và báo cáo.

Use Case ghi nhật ký ăn uống phụ thuộc vào dữ liệu thực phẩm. Dữ liệu thực phẩm có thể được lấy từ cơ sở dữ liệu nội bộ, từ kết quả quét mã vạch hoặc từ kết quả nhận

dạng bảng thành phần dinh dưỡng. Use Case Meal Planner phụ thuộc vào hồ sơ sức khỏe, mục tiêu dinh dưỡng, cấu hình bữa ăn, mẫu thực đơn và dữ liệu thực phẩm. Use Case Dashboard và xuất báo cáo phụ thuộc vào dữ liệu đã được ghi nhận trong các Use Case nhật ký ăn uống, nước uống, cân nặng và luyện tập.

Các quan hệ này cần được thể hiện trong biểu đồ Use Case tổng thể và tiếp tục được làm rõ trong phần thiết kế cơ sở dữ liệu, thiết kế API và thiết kế giao diện.

2.3.6. Tổng kết thiết kế chức năng

Thiết kế Use Case cho thấy hệ thống tập trung vào một tác nhân nghiệp vụ chính là người dùng cá nhân. Các chức năng được tổ chức theo những nhóm nghiệp vụ đã phân tích ở mục 2.2, bao gồm xác thực, hồ sơ sức khỏe, nhật ký ăn uống, nước uống, cân nặng, luyện tập, Scanner, Dashboard, Meal Planner và báo cáo.

Kết quả thiết kế chức năng là cơ sở để xây dựng mô hình dữ liệu, xác định các API cần thiết và thiết kế giao diện tương ứng trong các mục tiếp theo. Các đặc tả Use Case chi tiết hơn được trình bày trong Phụ lục A.

2.4. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được thiết kế theo mô hình quan hệ nhằm lưu trữ dữ liệu có cấu trúc, duy trì tính toàn vẹn giữa các thực thể và hỗ trợ các truy vấn tổng hợp phục vụ Dashboard, báo cáo PDF và các thuật toán phân tích dinh dưỡng.

Hệ thống sử dụng MySQL làm hệ quản trị cơ sở dữ liệu. Các bảng dữ liệu được ánh xạ với các Model trong Sequelize ORM. Cách tổ chức này giúp quá trình thao tác dữ liệu trong mã nguồn được thống nhất, đồng thời hỗ trợ định nghĩa quan hệ giữa các bảng thông qua khóa chính, khóa ngoại và các Association giữa các Model.

Trong phạm vi chương này, phần thiết kế cơ sở dữ liệu chỉ trình bày các nhóm bảng chính, quan hệ giữa các bảng và một số quyết định thiết kế quan trọng. Danh sách trường dữ liệu chi tiết, kiểu dữ liệu, ràng buộc cụ thể và ERD đầy đủ được trình bày trong Phụ lục C.

2.4.1. Nguyên tắc thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu của hệ thống được xây dựng dựa trên một số nguyên tắc chính.

Thứ nhất, dữ liệu cần được tổ chức theo từng nhóm nghiệp vụ rõ ràng. Các nhóm dữ liệu chính bao gồm người dùng và hồ sơ sức khỏe, thực phẩm, nhật ký ăn uống, theo dõi sức khỏe, Meal Planner, Adaptive TDEE và Scanner. Cách phân nhóm này giúp cấu trúc cơ sở dữ liệu bám sát các chức năng đã phân tích ở các mục trước.

Thứ hai, dữ liệu phát sinh trong quá trình sử dụng cần được gắn với người dùng cụ thể thông qua khóa ngoại. Các dữ liệu như nhật ký ăn uống, nước uống, cân nặng, luyện tập và log Adaptive TDEE đều liên kết với người dùng để bảo đảm dữ liệu được tách biệt theo từng tài khoản.

Thứ ba, các bản ghi có tính lịch sử cần được lưu theo thời gian. Nhật ký ăn uống, cân nặng, nước uống, luyện tập và Adaptive TDEE đều cần có thông tin ngày ghi nhận hoặc thời điểm tạo để phục vụ thống kê, biểu đồ và phân tích xu hướng.

Thứ tư, dữ liệu dinh dưỡng trong nhật ký ăn uống cần phản ánh thông tin tại thời điểm người dùng ghi nhận. Vì vậy, bản ghi nhật ký không chỉ liên kết với bảng thực phẩm mà còn lưu lại các giá trị dinh dưỡng snapshot. Cách thiết kế này giúp lịch sử ăn uống không bị sai lệch khi dữ liệu thực phẩm gốc được cập nhật.

Thứ năm, dữ liệu từ nguồn bên ngoài như Open Food Facts hoặc kết quả nhận dạng bằng AI Vision cần được chuẩn hóa trước khi lưu vào cơ sở dữ liệu. Việc chuẩn hóa giúp dữ liệu phù hợp với cấu trúc nội bộ và có thể sử dụng thống nhất trong các chức năng như nhật ký ăn uống, Scanner và Meal Planner.

Thứ sáu, các trường thường xuyên được sử dụng trong truy vấn như `userId`, `date`, `foodId` và `barcode` cần được xem xét thiết kế chỉ mục phù hợp. Điều này giúp hỗ trợ các truy vấn phổ biến như lấy nhật ký theo ngày, tra cứu mã vạch, tìm kiếm thực phẩm và tổng hợp dữ liệu cho Dashboard.

2.4.2. Phân nhóm bảng dữ liệu

Dựa trên các nghiệp vụ đã phân tích, cơ sở dữ liệu được chia thành các nhóm bảng chính sau.

Nhóm người dùng và hồ sơ sức khỏe lưu trữ thông tin tài khoản, thông tin cơ thể, mục tiêu cá nhân và các cấu hình phục vụ tính toán dinh dưỡng. Bảng trung tâm của nhóm này là `User`. Ngoài thông tin đăng nhập và hồ sơ sức khỏe, bảng người dùng còn lưu một số dữ liệu cấu hình như trạng thái hoàn tất Onboarding, mục tiêu nước, tỷ lệ chất đa lượng và các thông tin phục vụ quá trình xây dựng thực đơn.

Nhóm thực phẩm lưu dữ liệu về món ăn, nguyên liệu hoặc sản phẩm được sử dụng trong hệ thống. Bảng `Food` chứa các thông tin dinh dưỡng như năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri và một số vi chất. Dữ liệu này được sử dụng trong nhật ký ăn uống, Food Scoring, Meal Planner và Scanner.

Nhóm nhật ký ăn uống lưu các thực phẩm mà người dùng đã sử dụng theo ngày và theo bữa ăn. Bảng `DiaryEntry` liên kết với người dùng và thực phẩm, đồng thời lưu khẩu phần và các giá trị dinh dưỡng tại thời điểm ghi nhận. Đây là nhóm dữ liệu quan trọng vì được sử dụng trực tiếp trong Dashboard, báo cáo và các thuật toán phân tích.

Nhóm theo dõi sức khỏe gồm các bảng lưu cân nặng, nước uống và luyện tập. `WeightLog` lưu lịch sử cân nặng, `WaterLog` lưu lượng nước uống theo ngày, `ExerciseLog` lưu hoạt động vận động và năng lượng tiêu hao. Các bảng này đều phục vụ theo dõi hằng ngày và tổng hợp dữ liệu theo thời gian.

Nhóm Meal Planner gồm dữ liệu mẫu bữa ăn và cấu hình bữa ăn của người dùng.

MealTemplate mô tả cấu trúc bữa ăn theo các nhóm nguyên liệu. UserMealConfig lưu cách phân bổ mục tiêu dinh dưỡng theo từng bữa. Các dữ liệu này được dùng làm đầu vào cho thuật toán xây dựng thực đơn.

Nhóm Adaptive TDEE lưu lịch sử tính toán TDEE thích ứng. Bảng AdaptiveTDEELog ghi lại kết quả tính toán theo từng giai đoạn, trạng thái áp dụng hoặc bỏ qua và các giá trị liên quan đến quá trình điều chỉnh TDEE thực tế của người dùng.

Nhóm Scanner lưu dữ liệu sản phẩm được quét và dữ liệu đóng góp từ người dùng. ScannedProduct lưu thông tin sản phẩm, mã vạch, nguồn dữ liệu, thông tin dinh dưỡng và trạng thái độ tin cậy. ProductContribution lưu các đóng góp hoặc xác nhận dữ liệu của người dùng đối với sản phẩm được quét.

Báo cáo PDF không được thiết kế thành một nhóm bảng riêng. Dữ liệu báo cáo được tổng hợp trực tiếp từ hồ sơ người dùng, nhật ký ăn uống, nước uống, cân nặng, luyện tập và các dữ liệu phân tích tại thời điểm xuất báo cáo.

Bảng 2.x. Phân nhóm bảng dữ liệu của hệ thống

Nhóm dữ liệu

Một số bảng chính

Vai trò

Người dùng và hồ sơ

User

Lưu tài khoản, hồ sơ sức khỏe và cấu hình cá nhân

Thực phẩm

Food

Lưu dữ liệu dinh dưỡng của thực phẩm

Nhật ký ăn uống

DiaryEntry

Lưu thực phẩm người dùng đã sử dụng theo ngày và bữa

Theo dõi sức khỏe

WeightLog, WaterLog, ExerciseLog

Lưu cân nặng, nước uống và luyện tập

Meal Planner

MealTemplate, UserMealConfig

Lưu mẫu bữa ăn và cấu hình phân bổ bữa ăn

Adaptive TDEE

AdaptiveTDEELog

Lưu lịch sử tính toán TDEE thích ứng

Scanner

ScannedProduct, ProductContribution

Lưu sản phẩm quét và dữ liệu đóng góp

2.4.3. Quan hệ giữa các bảng chính

Quan hệ trung tâm trong cơ sở dữ liệu là quan hệ giữa User và các dữ liệu phát sinh trong quá trình sử dụng. Một người dùng có thể có nhiều bản ghi nhật ký ăn uống, nhiều bản ghi cân nặng, nhiều bản ghi nước uống, nhiều hoạt động luyện tập và nhiều log Adaptive TDEE.

Bảng Food có quan hệ với DiaryEntry. Một thực phẩm có thể xuất hiện trong nhiều bản ghi nhật ký của nhiều người dùng khác nhau. Khi người dùng thêm thực phẩm vào nhật ký, hệ thống lưu thông tin thực phẩm, khẩu phần và các giá trị dinh dưỡng snapshot trong DiaryEntry.

Đối với Meal Planner, MealTemplate được sử dụng để mô tả cấu trúc chung của bữa ăn, còn UserMealConfig liên kết với người dùng để lưu cấu hình phân bổ bữa. Dữ liệu thực phẩm trong bảng Food được sử dụng làm nguồn đầu vào cho quá trình tính toán khẩu phần.

Đối với Scanner, một sản phẩm trong ScannedProduct có thể có nhiều bản ghi đóng góp trong ProductContribution. Mỗi đóng góp liên kết với một người dùng và một sản phẩm được quét. Nếu sản phẩm đã được chuẩn hóa thành dữ liệu thực phẩm sử dụng chung, ScannedProduct có thể liên kết tùy chọn với Food.

Bảng 2.x. Tóm tắt khóa chính và khóa ngoại của một số bảng chính

Bảng

Khóa chính

Khóa ngoại chính

Vai trò

User

id

—

Lưu tài khoản, hồ sơ sức khỏe và cấu hình cá nhân

Food

id

userId, nếu là thực phẩm do người dùng tạo

Lưu dữ liệu thực phẩm

DiaryEntry

id

userId, foodId

Lưu nhật ký ăn uống

WeightLog

id

userId

Lưu lịch sử cân nặng

WaterLog

id

userId

Lưu dữ liệu nước uống theo ngày

ExerciseLog

id

userId

Lưu hoạt động luyện tập

MealTemplate

id

—

Lưu mẫu bữa ăn

UserMealConfig

id

userId

Lưu cấu hình phân bổ bữa ăn của người dùng

AdaptiveTDEELog

id

userId

Lưu lịch sử tính toán TDEE thích ứng

ScannedProduct

id

foodId, tùy chọn

Lưu sản phẩm được quét

ProductContribution

id

userId, scannedProductId

Lưu dữ liệu đóng góp cho sản phẩm quét

Gợi ý chèn Hình 2.x. Sơ đồ ERD rút gọn của hệ thống

Hình trong chương chính chỉ nên thể hiện các bảng trung tâm và quan hệ chính: User, Food, DiaryEntry, WeightLog, WaterLog, ExerciseLog, MealTemplate, UserMealConfig, AdaptiveTDEELog, ScannedProduct và ProductContribution. ERD đầy đủ nên

đưa sang Phụ lục C.

2.4.4. Thiết kế lưu trữ dữ liệu lịch sử

Một số dữ liệu trong hệ thống có tính lịch sử và cần được lưu theo thời gian để phục vụ thống kê và phân tích. Các dữ liệu này bao gồm nhật ký ăn uống, cân nặng, nước uống, luyện tập và log Adaptive TDEE.

Đối với nhật ký ăn uống, mỗi bản ghi cần gắn với ngày, bữa ăn, người dùng và thực phẩm. Các giá trị dinh dưỡng snapshot được lưu cùng bản ghi nhằm bảo toàn dữ liệu lịch sử tại thời điểm người dùng nhập liệu.

Đối với cân nặng, mỗi bản ghi lưu giá trị cân nặng tại một ngày cụ thể. Chuỗi dữ liệu cân nặng được sử dụng để hiển thị xu hướng và phục vụ thuật toán Adaptive TDEE.

Đối với nước uống và luyện tập, dữ liệu được lưu theo từng ngày để tổng hợp trên Dashboard và báo cáo PDF. Dữ liệu luyện tập phản ánh năng lượng tiêu hao, trong khi dữ liệu nước uống được so sánh với mục tiêu nước khuyến nghị.

Đối với Adaptive TDEE, hệ thống lưu lại kết quả tính toán theo từng giai đoạn. Việc lưu lịch sử tính toán giúp theo dõi quá trình điều chỉnh TDEE thực tế và xác định các bản ghi đã được áp dụng hoặc bị bỏ qua do chưa đủ điều kiện dữ liệu.

2.4.5. Thiết kế dữ liệu phục vụ Scanner

Chức năng Scanner có đặc điểm là dữ liệu có thể đến từ nhiều nguồn, bao gồm cơ sở dữ liệu nội bộ, Open Food Facts và kết quả nhận dạng bằng AI Vision. Vì vậy, dữ liệu sản phẩm được quét cần được lưu theo cấu trúc riêng trước khi chuyển đổi hoặc liên kết với dữ liệu thực phẩm chung.

Bảng ScannedProduct lưu thông tin sản phẩm, mã vạch, dữ liệu dinh dưỡng, nguồn dữ liệu, trạng thái xác thực và điểm tin cậy. Các thông tin này giúp hệ thống phân biệt dữ liệu đã kiểm chứng, dữ liệu lấy từ nguồn bên ngoài và dữ liệu do người dùng đóng góp.

Khi người dùng đóng góp hoặc xác nhận dữ liệu sản phẩm, thông tin đóng góp được lưu trong ProductContribution. Cách thiết kế này cho phép hệ thống theo dõi nguồn gốc dữ liệu, đối chiếu nhiều đóng góp và đánh giá độ ổn định của dữ liệu sản phẩm theo thời gian.

Trong trường hợp sản phẩm được chuẩn hóa thành thực phẩm sử dụng chung, bản ghi sản phẩm quét có thể liên kết với bảng Food. Quan hệ này giúp hạn chế trùng lặp giữa dữ liệu Scanner và dữ liệu thực phẩm chính của hệ thống.

2.4.6. Thiết kế hỗ trợ truy vấn và tổng hợp dữ liệu

Cơ sở dữ liệu cần hỗ trợ các truy vấn thường xuyên phát sinh trong hệ thống, chẳng hạn lấy nhật ký ăn uống theo người dùng và ngày, tổng hợp nước uống trong ngày, lấy lịch sử cân nặng theo khoảng thời gian, tìm kiếm thực phẩm và tra cứu sản phẩm theo mã vạch.

Để hỗ trợ các truy vấn này, các trường như `userId`, `date`, `foodId` và `barcode` cần được thiết kế chỉ mục phù hợp. Đối với các bảng dữ liệu lịch sử như `DiaryEntry`, `WeightLog`, `WaterLog` và `ExerciseLog`, dạng truy vấn phổ biến là kết hợp điều kiện người dùng và ngày ghi nhận. Đối với `Scanner`, mã vạch là trường quan trọng phục vụ quá trình tra cứu sản phẩm đóng gói.

Thiết kế chỉ mục hợp lý giúp hệ thống truy xuất dữ liệu ổn định hơn khi số lượng bản ghi tăng lên. Tuy nhiên, số lượng chỉ mục cần được cân nhắc để tránh làm tăng chi phí ghi dữ liệu không cần thiết.

2.4.7. Tổng kết thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu của hệ thống tập trung vào ba yêu cầu chính: quản lý dữ liệu theo người dùng, lưu trữ dữ liệu lịch sử và hỗ trợ các thuật toán phân tích dinh dưỡng.

Các bảng dữ liệu được tổ chức theo từng nhóm nghiệp vụ, trong đó `User` và `Food` là hai bảng trung tâm. Các dữ liệu phát sinh như nhật ký ăn uống, cân nặng, nước uống, luyện tập, log Adaptive TDEE và dữ liệu `Scanner` được liên kết thông qua khóa ngoại để bảo đảm tính nhất quán.

Phần thiết kế chi tiết các bảng, bao gồm danh sách trường dữ liệu, kiểu dữ liệu, ràng buộc và ERD đầy đủ, được trình bày trong Phụ lục C. Cách trình bày này giúp nội dung chương chính tập trung vào mô hình dữ liệu và các quan hệ quan trọng của hệ thống.

2.5. Thiết kế API

API là lớp trung gian cho phép Frontend giao tiếp với Backend để thực hiện các nghiệp vụ của hệ thống. Trong hệ thống quản lý dinh dưỡng cá nhân, API được thiết kế theo hướng RESTful, sử dụng giao thức HTTP và trao đổi dữ liệu dưới định dạng JSON.

Các API của hệ thống được đặt dưới tiền tố chung `/api/v1`. Cách tổ chức này giúp phân tách rõ các API nghiệp vụ với tài nguyên tĩnh hoặc giao diện phía Frontend. Đồng thời, việc sử dụng phiên bản trong đường dẫn API tạo điều kiện thuận lợi nếu hệ thống cần thay đổi hoặc mở rộng API trong các giai đoạn sau.

Trong phạm vi chương này, nội dung chỉ trình bày nguyên tắc thiết kế API, cấu trúc phản hồi, cơ chế xác thực, các nhóm API chính và một số endpoint tiêu biểu. Danh sách đầy đủ các endpoint, tham số truyền vào và cấu trúc phản hồi chi tiết được đưa vào Phụ lục B.

2.5.1. Nguyên tắc thiết kế API

Thiết kế API của hệ thống tuân theo một số nguyên tắc chính sau.

Thứ nhất, các API được tổ chức theo nhóm chức năng. Mỗi nhóm API tương ứng với một nghiệp vụ chính của hệ thống như xác thực, Dashboard, nhật ký ăn uống, hồ sơ cá nhân, cân nặng, nước uống, luyện tập, Meal Planner, Scanner, Adaptive TDEE và báo cáo. Cách tổ chức này giúp đường dẫn API phản ánh rõ phạm vi xử lý của từng chức

năng.

Thứ hai, API sử dụng các phương thức HTTP phù hợp với ý nghĩa thao tác dữ liệu. Phương thức GET được sử dụng để lấy dữ liệu, POST được sử dụng để tạo mới dữ liệu hoặc thực hiện một hành động xử lý, PUT được sử dụng để cập nhật dữ liệu và DELETE được sử dụng để xóa dữ liệu.

Thứ ba, các API trả về dữ liệu theo cấu trúc thống nhất. Kết quả phản hồi thường gồm trạng thái thực hiện, dữ liệu trả về và thông báo tương ứng. Khi xảy ra lỗi, hệ thống trả về thông báo lỗi theo cùng một cấu trúc để Frontend có thể xử lý và hiển thị nhất quán.

Thứ tư, các API liên quan đến dữ liệu cá nhân yêu cầu người dùng phải xác thực. Sau khi đăng nhập, Frontend gửi Access Token trong header của yêu cầu. Backend kiểm tra Token thông qua Middleware xác thực trước khi chuyển yêu cầu tới Controller và Service tương ứng.

Thứ năm, một số API nhạy cảm hoặc phụ thuộc dịch vụ bên ngoài cần được giới hạn tần suất gọi. Các API như đăng nhập, đăng ký và nhận dạng hình ảnh bằng AI Vision cần có cơ chế hạn chế số lượng yêu cầu trong một khoảng thời gian nhằm giảm rủi ro lạm dụng và hạn chế tải không cần thiết cho hệ thống.

Gợi ý chèn Hình 2.x. Luồng request/response giữa Frontend và Backend

Hình nên thể hiện: React Frontend → Axios/TanStack Query → RESTful API → Middleware xác thực → Controller → Service → Model → Database → Response JSON.

2.5.2. Cấu trúc phản hồi API

Để Frontend có thể xử lý dữ liệu một cách nhất quán, các API được thiết kế với cấu trúc phản hồi thống nhất. Khi xử lý thành công, API trả về trạng thái thành công, dữ liệu và thông báo nếu cần. Khi xử lý thất bại, API trả về trạng thái thất bại cùng thông báo lỗi. Đối với lỗi kiểm tra dữ liệu đầu vào, phản hồi có thể kèm theo danh sách trường dữ liệu không hợp lệ.

Cấu trúc phản hồi tổng quát có thể mô tả như sau:

"success": true,

"data": ,

"message": "..."

Trường hợp lỗi:

"success": false,

"message": "..."

Trường hợp lỗi kiểm tra dữ liệu đầu vào:

"success": false,

"message": "Dữ liệu không hợp lệ.",

"errors": [

```
"field": "email",  
"msg": "..."  
]
```

Cách thiết kế phản hồi thống nhất giúp giảm sự phức tạp trong xử lý lỗi ở Frontend. Các thành phần giao diện có thể dựa vào trường success, data, message và errors để xác định trạng thái xử lý.

Bảng 2.x. Cấu trúc phản hồi API

Trường dữ liệu

Ý nghĩa

success

Trạng thái xử lý thành công hoặc thất bại

data

Dữ liệu trả về khi xử lý thành công

message

Thông báo kết quả hoặc lỗi

errors

Danh sách lỗi chi tiết khi dữ liệu đầu vào không hợp lệ

2.5.3. Thiết kế xác thực API

Các API của hệ thống được chia thành hai nhóm: API công khai và API yêu cầu xác thực.

API công khai bao gồm đăng ký, đăng nhập, đăng xuất và làm mới Access Token. Đây là các API cần được truy cập trước khi người dùng có trạng thái xác thực hợp lệ hoặc trong quá trình duy trì phiên đăng nhập.

Các API còn lại như Dashboard, nhật ký ăn uống, cân nặng, nước uống, luyện tập, hồ sơ cá nhân, báo cáo, Adaptive TDEE, Meal Planner và Scanner yêu cầu người dùng phải xác thực trước khi sử dụng. Frontend gửi Access Token trong header của yêu cầu. Backend kiểm tra Token thông qua Middleware xác thực. Nếu Token hợp lệ, yêu cầu được chuyển tiếp tới Controller tương ứng. Nếu Token không hợp lệ hoặc hết hạn, hệ thống trả về lỗi xác thực.

Hệ thống sử dụng Access Token cho các yêu cầu API và Refresh Token để làm mới Access Token khi cần. Refresh Token được lưu dưới dạng HttpOnly Cookie nhằm hạn chế việc truy cập trực tiếp từ mã JavaScript phía trình duyệt. Khi Access Token hết hạn, Frontend có thể gọi API làm mới Token để nhận Access Token mới và tiếp tục thực hiện yêu cầu.

Cách thiết kế này giúp bảo vệ các API nghiệp vụ, đồng thời giảm việc yêu cầu người dùng đăng nhập lại trong quá trình sử dụng.

Gợi ý chèn Hình 2.x. Luồng xác thực API bằng JWT

Hình nên thể hiện: Đăng nhập → nhận Access Token và Refresh Token → gọi API kèm Authorization Header → Middleware kiểm tra Token → xử lý nghiệp vụ. Có thể bổ sung nhánh làm mới Access Token khi Access Token hết hạn.

2.5.4. Các nhóm API chính

Các API của hệ thống được tổ chức theo các nhóm nghiệp vụ chính. Bảng dưới đây trình bày các nhóm API và vai trò của từng nhóm.

Bảng 2.x. Các nhóm API chính của hệ thống

Nhóm API

Đường dẫn chính

Vai trò

Auth

/api/v1/auth

Đăng ký, đăng nhập, đăng xuất, làm mới Token, lấy thông tin người dùng hiện tại
Dashboard

/api/v1/dashboard

Lấy dữ liệu tổng hợp phục vụ Dashboard

Diary

/api/v1/diary

Quản lý nhật ký ăn uống và thực phẩm tùy chỉnh

Profile

/api/v1/profile

Quản lý hồ sơ sức khỏe, macro, dị ứng và onboarding

Weight

/api/v1/weight

Quản lý dữ liệu cân nặng

Water

/api/v1/water

Quản lý dữ liệu nước uống

Exercise

/api/v1/exercise

Quản lý dữ liệu luyện tập

Meal Planner

/api/v1/meal-planner

Cấu hình bữa ăn, lấy nguyên liệu, sinh thực đơn và thay thế nguyên liệu

Adaptive TDEE

/api/v1/adaptive-tdee

Quản lý trạng thái và lịch sử tính toán Adaptive TDEE

Scanner

/api/v1/scanner

Quét mã vạch, nhận dạng bảng thành phần và lưu dữ liệu sản phẩm

Report

/api/v1/report

Xuất báo cáo PDF

2.5.5. Một số endpoint tiêu biểu

Bảng dưới đây trình bày một số endpoint tiêu biểu đại diện cho các nhóm chức năng quan trọng của hệ thống. Các endpoint được lựa chọn nhằm thể hiện cách tổ chức API và vai trò của từng nhóm nghiệp vụ. Danh sách đầy đủ các endpoint được trình bày trong Phụ lục B.

Bảng 2.x. Một số endpoint tiêu biểu của hệ thống

Nhóm API

Phương thức

Endpoint

Mô tả

Auth

POST

/api/v1/auth/register

Đăng ký tài khoản mới

Auth

POST

/api/v1/auth/login

Đăng nhập và nhận Access Token

Auth

POST

/api/v1/auth/refresh-token

Làm mới Access Token

Dashboard

GET

/api/v1/dashboard?date=YYYY-MM-DD

Lấy dữ liệu tổng hợp cho Dashboard theo ngày

Diary

GET

/api/v1/diary
Lấy nhật ký ăn uống theo ngày
Diary
POST
/api/v1/diary/entries
Thêm thực phẩm vào nhật ký
Diary
GET
/api/v1/diary/foods/search
Tìm kiếm thực phẩm
Profile
POST
/api/v1/profile/onboarding
Lưu thông tin khởi tạo hồ sơ
Profile
PUT
/api/v1/profile/macros
Cập nhật cấu hình macro
Weight
POST
/api/v1/weight
Thêm bản ghi cân nặng
Water
POST
/api/v1/water
Ghi nhận lượng nước uống
Exercise
POST
/api/v1/exercise
Thêm hoạt động luyện tập
Meal Planner
POST
/api/v1/meal-planner/generate
Sinh gợi ý thực đơn
Adaptive TDEE
GET

/api/v1/adaptive-tdee/status

Lấy trạng thái Adaptive TDEE

Scanner

POST

/api/v1/scanner/barcode-lookup

Tra cứu sản phẩm theo mã vạch

Scanner

POST

/api/v1/scanner/ai-vision

Nhận dạng bảng thành phần dinh dưỡng từ ảnh

Scanner

POST

/api/v1/scanner/confirm-contribution

Xác nhận và lưu dữ liệu sản phẩm đóng góp

Report

GET

/api/v1/report/pdf?range=week

Xuất báo cáo PDF theo tuần

Các endpoint trên cho thấy API của hệ thống được tổ chức theo nhóm nghiệp vụ thay vì gom toàn bộ chức năng vào một đường dẫn chung. Các API như Dashboard, Meal Planner, Adaptive TDEE, Scanner và Report không chỉ thực hiện thao tác CRUD đơn giản mà còn gọi tới các Service tương ứng để tổng hợp dữ liệu, xử lý thuật toán hoặc tích hợp với dịch vụ bên ngoài.

2.5.6. Xử lý lỗi và mã trạng thái

Các API của hệ thống sử dụng mã trạng thái HTTP để phản ánh kết quả xử lý. Việc kết hợp mã trạng thái với cấu trúc phản hồi thống nhất giúp Frontend xác định được nguyên nhân lỗi và đưa ra phản hồi phù hợp cho người dùng.

Bảng 2.x. Một số mã trạng thái API

Mã trạng thái

Ý nghĩa

200

Xử lý thành công

201

Tạo mới dữ liệu thành công

400

Dữ liệu yêu cầu không hợp lệ

401

Chưa xác thực hoặc Token không hợp lệ

403

Không có quyền thực hiện thao tác

404

Không tìm thấy tài nguyên hoặc endpoint tương ứng

409

Xung đột dữ liệu

422

Lỗi kiểm tra dữ liệu đầu vào

429

Vượt quá giới hạn số lượng yêu cầu

500

Lỗi máy chủ

Đối với các API nhạy cảm như đăng nhập, đăng ký hoặc AI Vision, mã trạng thái 429 có thể được sử dụng khi người dùng gửi quá nhiều yêu cầu trong một khoảng thời gian. Các lỗi xác thực được xử lý bằng mã 401, trong khi lỗi dữ liệu đầu vào được biểu diễn bằng các mã như 400 hoặc 422 tùy theo trường hợp.

2.5.7. Tổng kết thiết kế API

Thiết kế API của hệ thống được tổ chức theo nhóm nghiệp vụ, sử dụng tiền tố /api/v1 và phản hồi dữ liệu dưới dạng JSON. Các API liên quan đến dữ liệu cá nhân đều yêu cầu xác thực bằng Access Token trước khi xử lý, trong khi Refresh Token được sử dụng để duy trì phiên đăng nhập khi Access Token hết hạn.

Trong chương chính, chỉ các nhóm API và một số endpoint tiêu biểu được trình bày nhằm làm rõ cách tổ chức giao tiếp giữa Frontend và Backend. Danh sách endpoint đầy đủ, tham số truyền vào, request body và cấu trúc phản hồi chi tiết được đưa vào Phụ lục B. Cách trình bày này giúp phần thiết kế API đủ rõ để thể hiện kiến trúc giao tiếp của hệ thống mà không làm nội dung chương bị dàn trải.

2.6. Thiết kế giao diện

Thiết kế giao diện được thực hiện nhằm tổ chức các chức năng nghiệp vụ thành các màn hình dễ thao tác, phù hợp với luồng sử dụng của người dùng cá nhân. Do hệ thống có nhiều nhóm chức năng như Dashboard, nhật ký ăn uống, theo dõi cân nặng, luyện tập, Meal Planner, Scanner và hồ sơ cá nhân, giao diện cần được thiết kế theo hướng phân tách rõ từng khu vực chức năng.

Frontend của hệ thống được xây dựng bằng React theo mô hình Single Page Application. Các trang chính được quản lý thông qua React Router. Một số trang công khai

như trang giới thiệu, đăng nhập và đăng ký có thể truy cập khi chưa xác thực. Trang Onboarding được sử dụng sau khi người dùng đăng nhập nhưng chưa hoàn tất khai báo hồ sơ sức khỏe ban đầu. Các trang nghiệp vụ chính chỉ được truy cập sau khi người dùng đã đăng nhập và hoàn tất bước Onboarding.

Trong phạm vi chương này, phần thiết kế giao diện tập trung vào cấu trúc điều hướng, bố cục ứng dụng chung và một số màn hình tiêu biểu. Các ảnh giao diện chi tiết hoặc các modal phụ có thể được đưa sang phụ lục để tránh làm nội dung chương bị dàn trải.

2.6.1. Nguyên tắc thiết kế giao diện

Giao diện của hệ thống được thiết kế dựa trên các nguyên tắc sau.

Thứ nhất, các chức năng thường xuyên sử dụng cần được đặt ở vị trí dễ truy cập. Đối với hệ thống quản lý dinh dưỡng cá nhân, các thao tác như xem Dashboard, ghi nhật ký ăn uống, ghi nhận nước uống, cập nhật cân nặng, ghi nhận luyện tập và thêm thực phẩm là các thao tác có thể lặp lại hằng ngày.

Thứ hai, dữ liệu tổng hợp cần được trình bày trực quan. Các chỉ số như năng lượng đã tiêu thụ, mục tiêu calo, tỷ lệ chất đa lượng, lượng nước uống, cân nặng và hoạt động luyện tập được thể hiện thông qua thẻ thông tin, biểu đồ hoặc thanh tiến trình. Cách trình bày này giúp người dùng nhanh chóng nắm được trạng thái hiện tại mà không cần đọc nhiều dữ liệu dạng bảng.

Thứ ba, các thao tác nhập liệu cần được tách thành biểu mẫu hoặc modal riêng. Các thao tác như thêm thực phẩm, cập nhật cân nặng hoặc ghi nhận luyện tập thường chỉ phục vụ một nhiệm vụ cụ thể. Việc sử dụng modal giúp người dùng thực hiện thao tác nhanh mà không phải rời khỏi hoàn toàn màn hình đang sử dụng.

Thứ tư, giao diện cần thể hiện rõ trạng thái xử lý dữ liệu. Do dữ liệu được lấy từ Backend thông qua API, các trạng thái như đang tải, lỗi, thành công hoặc không có dữ liệu cần được hiển thị phù hợp để người dùng hiểu được kết quả thao tác.

Thứ năm, bố cục giao diện cần phù hợp với nhiều kích thước màn hình. Với màn hình lớn, hệ thống có thể sử dụng thanh điều hướng bên để chuyển nhanh giữa các chức năng. Với màn hình nhỏ, thanh điều hướng cần được thu gọn nhằm ưu tiên không gian cho nội dung chính.

2.6.2. Cấu trúc điều hướng và bố cục ứng dụng

Hệ thống giao diện được chia thành ba nhóm chính: nhóm trang công khai, trang Onboarding và nhóm trang nghiệp vụ yêu cầu xác thực.

Nhóm trang công khai bao gồm trang giới thiệu, trang đăng nhập và trang đăng ký. Các trang này phục vụ người dùng trước khi truy cập vào hệ thống chính. Sau khi đăng nhập hoặc đăng ký thành công, hệ thống kiểm tra trạng thái khởi tạo hồ sơ sức khỏe của người dùng.

Trang Onboarding được sử dụng cho người dùng đã đăng nhập nhưng chưa hoàn tất hồ sơ sức khỏe ban đầu. Tại đây, người dùng nhập các thông tin như giới tính, tuổi hoặc ngày sinh, chiều cao, cân nặng, mức độ vận động và mục tiêu cá nhân. Các thông tin này là cơ sở để hệ thống tính toán các chỉ số như BMI, BMR, TDEE, mục tiêu năng lượng, tỷ lệ chất đa lượng và mục tiêu nước.

Nhóm trang nghiệp vụ gồm Dashboard, nhật ký ăn uống, Meal Planner, theo dõi cân nặng, luyện tập và hồ sơ cá nhân. Các trang này được đặt trong bố cục ứng dụng chung. Bố cục này gồm thanh điều hướng, vùng nội dung chính và các thành phần hỗ trợ như thông báo, trạng thái tải dữ liệu hoặc thông báo lỗi.

Thanh điều hướng được sử dụng để chuyển giữa các chức năng chính. Vùng nội dung chính thay đổi theo trang hiện tại. Các thành phần dùng chung như nút, thẻ thông tin, biểu đồ, modal và biểu mẫu được tái sử dụng giữa nhiều màn hình để giữ giao diện nhất quán.

Gợi ý chèn Hình 2.x. Sơ đồ điều hướng giao diện của hệ thống

Hình nên thể hiện luồng: Landing/Login/Register → Onboarding → AppLayout → Dashboard, Diary, Meal Planner, Weight, Exercise, Profile.

2.6.3. Thiết kế giao diện Dashboard

Dashboard là màn hình tổng hợp dữ liệu chính của hệ thống. Mục tiêu của màn hình này là giúp người dùng theo dõi nhanh tình trạng dinh dưỡng và sức khỏe trong ngày hoặc tại thời điểm được chọn.

Dashboard hiển thị dữ liệu từ nhiều nguồn như hồ sơ sức khỏe, nhật ký ăn uống, nước uống, cân nặng, luyện tập và kết quả phân tích. Các thành phần chính gồm bộ chọn ngày, vòng tiến độ calo, thẻ thông tin chất đa lượng, thanh tiến trình nước uống, biểu đồ cân nặng, danh sách bữa ăn gần đây, danh sách hoạt động gần đây và khối nhận xét dinh dưỡng trong ngày.

Ngoài các chỉ số theo dõi hằng ngày, Dashboard có thể hiển thị thông tin liên quan đến Adaptive TDEE khi người dùng có đủ dữ liệu cân nặng và nhật ký ăn uống. Chức năng xuất báo cáo PDF cũng có thể được bố trí tại Dashboard hoặc khu vực chức năng liên quan, cho phép người dùng tải báo cáo theo tuần hoặc tháng.

Dashboard không chỉ hiển thị dữ liệu mà còn đóng vai trò là điểm truy cập nhanh tới một số thao tác thường dùng như thêm thực phẩm, ghi nhận nước uống hoặc xem báo cáo. Vì vậy, giao diện cần ưu tiên khả năng tổng hợp, dễ quan sát và hạn chế số bước thao tác.

Gợi ý chèn Hình 2.x. Giao diện Dashboard

Ảnh nên thể hiện các thẻ chỉ số, vòng tiến độ calo, macro, nước uống, hoạt động gần đây và khu vực nhận xét dinh dưỡng.

2.6.4. Thiết kế giao diện nhật ký ăn uống và luồng thêm thực phẩm

Giao diện nhật ký ăn uống được thiết kế để người dùng ghi nhận thực phẩm theo từng bữa trong ngày. Các bữa ăn được chia thành các nhóm như bữa sáng, bữa trưa, bữa tối và bữa phụ. Cách phân chia này phù hợp với thói quen sử dụng và thuận tiện cho việc tổng hợp dinh dưỡng theo ngày.

Các thành phần chính của giao diện nhật ký ăn uống gồm bộ chọn ngày, danh sách bữa ăn, danh sách thực phẩm đã ghi nhận, thông tin năng lượng và dinh dưỡng của từng bản ghi, các chỉ số tổng hợp trong ngày và nút thêm thực phẩm. Khi người dùng thêm thực phẩm, hệ thống mở modal thêm thực phẩm để người dùng tìm kiếm, chọn thực phẩm, nhập khẩu phần và xác nhận bữa ăn tương ứng.

Luồng thêm thực phẩm có thể kết hợp nhiều phương thức nhập liệu. Người dùng có thể tìm kiếm thực phẩm trong cơ sở dữ liệu, tạo thực phẩm tùy chỉnh hoặc sử dụng Hybrid Nutrition Scanner. Scanner được thiết kế như một luồng hỗ trợ trong modal thêm thực phẩm, không phải một trang độc lập. Luồng này hỗ trợ quét mã vạch sản phẩm hoặc nhận dạng bảng thành phần dinh dưỡng từ hình ảnh.

Với luồng mã vạch, người dùng có thể quét hoặc nhập mã vạch thủ công. Hệ thống gửi mã vạch tới Backend để tra cứu dữ liệu sản phẩm. Với luồng AI Vision, người dùng gửi ảnh bảng thành phần dinh dưỡng, hệ thống trích xuất dữ liệu và hiển thị kết quả để người dùng kiểm tra trước khi sử dụng. Việc yêu cầu người dùng xác nhận kết quả là cần thiết vì dữ liệu từ hình ảnh có thể có sai lệch và cần được kiểm tra trước khi lưu.

Sau khi người dùng xác nhận thêm thực phẩm vào nhật ký, dữ liệu được gửi tới Backend để lưu lại. Các giá trị dinh dưỡng tại thời điểm ghi nhận cần được lưu cùng bản ghi nhật ký nhằm bảo toàn dữ liệu lịch sử khi thông tin thực phẩm gốc thay đổi.

Gợi ý chèn Hình 2.x. Giao diện nhật ký ăn uống và modal thêm thực phẩm

Ảnh nên thể hiện danh sách bữa ăn, modal thêm thực phẩm và khu vực Scanner trong modal.

2.6.5. Thiết kế giao diện Meal Planner

Giao diện Meal Planner phục vụ chức năng xây dựng thực đơn dựa trên mục tiêu dinh dưỡng của người dùng. Màn hình này cho phép người dùng chọn bữa ăn, chọn mẫu bữa ăn, thiết lập một số tùy chọn liên quan đến nguyên liệu và nhận kết quả gợi ý từ hệ thống.

Các thành phần chính của giao diện Meal Planner gồm cấu hình phân bổ bữa ăn, danh sách mẫu bữa ăn, khu vực chọn hoặc ghim nguyên liệu, kết quả gợi ý thực đơn, thông tin khối lượng từng nguyên liệu và thao tác thay thế nguyên liệu. Kết quả thực đơn cần được trình bày theo dạng có cấu trúc, gồm tên nguyên liệu, khối lượng và giá trị dinh dưỡng tương ứng để người dùng có thể kiểm tra trước khi sử dụng.

Sau khi hệ thống sinh thực đơn, người dùng có thể thay thế nguyên liệu nếu cần hoặc

xác nhận sử dụng kết quả. Kết quả thực đơn có thể được dùng làm dữ liệu đầu vào cho nhật ký ăn uống trong ngày. Do chức năng này liên quan trực tiếp đến thuật toán Meal Planner, giao diện cần thể hiện rõ dữ liệu đầu vào, trạng thái xử lý và kết quả đầu ra để người dùng hiểu được thao tác đang thực hiện.

Gợi ý chèn Hình 2.x. Giao diện Meal Planner

Ảnh nên thể hiện khu vực cấu hình bữa ăn, chọn mẫu bữa, kết quả gợi ý thực đơn và thao tác thay thế nguyên liệu.

2.6.6. Tổng hợp các màn hình chức năng còn lại

Ngoài các màn hình trọng tâm như Dashboard, nhật ký ăn uống và Meal Planner, hệ thống còn có một số màn hình phục vụ các nghiệp vụ theo dõi và cấu hình cá nhân. Các màn hình này được mô tả tóm tắt trong bảng dưới đây nhằm tránh làm nội dung chương bị dàn trải.

Bảng 2.x. Tổng hợp một số màn hình chức năng của hệ thống

Màn hình

Vai trò

Thành phần chính

Onboarding

Khởi tạo hồ sơ sức khỏe ban đầu cho người dùng

Biểu mẫu thông tin cá nhân, chiều cao, cân nặng, mức độ vận động, mục tiêu cá nhân

Theo dõi cân nặng

Lưu và hiển thị lịch sử cân nặng của người dùng

Biểu đồ cân nặng, danh sách bản ghi, biểu mẫu thêm cân nặng

Luyện tập

Ghi nhận hoạt động vận động và năng lượng tiêu hao

Danh sách hoạt động, biểu mẫu thêm luyện tập, thông tin năng lượng tiêu hao

Hồ sơ cá nhân

Cập nhật thông tin cơ thể, mục tiêu và cấu hình dinh dưỡng

Thông tin cá nhân, cấu hình macro, mục tiêu nước, thông tin dị ứng hoặc loại trừ thực phẩm

Xuất báo cáo PDF

Tổng hợp dữ liệu dinh dưỡng theo tuần hoặc tháng

Tùy chọn khoảng thời gian, nút xuất báo cáo, trạng thái tải file

Các màn hình trên có vai trò hỗ trợ cho quá trình theo dõi dinh dưỡng. Dữ liệu từ cân nặng, luyện tập và hồ sơ cá nhân được sử dụng trong Dashboard, báo cáo và một số thuật toán phân tích. Chức năng xuất báo cáo PDF không cần một trang riêng vì dữ liệu báo cáo được tổng hợp từ các nhóm dữ liệu đã có và có thể được kích hoạt từ Dashboard

hoặc khu vực chức năng liên quan.

2.6.7. Tổng kết thiết kế giao diện

Thiết kế giao diện của hệ thống được tổ chức theo luồng sử dụng của người dùng, bao gồm trang công khai, trang Onboarding và các trang nghiệp vụ trong bố cục ứng dụng chung. Các màn hình trọng tâm như Dashboard, nhật ký ăn uống, Meal Planner và luồng Hybrid Nutrition Scanner được thiết kế để hỗ trợ các nghiệp vụ chính của hệ thống.

Các màn hình còn lại như cân nặng, luyện tập, hồ sơ cá nhân và xuất báo cáo PDF được trình bày ở mức tổng hợp nhằm giữ nội dung chương ngắn gọn. Phần triển khai cụ thể các component, hook, route và cơ chế quản lý dữ liệu trên Frontend sẽ được trình bày trong Chương 3.

2.7. Kết luận chương

Chương 2 đã trình bày quá trình phân tích và thiết kế hệ thống quản lý dinh dưỡng cá nhân. Nội dung chương bắt đầu từ việc xác định yêu cầu chức năng và yêu cầu phi chức năng, sau đó phân tích các nghiệp vụ chính, thiết kế chức năng theo Use Case, thiết kế cơ sở dữ liệu, thiết kế API và thiết kế giao diện.

Kết quả phân tích yêu cầu cho thấy hệ thống cần đáp ứng nhiều nhóm chức năng liên quan đến quản lý dữ liệu cá nhân, khởi tạo hồ sơ sức khỏe, ghi nhận nhật ký ăn uống, ghi nhận nước uống, theo dõi cân nặng, luyện tập, phân tích dinh dưỡng, xây dựng thực đơn, hỗ trợ nhập liệu bằng Hybrid Nutrition Scanner và xuất báo cáo PDF. Các yêu cầu này không tồn tại độc lập mà có sự liên kết thông qua dữ liệu người dùng và dữ liệu dinh dưỡng được ghi nhận theo thời gian.

Phần phân tích nghiệp vụ đã mô tả các luồng xử lý chính trong quá trình người dùng sử dụng hệ thống, bao gồm đăng ký, đăng nhập, khởi tạo hồ sơ sức khỏe, ghi nhật ký ăn uống, ghi nhận nước uống, theo dõi cân nặng, ghi nhận luyện tập, sử dụng Hybrid Nutrition Scanner, xem Dashboard, xây dựng thực đơn và xuất báo cáo PDF. Các luồng nghiệp vụ này là cơ sở để xác định các Use Case, quan hệ giữa các chức năng và các nhóm dữ liệu cần thiết.

Về thiết kế chức năng, hệ thống được xác định với tác nhân nghiệp vụ chính là người dùng cá nhân. Các Use Case được tổ chức theo từng nhóm nghiệp vụ, trong đó một số Use Case tiêu biểu như đăng nhập, khởi tạo và cập nhật hồ sơ sức khỏe, ghi nhật ký ăn uống, ghi nhận nước uống, sử dụng Hybrid Nutrition Scanner, xây dựng thực đơn và xuất báo cáo PDF đã được mô tả ở mức khái quát. Các đặc tả chi tiết hơn được đưa vào phụ lục để giữ phần nội dung chính tập trung vào thiết kế tổng thể.

Về thiết kế dữ liệu, cơ sở dữ liệu được tổ chức theo các nhóm bảng chính như người dùng và hồ sơ sức khỏe, thực phẩm, nhật ký ăn uống, theo dõi sức khỏe, Meal Planner, Adaptive TDEE và Scanner. Thiết kế này giúp dữ liệu được quản lý theo từng người

dùng, lưu trữ được lịch sử theo thời gian và hỗ trợ các thuật toán phân tích dinh dưỡng. Bên cạnh đó, việc lưu giá trị dinh dưỡng snapshot trong nhật ký ăn uống giúp bảo toàn dữ liệu lịch sử khi thông tin thực phẩm gốc thay đổi.

Về thiết kế API, các endpoint được tổ chức theo nhóm chức năng dưới tiền tố `/api/v1`, sử dụng định dạng JSON và cấu trúc phản hồi thống nhất. Các API liên quan đến dữ liệu cá nhân yêu cầu xác thực bằng Access Token, trong khi Refresh Token được sử dụng để hỗ trợ duy trì phiên đăng nhập khi Access Token hết hạn. Thiết kế này tạo ra lớp giao tiếp rõ ràng giữa Frontend và Backend, đồng thời bảo đảm các chức năng nghiệp vụ được truy cập theo đúng quyền của người dùng.

Về thiết kế giao diện, hệ thống được chia thành các trang công khai, trang Onboarding và các trang nghiệp vụ trong bố cục ứng dụng chung. Các màn hình chính như Dashboard, nhật ký ăn uống, Meal Planner, cân nặng, luyện tập và hồ sơ cá nhân được thiết kế phù hợp với các nghiệp vụ đã phân tích. Bên cạnh đó, Hybrid Nutrition Scanner được thiết kế như một luồng hỗ trợ nhập liệu trong quá trình thêm thực phẩm, còn chức năng xuất báo cáo PDF được bố trí như một thao tác trên Dashboard hoặc khu vực chức năng liên quan.

Nhìn chung, Chương 2 đã xác định được cấu trúc chức năng, dữ liệu, API và giao diện của hệ thống. Các kết quả phân tích và thiết kế trong chương này là cơ sở trực tiếp cho quá trình xây dựng, triển khai các module chức năng và trình bày các thuật toán xử lý trong Chương 3.

CHƯƠNG 3. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

Trên cơ sở kết quả phân tích yêu cầu và thiết kế hệ thống ở Chương 2, Chương 3 trình bày quá trình xây dựng và triển khai hệ thống quản lý dinh dưỡng cá nhân NutriTrack. Nội dung của chương tập trung vào cách các thành phần đã được thiết kế được hiện thực hóa trong mã nguồn, bao gồm phía Backend, phía Frontend, các thuật toán xử lý dinh dưỡng và các cơ chế hỗ trợ trong quá trình vận hành hệ thống.

Về tổng thể, hệ thống được triển khai theo mô hình Client - Server. Frontend đảm nhiệm vai trò cung cấp giao diện tương tác cho người dùng, trong khi Backend chịu trách nhiệm xử lý nghiệp vụ, xác thực người dùng, truy xuất cơ sở dữ liệu và cung cấp dữ liệu thông qua các RESTful API. Cách tổ chức này giúp tách phần giao diện người dùng khỏi phần xử lý nghiệp vụ và lưu trữ dữ liệu, từ đó thuận tiện hơn trong quá trình phát triển, kiểm thử và mở rộng từng thành phần. Cấu trúc này phù hợp với phạm vi của hệ thống do các chức năng như nhật ký ăn uống, theo dõi cân nặng, ghi nhận nước uống, vận động, lập kế hoạch bữa ăn và quét thông tin dinh dưỡng đều cần trao đổi dữ liệu thường xuyên giữa giao diện và máy chủ.

Bên cạnh các chức năng quản lý dữ liệu cơ bản, hệ thống còn triển khai các xử lý tính toán liên quan đến dinh dưỡng cá nhân. Các thuật toán như Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner được tách thành các nhóm xử lý riêng nhằm bảo đảm tính rõ ràng trong mã nguồn và thuận tiện cho việc kiểm tra kết quả. Ngoài ra, chức năng Hybrid Nutrition Scanner được triển khai để hỗ trợ nhận diện và trích xuất thông tin dinh dưỡng từ nhiều nguồn khác nhau, gồm cơ sở dữ liệu cục bộ, nguồn dữ liệu bên ngoài và ảnh nhận sản phẩm.

Các mục tiếp theo của chương lần lượt trình bày cách triển khai Backend, Frontend, các thuật toán cốt lõi, chức năng scanner, cơ chế bảo mật và xử lý lỗi. Việc trình bày được thực hiện theo hướng bám sát mã nguồn thực tế, hạn chế lặp lại phần lý thuyết đã nêu ở Chương 2 và tập trung vào vai trò của từng thành phần trong quá trình vận hành hệ thống.

3.1. Triển khai Backend

Backend của hệ thống NutriTrack được triển khai nhằm đảm nhiệm các xử lý phía máy chủ, bao gồm tiếp nhận yêu cầu từ Frontend, xác thực người dùng, truy xuất cơ sở dữ liệu, xử lý nghiệp vụ và trả kết quả thông qua các RESTful API. Trong kiến trúc tổng thể Client - Server, Backend giữ vai trò là lớp trung gian giữa giao diện người dùng và cơ sở dữ liệu. Các thao tác như đăng nhập, cập nhật hồ sơ, ghi nhật ký ăn uống, theo dõi cân nặng, ghi nhận nước uống, vận động hoặc tạo kế hoạch bữa ăn đều được gửi đến Backend để xử lý và lưu trữ.

Backend được xây dựng bằng Node.js kết hợp Express.js. Express.js được sử dụng

để khai báo middleware, tổ chức route và điều phối HTTP request. Trong quá trình triển khai, mã nguồn Backend không đặt toàn bộ xử lý trong một file duy nhất mà được phân tách thành các lớp route, controller, service và model. Cách tổ chức này giúp giới hạn trách nhiệm của từng thành phần, đồng thời phù hợp với hệ thống có nhiều nhóm xử lý như quản lý nhật ký, tổng hợp dashboard, lập kế hoạch bữa ăn, xuất báo cáo và xử lý dữ liệu sản phẩm.

Đối với NutriTrack, tầng service giữ vai trò quan trọng vì hệ thống không chỉ thực hiện các thao tác thêm, sửa, xóa dữ liệu. Nhiều chức năng cần tổng hợp dữ liệu theo ngày, tính toán chỉ số dinh dưỡng, đối chiếu với mục tiêu cá nhân và tạo kết quả đầu ra cho giao diện. Vì vậy, controller chủ yếu tiếp nhận request và trả response, còn phần xử lý nghiệp vụ được chuyển xuống service để giảm lặp logic giữa các module.

Gợi ý chèn hình 3.1: Sơ đồ kiến trúc phân lớp Backend của NutriTrack, thể hiện luồng Frontend → Route → Middleware → Controller → Service → Sequelize Model → MySQL.

3.1.1. Cấu trúc khởi tạo và middleware

File khởi tạo chính của Backend là app.js. Đây là điểm vào của ứng dụng, chịu trách nhiệm thiết lập Express app, cấu hình middleware, gắn các tuyến API, xử lý lỗi và khởi động kết nối cơ sở dữ liệu. Trong file này, hệ thống sử dụng dotenv để đọc biến môi trường, cors để cấu hình chia sẻ tài nguyên giữa Frontend và Backend, cookie-parser để xử lý cookie và path để phục vụ tài nguyên tĩnh trong môi trường triển khai.

Một điểm triển khai đáng chú ý là Backend có thể phục vụ cả API và bản build của React SPA. Trong môi trường production, thư mục build của Frontend được khai báo là thư mục tĩnh để Express trả về giao diện React. Tuy nhiên, các request bắt đầu bằng /api được xử lý riêng theo cơ chế API. Nếu endpoint API không tồn tại, server trả về phản hồi JSON thay vì trả về file HTML của Frontend. Cách xử lý này giúp phân biệt request dữ liệu với request điều hướng giao diện, đồng thời hạn chế lỗi khi Frontend gọi sai endpoint.

Trong phần xử lý dữ liệu đầu vào, hệ thống sử dụng express.json(limit: '10mb') để đọc dữ liệu JSON từ request body. Giới hạn này được đặt cao hơn mặc định nhằm hỗ trợ các chức năng có thể gửi dữ liệu ảnh ở dạng mã hóa, chẳng hạn quét nhân dinh dưỡng. Bên cạnh đó, express.urlencoded(extended: true) được dùng để xử lý dữ liệu form, còn cookieParser() hỗ trợ đọc Refresh Token từ cookie.

Backend cũng bổ sung response helper cho các request API. Response helper cung cấp các phương thức như res.success, res.error và res.validationError, giúp thống nhất cấu trúc phản hồi giữa các controller. Một response thành công thường có các trường success, data và message; trong khi response lỗi có success: false, thông báo lỗi và có

thể kèm danh sách lỗi kiểm tra dữ liệu. Cách tổ chức này giúp Frontend xử lý kết quả API theo một quy ước chung.

Sau khi cấu hình middleware và route, Backend thực hiện kết nối MySQL thông qua Sequelize. Khi kết nối cơ sở dữ liệu sẵn sàng, hệ thống có thể khởi động các xử lý liên quan, trong đó có tác vụ định kỳ phục vụ Adaptive TDEE. Điều này cho thấy Backend không chỉ phản hồi theo request trực tiếp từ người dùng mà còn có một số xử lý được thực hiện theo lịch.

3.1.2. Tổ chức route và RESTful API

Các API của hệ thống được đặt dưới tiền tố `/api/v1`. Việc sử dụng tiền tố phiên bản giúp cấu trúc API rõ ràng hơn và hỗ trợ khả năng thay đổi trong tương lai khi cần bổ sung hoặc điều chỉnh endpoint. Các route được tổ chức theo từng nhóm chức năng, phản ánh các module chính của hệ thống như xác thực, hồ sơ người dùng, dashboard, nhật ký ăn uống, cân nặng, nước uống, vận động, lập kế hoạch bữa ăn, báo cáo, Adaptive TDEE và scanner.

Trong file route tổng hợp, các nhóm route nghiệp vụ được gắn với middleware xác thực trước khi chuyển đến route chi tiết. Các route như dashboard, diary, weight, water, exercise, profile, meal planner, report, adaptive TDEE và scanner đều yêu cầu người dùng đã đăng nhập. Riêng nhóm route xác thực được tách riêng để phục vụ đăng ký, đăng nhập, đăng xuất, làm mới token và lấy thông tin người dùng hiện tại.

Các phương thức HTTP được sử dụng theo ý nghĩa của từng thao tác. Thao tác đọc dữ liệu sử dụng GET, thao tác tạo dữ liệu sử dụng POST, thao tác cập nhật sử dụng PUT hoặc PATCH, còn thao tác xóa sử dụng DELETE. Cách tổ chức này giúp API dễ kiểm tra, dễ tích hợp với Frontend và giữ được sự thống nhất khi hệ thống mở rộng thêm chức năng.

Bảng 3.1. Một số nhóm API chính của Backend

Nhóm API

Vai trò chính

`/api/v1/auth`

Đăng ký, đăng nhập, đăng xuất, làm mới token

`/api/v1/profile`

Quản lý hồ sơ cá nhân và mục tiêu dinh dưỡng

`/api/v1/dashboard`

Tổng hợp dữ liệu dinh dưỡng, nước uống, vận động và cân nặng

`/api/v1/diary`

Quản lý nhật ký ăn uống và thực phẩm người dùng

`/api/v1/weight, /water, /exercise`

Ghi nhận cân nặng, nước uống và vận động

/api/v1/meal-planner

Cấu hình và tạo kế hoạch bữa ăn

/api/v1/adaptive-tdee

Theo dõi và cập nhật TDEE thích ứng

/api/v1/scanner

Xử lý mã vạch, ảnh nhãn dinh dưỡng và dữ liệu sản phẩm

/api/v1/report

Tổng hợp dữ liệu và xuất báo cáo PDF

Gợi ý chèn hình 3.2: Sơ đồ luồng xử lý một request có xác thực: Frontend gửi request

→ middleware kiểm tra Access Token → controller → service → model truy vấn MySQL
→ response trả về Frontend.

3.1.3. Controller, service và xử lý nghiệp vụ

Controller được sử dụng làm lớp trung gian giữa route và service. Controller nhận dữ liệu từ request, kiểm tra một số điều kiện cơ bản, gọi service tương ứng và trả kết quả về client. Các xử lý phức tạp như tổng hợp dữ liệu, tính toán chỉ số, kiểm tra điều kiện nghiệp vụ hoặc gọi nguồn dữ liệu bên ngoài được chuyển xuống tầng service. Cách phân chia này giúp controller không bị trộn lẫn giữa xử lý HTTP và xử lý nghiệp vụ.

Service layer là nơi chứa phần lớn logic nghiệp vụ của Backend. Các service được tổ chức theo từng nhóm chức năng, chẳng hạn như nutrition service, dashboard service, health insight service, food scoring service, meal planner service, scanner service, physics validation service, Adaptive TDEE service và PDF service. Mỗi service phụ trách một phạm vi xử lý riêng, giúp mã nguồn có ranh giới rõ ràng hơn.

Một ví dụ là chức năng dashboard. Khi người dùng xem dữ liệu trong ngày, Backend không chỉ lấy nhật ký ăn uống mà còn cần tổng hợp năng lượng, protein, carbohydrate, lipid, chất xơ, nước uống, vận động và cân nặng. Các dữ liệu này được xử lý ở service rồi trả về cho Frontend dưới dạng cấu trúc thống nhất. Nếu toàn bộ xử lý đặt trong controller, mã nguồn sẽ khó bảo trì khi chức năng thay đổi.

Tương tự, các chỉ số nền tảng như BMI, BMR, TDEE, năng lượng mục tiêu, phân bố protein, carbohydrate, lipid và mục tiêu nước được đặt trong nhóm xử lý dinh dưỡng. Các kết quả này được sử dụng lại ở nhiều chức năng như hồ sơ người dùng, dashboard, lập kế hoạch bữa ăn và báo cáo PDF. Việc tập trung công thức tại service giúp giảm nguy cơ sai lệch kết quả giữa các module.

3.1.4. Kết nối cơ sở dữ liệu và tổ chức model

Backend sử dụng MySQL làm hệ quản trị cơ sở dữ liệu và Sequelize làm ORM để ánh xạ giữa bảng dữ liệu và đối tượng trong mã nguồn. Sequelize được sử dụng để định

nghĩa model, kiểu dữ liệu, ràng buộc, khóa ngoại và quan hệ giữa các bảng. Cách tiếp cận này giúp các thao tác dữ liệu trong service và controller được viết theo hướng đối tượng, đồng thời vẫn duy trì cấu trúc dữ liệu quan hệ đã thiết kế ở Chương 2.

Các model trong hệ thống phản ánh các thực thể chính của NutriTrack. Nhóm người dùng lưu thông tin tài khoản, hồ sơ cá nhân, chiều cao, cân nặng, giới tính, ngày sinh, mức độ vận động và mục tiêu. Nhóm dữ liệu dinh dưỡng gồm thực phẩm và nhật ký ăn uống. Nhóm theo dõi tiến trình gồm cân nặng, nước uống và vận động. Ngoài ra, hệ thống còn có các model phục vụ scanner, đóng góp dữ liệu sản phẩm và Adaptive TDEE.

Trong các bảng dữ liệu cá nhân, trường userId được sử dụng để liên kết bản ghi với tài khoản người dùng. Đây là yêu cầu quan trọng vì nhật ký ăn uống, cân nặng, nước uống, vận động và lịch sử tính toán đều là dữ liệu riêng của từng người dùng. Các truy vấn trong service cần lọc theo userId để bảo đảm người dùng chỉ truy cập dữ liệu thuộc tài khoản của mình.

Đối với dữ liệu thực phẩm, model không chỉ lưu tên món và năng lượng mà còn lưu nhiều trường dinh dưỡng như protein, carbohydrate, lipid, chất xơ, đường, natri, vitamin và khoáng chất nếu có. Hệ thống cũng phân biệt dữ liệu thực phẩm dùng chung và dữ liệu do người dùng tạo. Cách thiết kế này giúp dữ liệu thực phẩm có thể phục vụ nhiều chức năng như tìm kiếm món ăn, ghi nhật ký, chấm điểm thực phẩm, lập thực đơn và xử lý dữ liệu từ scanner.

Một điểm cần chú ý là nhật ký ăn uống có thể lưu các giá trị dinh dưỡng tại thời điểm ghi nhận. Việc lưu dữ liệu dạng snapshot giúp hạn chế ảnh hưởng khi thông tin thực phẩm gốc thay đổi sau này. Ví dụ, nếu một món ăn được cập nhật lại thành phần dinh dưỡng, các bản ghi nhật ký cũ vẫn giữ giá trị đã được ghi nhận tại thời điểm người dùng thêm món.

3.1.5. Xác thực và tổng hợp xử lý phía Backend

Cơ chế xác thực của Backend được xây dựng dựa trên Access Token và Refresh Token. Khi người dùng đăng nhập, hệ thống kiểm tra thông tin tài khoản, so sánh mật khẩu đã bấm và tạo token phục vụ quá trình truy cập API. Access Token được Frontend gửi kèm trong header Authorization khi gọi các API cần đăng nhập. Refresh Token được lưu trong cookie HttpOnly và được sử dụng khi cần cấp lại Access Token.

Đối với các API cần xác thực, middleware kiểm tra token được thực hiện trước khi request đi vào controller. Nếu token hợp lệ, thông tin người dùng được gắn vào request để các controller và service phía sau sử dụng khi truy vấn dữ liệu. Sau khi xác thực thành công, các thao tác như thêm nhật ký ăn uống, ghi nhận nước uống, thêm vận động hoặc cập nhật cân nặng đều sử dụng userId của người dùng hiện tại để lưu và truy xuất dữ liệu.

Các module nghiệp vụ phía Backend được triển khai xoay quanh hai nhóm nhiệm vụ chính: ghi nhận dữ liệu đầu vào và tính toán dữ liệu tổng hợp. Nhóm ghi nhận dữ liệu gồm hồ sơ cá nhân, nhật ký ăn uống, cân nặng, nước uống và vận động. Nhóm tính toán dữ liệu tổng hợp được thực hiện khi người dùng xem dashboard, tạo báo cáo, lập kế hoạch bữa ăn hoặc sử dụng các chức năng phân tích.

Các module có thuật toán riêng như Nutrition Core, Food Scoring, Health Insights, Meal Planner và Adaptive TDEE được đặt trong service để tách khỏi xử lý HTTP. Trong mục này, các module trên chỉ được trình bày ở mức vai trò trong Backend. Nội dung chi tiết về công thức, quy tắc tính toán và luồng xử lý sẽ được trình bày ở mục 3.3. Tương tự, Hybrid Nutrition Scanner được triển khai thành nhóm route và service riêng; luồng xử lý chi tiết của chức năng này được trình bày ở mục 3.4.

Nhìn chung, Backend của NutriTrack được triển khai theo hướng phân lớp, trong đó Express.js điều phối request, route định nghĩa tài nguyên API, controller xử lý giao tiếp HTTP, service xử lý nghiệp vụ và Sequelize model làm việc với MySQL. Cách tổ chức này phù hợp với phạm vi của hệ thống quản lý dinh dưỡng cá nhân, do hệ thống cần vừa quản lý dữ liệu, vừa tính toán và tổng hợp thông tin theo từng người dùng. Việc tách rõ các thành phần cũng tạo cơ sở để các thuật toán và chức năng mở rộng được trình bày chi tiết ở các mục tiếp theo của Chương 3.

3.2. Triển khai Frontend

Frontend của hệ thống NutriTrack được triển khai dưới dạng ứng dụng React theo mô hình Single Page Application. Phần giao diện chịu trách nhiệm tiếp nhận thao tác của người dùng, gọi API đến Backend, hiển thị dữ liệu dinh dưỡng và phản hồi trạng thái xử lý trong quá trình sử dụng. Trong khi Backend tập trung vào xử lý nghiệp vụ và quản lý dữ liệu, Frontend đảm nhiệm lớp trình bày và tương tác, giúp người dùng thực hiện các chức năng như đăng nhập, cập nhật hồ sơ, ghi nhật ký ăn uống, theo dõi cân nặng, ghi nhận nước uống, vận động, xem dashboard, lập kế hoạch bữa ăn và sử dụng scanner.

Frontend không đảm nhiệm trực tiếp các thuật toán dinh dưỡng phức tạp. Các xử lý như Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner được thực hiện chủ yếu ở Backend để bảo đảm kết quả tính toán thống nhất giữa các màn hình. Frontend nhận dữ liệu đã được xử lý từ API, sau đó tổ chức hiển thị dưới dạng thể thông tin, biểu đồ, bảng dữ liệu, modal nhập liệu và thông báo trạng thái. Cách phân chia này làm rõ ranh giới giữa tầng giao diện và tầng nghiệp vụ, đồng thời hạn chế việc lặp lại công thức ở phía client.

Gợi ý chèn hình 3.x: Sơ đồ tổng quan Frontend của NutriTrack, thể hiện các thành phần chính gồm React App, Router, Auth Context, Custom Hooks, Axios, TanStack Query và Backend API.

3.2.1. Tổ chức ứng dụng React

Ứng dụng Frontend được xây dựng bằng React kết hợp Vite. React được sử dụng để xây dựng giao diện theo component, còn Vite hỗ trợ quá trình phát triển, chạy giao diện cục bộ và tạo bản build khi triển khai. Trong mã nguồn, các thành phần được chia thành nhiều nhóm như pages, components, hooks, contexts và lib. Cách tổ chức này giúp phân tách rõ màn hình chức năng, thành phần giao diện dùng lại, logic gọi API, trạng thái xác thực và cấu hình dùng chung.

File App.jsx giữ vai trò khai báo cấu trúc điều hướng chính của ứng dụng. Tại đây, ứng dụng được bao quanh bởi các provider và thành phần nền như QueryClientProvider, AuthProvider, BrowserRouter và Toaster. QueryClientProvider cung cấp cơ chế quản lý dữ liệu server cho toàn bộ cây component. AuthProvider cung cấp trạng thái đăng nhập và thông tin người dùng. BrowserRouter xử lý điều hướng theo URL, còn Toaster hỗ trợ hiển thị thông báo sau các thao tác như đăng nhập, thêm dữ liệu, cập nhật hoặc xóa bản ghi.

Một điểm đáng chú ý là các trang chính sau đăng nhập được nạp bằng React.lazy. Các màn hình như Dashboard, Diary, Meal Planner, Weight, Exercise, Onboarding và Profile không cần tải đồng thời ngay khi người dùng mở ứng dụng. Khi người dùng truy cập đến route tương ứng, component mới được nạp và trong thời gian chờ hệ thống hiển thị PageLoader. Cách triển khai này giúp chia nhỏ phần giao diện theo route, hạn chế việc tải cùng lúc toàn bộ mã của các màn hình chưa sử dụng.

3.2.2. Điều hướng và kiểm soát truy cập giao diện

Frontend sử dụng React Router để ánh xạ URL với từng màn hình chức năng. Việc sử dụng routing phía client phù hợp với mô hình React SPA, trong đó người dùng có thể chuyển giữa các màn hình mà không cần tải lại toàn bộ trang. Các route được chia thành ba nhóm chính: route công khai, route onboarding và route nghiệp vụ. Route công khai gồm trang giới thiệu, đăng nhập và đăng ký. Route onboarding dành cho người dùng đã đăng nhập nhưng chưa hoàn thành thông tin ban đầu. Các route nghiệp vụ như dashboard, diary, meal planner, weight, exercise và profile yêu cầu người dùng đã đăng nhập và đã hoàn thành onboarding.

Cơ chế kiểm soát truy cập được triển khai thông qua hai component chính là ProtectedRoute và AuthOnlyRoute. ProtectedRoute kiểm tra trạng thái đăng nhập và thông tin người dùng từ AuthContext. Nếu người dùng chưa đăng nhập, giao diện chuyển về trang đăng nhập. Nếu người dùng đã đăng nhập nhưng chưa hoàn thành onboarding, giao diện chuyển sang trang onboarding. Chỉ khi hai điều kiện này hợp lệ, các màn hình nghiệp vụ mới được hiển thị.

AuthOnlyRoute được sử dụng cho trang onboarding. Component này chỉ kiểm tra

người dùng đã đăng nhập hay chưa, không kiểm tra trạng thái đã hoàn thành onboarding. Cách tách riêng này giúp tránh vòng lặp điều hướng giữa onboarding và các route được bảo vệ. Nhờ đó, luồng đăng nhập, hoàn thiện hồ sơ ban đầu và truy cập hệ thống chính được tổ chức rõ ràng hơn.

3.2.3. Quản lý trạng thái xác thực và dữ liệu server

Trạng thái xác thực của người dùng được quản lý thông qua AuthContext. Thay vì truyền thông tin đăng nhập qua nhiều tầng component, trạng thái này được đặt trong context để các thành phần cần thiết có thể truy cập trực tiếp. Các thông tin chính gồm trạng thái đã đăng nhập, trạng thái đang tải, dữ liệu người dùng và các hàm liên quan đến đăng nhập, đăng ký hoặc đăng xuất.

Khi người dùng đăng nhập thành công, Frontend nhận Access Token từ Backend và sử dụng token này trong các request cần xác thực. Refresh Token không được xử lý trực tiếp bằng JavaScript ở Frontend vì token này được Backend lưu trong cookie HttpOnly. Cách phối hợp này giúp Frontend chỉ quản lý Access Token phục vụ gọi API, còn việc làm mới phiên đăng nhập được thực hiện thông qua endpoint refresh token ở Backend.

Đối với dữ liệu lấy từ server, Frontend sử dụng Axios để gửi request HTTP và TanStack Query để quản lý trạng thái dữ liệu. Các logic gọi API được đóng gói trong các custom hook như useDiary, useWeight, useExercise, useMealPlanner và các hook liên quan đến scanner. Component không gọi API trực tiếp rải rác mà sử dụng hook để nhận dữ liệu, trạng thái đang tải, lỗi và các hàm thao tác. Cách tổ chức này giúp mã giao diện dễ đọc hơn và giảm lặp xử lý gọi API giữa các màn hình.

TanStack Query được sử dụng thông qua hai nhóm xử lý chính là useQuery và useMutation. useQuery dùng cho các request lấy dữ liệu, ví dụ lấy nhật ký theo ngày, lấy lịch sử cân nặng, lấy cấu hình Meal Planner hoặc lấy danh sách thực phẩm. useMutation dùng cho các thao tác làm thay đổi dữ liệu, ví dụ thêm món ăn, xóa entry, ghi nhận nước uống, thêm vận động, tạo khẩu phần hoặc đổi nguyên liệu trong Meal Planner.

Sau các thao tác làm thay đổi dữ liệu, Frontend gọi invalidateQueries để cập nhật lại các dữ liệu liên quan. Ví dụ, khi thêm món ăn vào nhật ký, dữ liệu nhật ký và dashboard của ngày hiện tại cần được làm mới. Khi thêm vận động, dữ liệu exercise và dashboard cũng cần được cập nhật. Cách xử lý này giúp dữ liệu giữa các màn hình giữ được sự nhất quán, tránh trường hợp một màn hình đã thay đổi nhưng màn hình khác vẫn hiển thị dữ liệu cũ.

Bảng 3.x. Vai trò của một số thành phần chính trong Frontend

Thành phần

Vai trò triển khai

App.jsx

Khai báo provider, router và cấu trúc route chính của ứng dụng

AuthContext

Quản lý trạng thái đăng nhập và thông tin người dùng

ProtectedRoute

Chặn truy cập vào màn hình nghiệp vụ khi chưa đăng nhập hoặc chưa onboarding

AuthOnlyRoute

Cho phép người dùng đã đăng nhập truy cập trang onboarding

Custom hooks

Đóng gói logic gọi API và xử lý dữ liệu theo từng module

Axios

Gửi HTTP request đến Backend API

TanStack Query

Quản lý dữ liệu server, cache, trạng thái tải và cập nhật lại query

Components

Hiển thị giao diện và xử lý tương tác tại từng màn hình

Gợi ý chèn hình 3.x: Sơ đồ luồng gọi API ở Frontend: Component sử dụng custom hook → TanStack Query quản lý trạng thái → Axios gửi request → Backend API trả response → cache được cập nhật → giao diện hiển thị lại dữ liệu.

3.2.4. Xây dựng giao diện theo component

Giao diện Frontend được tổ chức theo hướng component-based. Mỗi màn hình lớn được chia thành nhiều component nhỏ hơn, tương ứng với từng khối hiển thị hoặc thao tác cụ thể. Dashboard gồm các thẻ tổng quan, biểu đồ, tiến trình nước uống và danh sách insight. Diary gồm danh sách bữa ăn, modal thêm món, tìm kiếm thực phẩm và phần hỗ trợ scanner. Weight và Exercise được chia thành khu vực nhập dữ liệu, biểu đồ và danh sách lịch sử. Meal Planner có các thành phần phục vụ cấu hình bữa ăn, tạo khẩu phần và thay thế nguyên liệu.

Cách chia component giúp giảm kích thước của từng file giao diện và làm rõ phạm vi xử lý của mỗi thành phần. Component nhập cân nặng chỉ xử lý dữ liệu cân nặng, component nước uống xử lý tiến độ và thao tác cộng lượng nước, component insight hiển thị cảnh báo hoặc gợi ý trong ngày. Các phần dùng lại được tách thành component riêng để hạn chế viết lặp giao diện ở nhiều màn hình.

Một số chức năng có xử lý tương tác nhiều được hỗ trợ bằng custom hook. Ví dụ, giao diện tìm kiếm món ăn sử dụng useDebounce để trì hoãn việc gọi API cho đến khi người dùng ngừng nhập trong một khoảng thời gian ngắn. Cách xử lý này giúp giảm số lượng request phát sinh trong quá trình nhập từ khóa. Đối với scanner, các trạng thái như tra cứu mã vạch, xử lý ảnh, xác nhận dữ liệu và thêm vào nhật ký được gom trong hook

riêng, giúp component giao diện tập trung vào luồng hiển thị và thao tác của người dùng.

3.2.5. Liên kết Frontend với Backend

Các màn hình Frontend được triển khai bám theo các module nghiệp vụ của hệ thống. Tuy nhiên, dữ liệu không được truyền trực tiếp giữa các màn hình mà được đồng bộ thông qua API và cache của TanStack Query. Khi người dùng thêm món ăn ở Diary, dữ liệu Dashboard có thể được cập nhật lại thông qua việc làm mới query liên quan. Khi người dùng thay đổi hồ sơ cá nhân, các chức năng phụ thuộc vào thông tin mục tiêu và chỉ số cơ thể có thể lấy lại dữ liệu mới từ Backend.

Cách liên kết này phù hợp với kiến trúc Client - Server, trong đó Backend vẫn là nguồn dữ liệu chính. Frontend không tự quyết định các kết quả tính toán dinh dưỡng quan trọng mà chủ yếu hiển thị dữ liệu, thu thập thao tác người dùng và gửi yêu cầu đến API. Nhờ đó, các màn hình như Dashboard, Diary, Meal Planner, Weight, Exercise, Profile và Scanner có thể hoạt động độc lập về giao diện nhưng vẫn dùng chung nguồn dữ liệu đã được xử lý ở Backend.

Nhìn chung, Frontend của NutriTrack được triển khai theo hướng tách rõ giao diện, điều hướng, trạng thái xác thực và logic gọi API. React Router đảm nhiệm điều hướng theo màn hình, Auth Context quản lý phiên người dùng, TanStack Query quản lý dữ liệu lấy từ server, Axios thực hiện request HTTP, còn các component đảm nhiệm hiển thị và tương tác cụ thể. Các thuật toán dinh dưỡng không được trình bày chi tiết trong mục này vì phần lớn được xử lý ở Backend và sẽ được phân tích ở các mục tiếp theo của Chương 3.

3.2.6. Nutrition Core

Nutrition Core là nhóm xử lý nền tảng của hệ thống NutriTrack, được sử dụng để tính các chỉ số dinh dưỡng cá nhân ban đầu cho người dùng. Vai trò chính của nhóm xử lý này là tạo ra các giá trị cơ sở như BMI, phân loại BMI, BMR, TDEE, năng lượng mục tiêu, phân bổ protein, carbohydrate, lipid và mục tiêu nước uống hằng ngày. Các giá trị này không phải là kết quả phân tích cuối cùng, mà là dữ liệu nền để Dashboard, Profile, Meal Planner, Health Insights, Adaptive TDEE và PDF Report sử dụng trong các bước xử lý tiếp theo.

Trong mã nguồn Backend, Nutrition Core được triển khai tại service xử lý dinh dưỡng. Việc đặt nhóm hàm này trong tầng service giúp các phép tính không phụ thuộc trực tiếp vào request HTTP và có thể được tái sử dụng ở nhiều module khác nhau. Controller chỉ cần gọi service để nhận kết quả đã chuẩn hóa, trong khi Frontend không phải tự viết lại các công thức tính toán. Cách tổ chức này giúp bảo đảm cùng một chỉ số được tính nhất quán trên các màn hình khác nhau.

Dữ liệu đầu vào của Nutrition Core chủ yếu được lấy từ hồ sơ cá nhân của người dùng,

gồm cân nặng, chiều cao, giới tính, ngày sinh, mức độ vận động, mục tiêu cá nhân và tỷ lệ macro tùy chỉnh nếu có. Các dữ liệu này được thu thập trong quá trình onboarding hoặc được cập nhật tại trang hồ sơ cá nhân. Nếu thiếu dữ liệu nền, một số chỉ số có thể không được tính đầy đủ và cần trả về giá trị rỗng để giao diện xử lý.

Bảng 3.x. Dữ liệu đầu vào và kết quả xử lý của Nutrition Core

Dữ liệu đầu vào

Vai trò trong xử lý

Kết quả phụ thuộc

Cân nặng

Tính BMI, BMR và mục tiêu nước

BMI, BMR, waterGoal

Chiều cao

Tính BMI và BMR

BMI, BMR

Giới tính

Chọn công thức BMR tương ứng

BMR

Ngày sinh

Tính tuổi người dùng

BMR

Mức độ vận động

Chọn hệ số vận động

TDEE

Mục tiêu cá nhân

Điều chỉnh năng lượng nạp vào

targetCalories

Tỷ lệ macro tùy chỉnh

Phân bổ năng lượng sang gram

protein, carbs, fat

Chuỗi xử lý của Nutrition Core được thực hiện theo thứ tự: tính tuổi từ ngày sinh, tính BMI, tính BMR, tính TDEE tĩnh, chọn TDEE áp dụng, tính năng lượng mục tiêu, sau đó chuyển năng lượng mục tiêu thành lượng protein, carbohydrate, lipid và mục tiêu nước. Cách xử lý theo chuỗi giúp kết quả đầu ra của bước trước trở thành dữ liệu đầu vào cho bước sau, đồng thời làm rõ quan hệ giữa các chỉ số.

Chỉ số BMI được tính theo công thức: $BMI = W / H^2$. Trong đó, **W** là cân nặng tính bằng kilogram và **H** là chiều cao tính bằng mét. Kết quả BMI có đơn vị kg/m^2 . Trong hệ

thông, BMI được dùng để mô tả sơ bộ tình trạng cơ thể như thiếu cân, bình thường, thừa cân hoặc béo phì. Backend trả về cả giá trị BMI và nhãn phân loại để Frontend hiển thị thống nhất, không cần tự viết lại quy tắc phân loại ở từng màn hình.

BMR biểu diễn mức năng lượng cơ thể tiêu hao trong điều kiện nghỉ ngơi. Trong NutriTrack, BMR được tính theo công thức Mifflin-St Jeor. Với nam giới: **BMR nam = $10 \times W + 6,25 \times H - 5 \times A + 5$** . Với nữ giới: **BMR nữ = $10 \times W + 6,25 \times H - 5 \times A - 161$** . Trong đó, **W** là cân nặng tính bằng kilogram, **H** là chiều cao tính bằng centimet và **A** là tuổi tính theo năm. Việc tách công thức theo giới tính được xử lý ở Backend dựa trên dữ liệu hồ sơ người dùng.

Sau khi có BMR, hệ thống tính TDEE tính theo công thức: **TDEE = BMR $\times$ PAL**. Trong đó, **PAL** là hệ số vận động tương ứng với mức độ vận động do người dùng khai báo. Nếu người dùng bật Adaptive TDEE và đã có giá trị adaptiveTDEE hợp lệ, hệ thống có thể sử dụng adaptiveTDEE làm TDEE áp dụng. Nếu chưa có dữ liệu phù hợp, hệ thống dùng staticTDEE được tính từ BMR và hệ số vận động. Phần tính toán Adaptive TDEE được tách riêng và trình bày ở mục 3.3.2.

Từ TDEE áp dụng, hệ thống tính năng lượng mục tiêu theo mục tiêu cá nhân. Trong phạm vi triển khai của NutriTrack, nếu mục tiêu là giảm cân thì **targetCalories = max(TDEE - 500, BMR)**; nếu mục tiêu là duy trì cân nặng thì **targetCalories = TDEE**; nếu mục tiêu là tăng cân thì **targetCalories = TDEE + 300**. Việc chặn dưới bằng BMR trong trường hợp giảm cân giúp hệ thống không đề xuất mức năng lượng thấp hơn năng lượng nền.

Sau khi có năng lượng mục tiêu, hệ thống chuyển đổi sang lượng protein, carbohydrate và lipid theo đơn vị gram. Tỷ lệ mặc định trong mã nguồn là protein 30%, carbohydrate 40% và lipid 30%. Nếu người dùng thiết lập tỷ lệ riêng trong hồ sơ, hệ thống sử dụng tỷ lệ tùy chỉnh; nếu không, hệ thống quay về tỷ lệ mặc định. Quy tắc chuyển đổi được tính như sau: **proteinGram = targetCalories $\times$ proteinRate / 4**, **carbGram = targetCalories $\times$ carbRate / 4**, **fatGram = targetCalories $\times$ fatRate / 9**. Các hệ số 4, 4 và 9 tương ứng với năng lượng sinh ra từ 1g protein, 1g carbohydrate và 1g lipid.

Bên cạnh macro, Nutrition Core còn tính mục tiêu nước uống theo cân nặng. Trong phạm vi triển khai của hệ thống, mục tiêu nước được tính theo công thức: **waterGoal = W $\times$ 35**. Trong đó, **W** là cân nặng tính bằng kilogram và kết quả có đơn vị ml/ngày. Kết quả này được sử dụng trong chức năng theo dõi nước uống và hiển thị tiến độ hoàn thành trên Dashboard.

Thay vì để các module gọi từng hàm rời rạc, hệ thống có hàm tổng hợp để tính toàn bộ chỉ số dinh dưỡng cho một người dùng. Hàm này trả về các thông tin như tuổi, BMI, phân loại BMI, BMR, staticTDEE, TDEE áp dụng, trạng thái dùng Adaptive TDEE,

năng lượng mục tiêu, lượng macro theo gram, tỷ lệ macro và mục tiêu nước. Các dữ liệu này được dùng trực tiếp cho API hoặc các service khác.

Gợi ý chèn hình 3.x: Sơ đồ luồng xử lý Nutrition Core: Hồ sơ người dùng → BMI/BMR → TDEE tĩnh → chọn TDEE áp dụng → targetCalories → macro và waterGoal → cung cấp dữ liệu cho Dashboard, Meal Planner và PDF Report.

Nhìn chung, Nutrition Core được triển khai như lớp xử lý nền cho toàn bộ hệ thống dinh dưỡng cá nhân. Phạm vi của nhóm xử lý này được giữ ở mức tính toán chỉ số cơ bản và mục tiêu cá nhân. Các xử lý có tính thích ứng theo thời gian, đánh giá chất lượng món ăn, sinh cảnh báo sức khỏe hoặc lập thực đơn không được đặt trong Nutrition Core, mà được tách sang các module chuyên biệt ở các mục tiếp theo.

3.2.7. Adaptive TDEE

Adaptive TDEE là nhóm xử lý được xây dựng để điều chỉnh giá trị TDEE dựa trên dữ liệu thực tế của người dùng theo từng tuần. Trong Nutrition Core, TDEE được tính từ BMR và hệ số vận động do người dùng khai báo. Cách tính này tạo ra giá trị nền ban đầu, tuy nhiên chưa phản ánh đầy đủ sự khác biệt giữa hành vi ăn uống thực tế, mức độ ghi nhận dữ liệu, biến động cân nặng và đặc điểm đáp ứng của từng người dùng. Vì vậy, hệ thống bổ sung cơ chế Adaptive TDEE nhằm ước lượng lại mức tiêu hao năng lượng dựa trên nhật ký ăn uống và dữ liệu cân nặng đã được ghi nhận.

Trong mã nguồn Backend, Adaptive TDEE được triển khai thành một service riêng. Service này không thay thế Nutrition Core mà hoạt động như lớp điều chỉnh bổ sung. Khi chưa đủ dữ liệu, hệ thống vẫn sử dụng TDEE tĩnh. Khi đã có dữ liệu hợp lệ qua nhiều tuần và người dùng bật chế độ sử dụng Adaptive TDEE, giá trị TDEE áp dụng trong hệ thống có thể được lấy từ User.adaptiveTDEE thay vì chỉ lấy từ công thức tĩnh. Cách thiết kế này giúp hệ thống vẫn hoạt động được với người dùng mới, đồng thời có khả năng điều chỉnh khi dữ liệu thực tế đã đủ.

a. Dữ liệu đầu vào và điều kiện tính toán

Adaptive TDEE sử dụng hai nhóm dữ liệu chính: năng lượng nạp vào và biến động cân nặng trong tuần. Năng lượng nạp vào được tổng hợp từ nhật ký ăn uống của người dùng. Các bản ghi món ăn trong từng ngày được cộng lại để tính mức năng lượng trung bình trong tuần. Biến động cân nặng được lấy từ dữ liệu cân nặng mà người dùng ghi nhận trong cùng khoảng thời gian. Từ dữ liệu đầu tuần và cuối tuần, hệ thống xác định mức thay đổi cân nặng theo đơn vị kg.

Khoảng thời gian tính toán được tổ chức theo tuần, bắt đầu từ thứ Hai và kết thúc sau bảy ngày. Việc chọn chu kỳ tuần giúp giảm ảnh hưởng của dao động ngắn hạn trong một ngày, chẳng hạn thay đổi nước, muối hoặc thời điểm cân. Đồng thời, chu kỳ tuần đủ ngắn để hệ thống phản hồi với thay đổi của người dùng, nhưng không quá ngắn đến mức

kết quả bị chi phối bởi một vài bản ghi đơn lẻ.

Bảng 3.x. Dữ liệu đầu vào của Adaptive TDEE

Nhóm dữ liệu

Nguồn dữ liệu

Vai trò trong thuật toán

Nhật ký ăn uống

Các bản ghi món ăn theo ngày

Tính năng lượng nạp trung bình trong tuần

Cân nặng

Các bản ghi cân nặng theo ngày

Xác định cân nặng đầu tuần, cuối tuần và mức thay đổi

Hồ sơ người dùng

Thông tin cá nhân và mục tiêu

Tính TDEE tĩnh, targetCalories và xác định mục tiêu hiện tại

Trạng thái cấu hình

useAdaptiveTDEE

Quyết định có áp dụng Adaptive TDEE vào các chỉ số hay không

Lịch sử Adaptive TDEE

Bảng log theo tuần

Tính rolling TDEE và theo dõi các lần tính trước

Trước khi tính toán, service kiểm tra điều kiện dữ liệu của tuần. Nếu người dùng không nhập đủ nhật ký ăn uống hoặc thiếu dữ liệu cân nặng hợp lệ, kết quả tính có độ tin cậy thấp và không được dùng để cập nhật Adaptive TDEE. Trong trường hợp đó, log của tuần được gán trạng thái `skipped_low_data`. Nếu tuần đã được người dùng chủ động bỏ qua, hệ thống giữ trạng thái `skipped_by_user` và không tính lại. Cơ chế này giúp hệ thống tránh tạo kết quả từ những tuần có dữ liệu thiếu hoặc bất thường.

Khi dữ liệu đủ điều kiện, hệ thống thực hiện tính toán và gán trạng thái `applied` hoặc `clamped` tùy kết quả. Trạng thái `applied` thể hiện kết quả được sử dụng sau khi tính. Trạng thái `clamped` thể hiện kết quả ban đầu vượt khỏi biên cho phép và đã được giới hạn lại. Ngoài ra, log còn có trường `confidence`, được xác định theo số ngày ghi nhận trong tuần. Nếu đủ 7 ngày, độ tin cậy được gán là `high`; nếu có 6 ngày, độ tin cậy là `medium`; nếu chỉ đạt ngưỡng tối thiểu, độ tin cậy là `low`.

Bảng 3.x. Các trạng thái chính của log Adaptive TDEE

Trạng thái

Ý nghĩa

Ảnh hưởng

applied

Dữ liệu đủ điều kiện và kết quả nằm trong biên cho phép

Có thể dùng để tính rolling TDEE

clamped

Kết quả vượt biên và đã được giới hạn lại

Có thể dùng nhưng đã qua bước kiểm soát biên

skipped_low_data

Thiếu nhật ký ăn uống hoặc thiếu dữ liệu cân nặng

Không cập nhật User.adaptiveTDEE

skipped_by_user

Người dùng chủ động bỏ qua tuần đó

Không tính lại tuần đã bỏ qua

b. Công thức tính TDEE thích ứng theo tuần

Phần cốt lõi của Adaptive TDEE là ước lượng TDEE theo tuần từ năng lượng nạp trung bình và biến động cân nặng. Trong phạm vi triển khai của hệ thống, TDEE tuần được tính theo công thức:

$$\text{TDEE tuần} = \text{năng lượng nạp trung bình} - \text{biến động cân nặng} \times 1100$$

Trong đó, năng lượng nạp trung bình được tính theo kcal/ngày, còn biến động cân nặng được tính theo kg/tuần. Hệ số 1100 được sử dụng do 1 kg thay đổi cân nặng được quy đổi xấp xỉ 7700 kcal/tuần, tương ứng khoảng 1100 kcal/ngày khi chia trung bình cho 7 ngày. Khi đưa vào báo cáo hoàn chỉnh, quy đổi này cần được ghi nguồn tham khảo ở phần tài liệu tham khảo nếu chưa được trình bày ở Chương 2.

Nếu người dùng tăng cân, biến động cân nặng mang giá trị dương. Khi đó, TDEE thực tế được ước lượng thấp hơn mức năng lượng nạp, vì một phần năng lượng đã tạo ra tăng cân. Ngược lại, nếu người dùng giảm cân, biến động cân nặng mang giá trị âm. Khi đó, TDEE thực tế được ước lượng cao hơn mức năng lượng nạp, vì cơ thể đã tiêu hao nhiều hơn mức ăn vào. Trường hợp cân nặng không đổi, TDEE thích ứng xấp xỉ bằng mức năng lượng nạp trung bình.

Ví dụ, nếu năng lượng nạp trung bình là 2500 kcal/ngày và cân nặng tăng 0,5 kg trong tuần, TDEE thích ứng được tính là $2500 - 0,5 \times 1100 = 1950$ kcal/ngày. Nếu năng lượng nạp trung bình là 1800 kcal/ngày và cân nặng giảm 0,5 kg trong tuần, kết quả là $1800 - (-0,5 \times 1100) = 2350$ kcal/ngày. Hai trường hợp này cho thấy cùng một lượng calo nạp vào có thể dẫn đến ước lượng TDEE khác nhau tùy theo xu hướng cân nặng thực tế.

Cách tính này không nhằm thay thế hoàn toàn các công thức nền như BMR và TDEE tĩnh. Kết quả Adaptive TDEE chỉ là một ước lượng dựa trên dữ liệu ghi nhận thực tế. Vì dữ liệu đầu vào có thể bị sai lệch do người dùng nhập thiếu món ăn, cân không đều

hoặc có dao động nước trong cơ thể, hệ thống không đưa trực tiếp mọi kết quả vào hồ sơ người dùng mà tiếp tục qua bước kiểm soát biên và làm mượt.

c. Kiểm soát biên và rolling TDEE

Sau khi tính `calculatedTDEE`, hệ thống so sánh giá trị này với TDEE tính lấy từ Nutrition Core. Nếu kết quả nhỏ hơn 70% TDEE tĩnh hoặc lớn hơn 130% TDEE tĩnh, giá trị được giới hạn về biên tương ứng. Khi đó, log được gán trạng thái `clamped`. Quy tắc này nhằm giảm ảnh hưởng của các tuần có dữ liệu bất thường, ví dụ người dùng nhập thiếu năng lượng, cân vào thời điểm khác nhau hoặc có biến động cân nặng chủ yếu do nước.

Sau bước giới hạn biên, hệ thống tính `rollingTDEE`. `Rolling TDEE` được tính bằng cách lấy trung bình giữa TDEE tuần hiện tại và các log hợp lệ trước đó có cùng mục tiêu. Trong quá trình tính `rolling`, hệ thống sử dụng `calculatedTDEE` của các tuần trước, không dùng lại `rollingTDEE`. Quyết định này giúp tránh hiện tượng làm mượt hai lần. Nếu lấy trung bình của các giá trị đã được làm mượt, thuật toán sẽ phản ứng chậm hơn với thay đổi thực tế của người dùng.

Một điểm quan trọng khác là `rolling TDEE` được gán với mục tiêu hiện tại của người dùng. Khi mục tiêu thay đổi, ví dụ từ giảm cân sang duy trì cân nặng, hệ thống không trộn lịch sử của hai mục tiêu khác nhau. Điều này phù hợp với đặc điểm của bài toán, vì hành vi ăn uống, mức năng lượng mục tiêu và cách diễn giải biến động cân nặng có thể khác nhau giữa các mục tiêu.

Kết quả Adaptive TDEE không được cập nhật vào `User.adaptiveTDEE` ngay từ tuần hợp lệ đầu tiên. Hệ thống chỉ cập nhật khi đã có ít nhất hai tuần hợp lệ với trạng thái `applied` hoặc `clamped` cùng mục tiêu. Tuần đầu tiên được xem như dữ liệu nền để theo dõi. Khi chưa đủ số tuần hợp lệ, Nutrition Core vẫn dùng TDEE tĩnh. Cách xử lý này giúp tránh việc một tuần đơn lẻ làm thay đổi mục tiêu năng lượng của toàn hệ thống.

d. Lưu lịch sử, tích hợp và tác vụ định kỳ

Mỗi lần xử lý một tuần, hệ thống tạo mới hoặc cập nhật bản ghi trong bảng Adaptive TDEE Log. Các trường được lưu gồm người dùng, ngày bắt đầu tuần, ngày kết thúc tuần, năng lượng nạp trung bình, số ngày có dữ liệu, cân nặng đầu tuần, cân nặng cuối tuần, biến động cân nặng, TDEE tính được, `rolling TDEE`, TDEE tĩnh, mục tiêu người dùng, năng lượng mục tiêu, trạng thái và độ tin cậy. Việc lưu đầy đủ các trường này giúp hệ thống có khả năng giải thích lại quá trình tính thay vì chỉ lưu một con số cuối cùng.

Adaptive TDEE được tích hợp với Nutrition Core thông qua giá trị TDEE áp dụng. Khi hàm tổng hợp chỉ số dinh dưỡng được gọi, hệ thống kiểm tra người dùng có bật `useAdaptiveTDEE` và có giá trị `adaptiveTDEE` hợp lệ hay không. Nếu có, hệ thống dùng Adaptive TDEE làm TDEE áp dụng. Nếu không, hệ thống dùng TDEE tĩnh. Nhờ vậy,

các module phụ thuộc vào Nutrition Core như Dashboard, Meal Planner và PDF Report có thể sử dụng cùng một giá trị TDEE đang có hiệu lực mà không cần tự xử lý lại logic lựa chọn.

Về vận hành, Adaptive TDEE được triển khai theo cả hai cách: tính tự động theo lịch và tính thủ công. Tác vụ định kỳ chạy vào 00:00 mỗi sáng thứ Hai, lấy danh sách người dùng đang bật Adaptive TDEE, xác định tuần trước đó và gọi service xử lý từng người dùng. Ngoài cron job, Backend còn cung cấp API tính thủ công cho tuần trước để phục vụ kiểm thử hoặc cập nhật lại sau khi người dùng bổ sung dữ liệu. Dù tính tự động hay thủ công, toàn bộ xử lý vẫn đi qua cùng một service, do đó kết quả được tạo theo cùng một quy tắc.

Gợi ý chèn hình 3.x: Sơ đồ luồng Adaptive TDEE gồm các bước: lấy dữ liệu tuần trước → tính calo trung bình → lấy cân nặng đầu/cuối tuần → tính TDEE tuần → giới hạn biên $\pm 30\%$ → tính rolling TDEE → lưu log → cập nhật User.adaptiveTDEE nếu đủ điều kiện → cung cấp lại cho Nutrition Core.

Nhìn chung, Adaptive TDEE được triển khai như cơ chế điều chỉnh TDEE theo dữ liệu thực tế, không thay thế hoàn toàn Nutrition Core. Nutrition Core vẫn cung cấp TDEE tính làm giá trị nền, còn Adaptive TDEE chỉ được áp dụng khi có đủ dữ liệu và người dùng bật cấu hình tương ứng. Các bước kiểm tra số ngày ghi nhận, kiểm tra dữ liệu cân nặng, gán trạng thái log, giới hạn biên so với TDEE tĩnh, làm mượt theo các tuần hợp lệ và chỉ cập nhật sau ít nhất hai tuần hợp lệ giúp kết quả ít bị phụ thuộc vào một bản ghi đơn lẻ hoặc một tuần dữ liệu bất thường.

3.2.8. Food Scoring

Food Scoring là nhóm xử lý được xây dựng để đánh giá chất lượng dinh dưỡng của thực phẩm trong hệ thống NutriTrack. Khác với cách theo dõi chỉ dựa trên tổng năng lượng, module này xem xét mật độ các thành phần dinh dưỡng trên cùng một mức năng lượng, từ đó tạo ra điểm chất lượng và nhãn hiển thị tương ứng. Kết quả chấm điểm không thay thế việc theo dõi calo, mà đóng vai trò bổ sung để phân biệt các thực phẩm có cùng mức năng lượng nhưng khác nhau về thành phần dinh dưỡng.

Trong mã nguồn Backend, Food Scoring được triển khai tại service riêng. Việc tách thuật toán chấm điểm khỏi controller giúp phần xử lý này không phụ thuộc trực tiếp vào request HTTP và có thể tái sử dụng trong các chức năng khác khi cần. Về mặt thiết kế, module này được xem như một lớp đánh giá bổ sung trên dữ liệu thực phẩm, phục vụ cho việc hiển thị, phân tích nhật ký ăn uống hoặc hỗ trợ các module đánh giá khác.

a. Dữ liệu đầu vào và nguyên tắc chuẩn hóa

Dữ liệu đầu vào của Food Scoring là thông tin dinh dưỡng của thực phẩm được lưu trong cơ sở dữ liệu. Các trường chính gồm năng lượng, protein, chất xơ, đường, natri và

một số vi chất như vitamin A, vitamin C, canxi, sắt. Trong đó, năng lượng được dùng làm cơ sở chuẩn hóa. Protein, chất xơ và vi chất là nhóm có thể cộng điểm; đường và natri là nhóm có thể làm giảm điểm nếu mật độ cao.

Trước khi chấm điểm, hệ thống kiểm tra tính hợp lệ của dữ liệu. Nếu thực phẩm không tồn tại, năng lượng không phải là số hoặc năng lượng quá thấp, hệ thống không thực hiện chấm điểm và trả về trạng thái bỏ qua. Việc kiểm tra này giúp tránh tạo điểm chất lượng từ dữ liệu thiếu hoặc không phù hợp.

Điểm chính của Food Scoring là chuẩn hóa các thành phần dinh dưỡng theo 100 kcal. Công thức tổng quát được mô tả như sau:

$$\text{factor} = 100 / \text{calories}$$

$$\text{nutrientPer100Kcal} = \text{nutrient} \times \text{factor}$$

Cách chuẩn hóa này giúp so sánh các thực phẩm có mức năng lượng khác nhau trên cùng một cơ sở. Ví dụ, một thực phẩm có tổng protein cao nhưng năng lượng rất lớn chưa chắc có mật độ protein tốt hơn một thực phẩm có ít protein hơn nhưng năng lượng thấp hơn. Vì vậy, thuật toán không đánh giá thực phẩm theo tổng lượng tuyệt đối, mà dựa trên lượng dinh dưỡng thu được trong 100 kcal.

Bảng 3.x trình bày các nhóm dữ liệu chính được sử dụng trong Food Scoring.

Nhóm dữ liệu

Thành phần

Vai trò

Cơ sở chuẩn hóa

Năng lượng

Quy đổi các chất về mật độ trên 100 kcal

Thành phần cộng điểm chính

Protein, chất xơ

Phản ánh mật độ đậm và chất xơ

Thành phần trừ điểm

Đường, natri

Kiểm soát thành phần cần hạn chế

Vi chất

Vitamin A, vitamin C, canxi, sắt

Bổ sung đánh giá về mật độ vi chất

Dữ liệu phân loại

Loại thực phẩm, nhóm thực phẩm

Xử lý ngoại lệ theo đặc điểm thực phẩm

b. Quy tắc tính điểm

Thuật toán bắt đầu từ điểm nền 50. Sau đó, hệ thống cộng điểm cho các thành phần có lợi và trừ điểm cho các thành phần cần kiểm soát. Điểm cuối cùng được giới hạn trong khoảng từ 0 đến 100 để thuận tiện cho việc hiển thị và phân loại.

Protein và chất xơ là hai nhóm cộng điểm chính. Nếu mật độ của các chất này đạt mức tốt, hệ thống cộng 15 điểm; nếu đạt mức cao hơn, hệ thống cộng 25 điểm. Các vi chất như vitamin A, vitamin C, canxi và sắt có mức tác động nhỏ hơn, thường được cộng 5 hoặc 10 điểm tùy theo ngưỡng đạt được. Cách xử lý này giúp điểm số không chỉ phụ thuộc vào đa lượng mà còn phản ánh một phần giá trị vi chất của thực phẩm.

Đối với đường và natri, hệ thống áp dụng cơ chế trừ điểm theo hai mức. Nếu mật độ vượt ngưỡng cảnh báo, điểm bị trừ 10. Nếu vượt ngưỡng cao hơn, điểm bị trừ 25. Nhờ đó, thuật toán có thể phân biệt giữa trường hợp cần lưu ý và trường hợp có mật độ đường hoặc natri cao.

Thành phần

Điều kiện đánh giá

Tác động đến điểm

Protein

Đạt mức tốt hoặc cao

+15 hoặc +25

Chất xơ

Đạt mức tốt hoặc cao

+15 hoặc +25

Đường

Vượt ngưỡng cảnh báo hoặc cao

-10 hoặc -25

Natri

Vượt ngưỡng cảnh báo hoặc cao

-10 hoặc -25

Vi chất

Đạt mức tốt hoặc cao

+5 hoặc +10

Quy tắc trên giúp điểm số có thể giải thích được. Một thực phẩm đạt điểm cao thường có mật độ protein, chất xơ hoặc vi chất tương đối tốt, đồng thời không có mật độ đường và natri quá cao. Ngược lại, thực phẩm đạt điểm thấp thường do chứa nhiều đường, natri hoặc không cung cấp nhiều thành phần có lợi trên mỗi 100 kcal.

c. Xử lý ngoại lệ và giới hạn điểm

Food Scoring không áp dụng cùng một quy tắc cứng cho mọi loại thực phẩm. Một

số nhóm như thịt, cá, nhóm protein hoặc chất béo không bị đánh giá bất lợi chỉ vì ít chất xơ, vì các nhóm này vốn không phải nguồn cung cấp chất xơ chính. Tương tự, đường trong thực phẩm tự nhiên như trái cây hoặc rau củ cần được phân biệt với đường trong sản phẩm chế biến.

Ngoài cơ chế cộng và trừ điểm, thuật toán còn có bước giới hạn điểm trong một số trường hợp đặc biệt. Nếu thực phẩm quá nghèo vi chất, điểm tối đa có thể bị giới hạn ở mức nhất định. Nếu thực phẩm thiếu chất xơ nghiêm trọng và không thuộc nhóm được miễn xét chất xơ, điểm cũng có thể bị giới hạn. Cách xử lý này giúp tránh trường hợp một thực phẩm đạt điểm cao chỉ nhờ một thành phần đơn lẻ, trong khi tổng thể vẫn nghèo dinh dưỡng.

d. Phân loại kết quả và dữ liệu trả về

Sau khi tính điểm, hệ thống ánh xạ điểm số sang các mức phân loại để Frontend hiển thị. Kết quả trả về không chỉ gồm điểm chất lượng, mà còn có nhãn, cấp độ, biểu tượng huy hiệu, tooltip giải thích và trạng thái bỏ qua nếu dữ liệu không hợp lệ. Nhờ vậy, Frontend không cần tự viết lại logic phân loại và kết quả được giữ thống nhất ở các màn hình.

Khoảng điểm

Mức phân loại

Ý nghĩa hiển thị

Từ 75 trở lên

Lành mạnh

Mật độ dinh dưỡng tốt trong phạm vi thuật toán

Từ 60 đến dưới 75

Khá tốt

Có thành phần có lợi, ít yếu tố bất lợi

Từ 40 đến dưới 60

Trung bình

Cần cân nhắc theo khẩu phần và mục tiêu

Dưới 40

Cần hạn chế

Có dấu hiệu nghèo dinh dưỡng hoặc nhiều thành phần cần kiểm soát

Các mức phân loại này là quy ước nội bộ của hệ thống, không phải kết luận y khoa. Một thực phẩm có điểm thấp không có nghĩa là bị loại bỏ trong mọi trường hợp, và một thực phẩm có điểm cao cũng không tự động phù hợp với mọi mục tiêu. Điểm số chủ yếu đóng vai trò tham khảo trong quá trình so sánh và lựa chọn thực phẩm.

e. Vai trò của Food Scoring trong hệ thống

Food Scoring bổ sung một thước đo định lượng về chất lượng thực phẩm cho Nutri-Track. Trong nhật ký ăn uống, điểm thực phẩm có thể giúp người dùng nhận biết món nào có mật độ dinh dưỡng tốt hơn. Trong Health Insights, kết quả này có thể hỗ trợ phát hiện khẩu phần có nhiều thực phẩm điểm thấp. Trong Meal Planner, điểm thực phẩm có thể được dùng như một tín hiệu phụ khi lựa chọn hoặc thay thế nguyên liệu, bên cạnh các ràng buộc chính như năng lượng mục tiêu, protein, carbohydrate, chất béo và cấu hình bữa ăn.

Gợi ý chèn hình 3.x: Sơ đồ luồng Food Scoring gồm các bước: lấy dữ liệu thực phẩm → kiểm tra dữ liệu hợp lệ → chuẩn hóa trên 100 kcal → cộng điểm cho protein, chất xơ và vi chất → trừ điểm cho đường và natri → áp dụng ngoại lệ và giới hạn điểm → trả về điểm, nhãn và tooltip cho Frontend.

Nhìn chung, Food Scoring được triển khai nhằm bổ sung góc nhìn về mật độ dinh dưỡng của thực phẩm. Thuật toán không chỉ dựa vào tổng năng lượng mà đánh giá thực phẩm theo lượng thành phần có lợi và thành phần cần kiểm soát trên 100 kcal. Việc tách Food Scoring thành service riêng giúp kết quả chấm điểm có thể được sử dụng lại khi cần, đồng thời giữ ranh giới rõ ràng giữa xử lý dữ liệu thực phẩm, phân tích sức khỏe và lập kế hoạch bữa ăn.

3.2.9. Health Insights

Health Insights là nhóm xử lý được xây dựng để tạo ra các nhận xét và gợi ý ngắn dựa trên dữ liệu dinh dưỡng trong ngày của người dùng. Nếu Nutrition Core tạo ra các chỉ số nền như năng lượng mục tiêu, macro và mục tiêu nước, thì Health Insights sử dụng các chỉ số đó để so sánh với dữ liệu thực tế đã ghi nhận. Kết quả của module này không phải là chẩn đoán y khoa, mà là thông tin tham khảo giúp người dùng theo dõi tình trạng ăn uống, nước uống và một số yếu tố cần điều chỉnh trong ngày.

Trong mã nguồn Backend, Health Insights và Daily Scoring được triển khai trong `suggestion.service.js`. Việc đặt nhóm xử lý này ở tầng service giúp các quy tắc phân tích không phụ thuộc trực tiếp vào controller. Backend thực hiện tổng hợp, so sánh và tạo insight, sau đó Frontend chỉ nhận danh sách kết quả để hiển thị trên giao diện. Cách tổ chức này giúp các cảnh báo và điểm sức khỏe được xử lý thống nhất, thay vì để mỗi màn hình tự so sánh dữ liệu theo cách riêng.

a. Dữ liệu đầu vào của Health Insights

Dữ liệu đầu vào của Health Insights gồm dữ liệu thực tế trong ngày và mục tiêu cá nhân của người dùng. Dữ liệu thực tế được tổng hợp từ nhật ký ăn uống và log nước uống, gồm tổng năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri và lượng nước đã uống. Mục tiêu cá nhân được lấy từ Nutrition Core, gồm `targetCalories`, mục tiêu macro và `waterGoal`.

Trong mã nguồn, hàm `getHealthInsights` nhận các nhóm dữ liệu chính như `consumed`, `metrics`, `mealGroups`, `waterTotal`, `waterGoal`, `gender`, `isHistorical` và `clientHour`. Trong đó, `consumed` chứa dữ liệu dinh dưỡng đã tổng hợp; `metrics` chứa mục tiêu cá nhân; `mealGroups` cho biết người dùng đã ghi nhận những bữa nào; `clientHour` hỗ trợ xác định thời điểm trong ngày theo phía người dùng. Đây là các dữ liệu cần thiết để hệ thống không chỉ biết người dùng ăn bao nhiêu, mà còn biết nên đưa ra cảnh báo vào thời điểm nào.

Bảng 3.x trình bày các nhóm dữ liệu đầu vào chính của Health Insights.

Nhóm dữ liệu

Nguồn dữ liệu

Vai trò trong xử lý

Năng lượng và macro

Nhật ký ăn uống

So sánh với `targetCalories` và mục tiêu macro

Chất xơ, đường, natri

Nhật ký ăn uống

Tạo cảnh báo hoặc gợi ý liên quan đến chất lượng khẩu phần

Nước uống

Water log trong ngày

So sánh với mục tiêu nước

Nhóm bữa ăn

Nhật ký ăn uống theo bữa

Xác định ngày đã đủ bữa hay chưa

Giờ phía client

Giao diện người dùng

Hỗ trợ tránh cảnh báo thiếu quá sớm

Giới tính

Hồ sơ người dùng

Chọn một số ngưỡng tham chiếu phù hợp

Nếu người dùng chưa ghi nhận món ăn và cũng chưa ghi nhận nước uống, hệ thống trả về danh sách insight rỗng. Cách xử lý này giúp giao diện không hiển thị nhận xét khi dữ liệu chưa có cơ sở.

b. Cơ chế Triage theo mức đạt calo

Điểm đáng chú ý của Health Insights là cơ chế Triage, tức phân tầng ưu tiên cảnh báo theo mức năng lượng đã nạp trong ngày. Cơ chế này chỉ áp dụng cho nhóm cảnh báo thiếu hụt dinh dưỡng. Các cảnh báo thuộc nhóm thừa, ví dụ thừa calo, thừa đường hoặc

thừa natri, vẫn có thể được kích hoạt khi vượt ngưỡng, không phụ thuộc vào tăng calo hiện tại.

Hệ thống phân chia lượng calo tiêu thụ so với mục tiêu thành ba tầng. Khi năng lượng nạp vào dưới 50% mục tiêu, hệ thống xem đây là tầng sinh tồn. Ở tầng này, các cảnh báo thiếu đa lượng và vi chất được tắt để tránh tạo quá nhiều nhận xét không cần thiết. Hệ thống chỉ ưu tiên cảnh báo rằng lượng ăn đang quá thấp và cần bổ sung năng lượng.

Khi năng lượng đạt từ 50% đến dưới 70% mục tiêu, hệ thống chuyển sang tầng đa lượng. Ở tầng này, nhu cầu năng lượng cơ bản đã được đáp ứng một phần nhưng vẫn còn thiếu. Vì vậy, hệ thống có thể cảnh báo thiếu calo, protein hoặc chất béo. Tuy nhiên, các cảnh báo thiếu vi chất như vitamin, canxi hoặc sắt vẫn chưa được kích hoạt, vì khi tổng lượng ăn còn thấp thì việc thiếu vi chất là kết quả dễ xảy ra và nếu cảnh báo quá sớm sẽ làm danh sách insight bị nhiễu.

Khi năng lượng đạt từ 70% mục tiêu trở lên, hệ thống chuyển sang tầng vi lượng. Lúc này, mức năng lượng và đa lượng đã có đủ cơ sở để đánh giá chi tiết hơn. Hệ thống bắt đầu kiểm tra các yếu tố như chất xơ, canxi, sắt, vitamin C và vitamin A. Cách phân tầng này giúp Health Insights không đưa ra cùng một loại cảnh báo trong mọi thời điểm, mà lựa chọn nội dung phù hợp với mức độ đầy đủ của dữ liệu trong ngày.

Bảng 3.x tóm tắt cơ chế phân tầng trong Health Insights.

Tầng xử lý

Điều kiện theo calo

Cảnh báo thiếu hụt được ưu tiên

Sinh tồn

Dưới 50% mục tiêu

Chỉ cảnh báo lượng ăn quá thấp

Đa lượng

Từ 50% đến dưới 70% mục tiêu

Thiếu calo, protein và chất béo

Vi lượng

Từ 70% mục tiêu trở lên

Chất xơ, canxi, sắt, vitamin C, vitamin A

Cơ chế Triage giúp insight phù hợp hơn với bối cảnh dữ liệu. Nếu người dùng mới ghi nhận một bữa nhỏ, hệ thống không nên lập tức cảnh báo thiếu nhiều loại vi chất. Ngược lại, khi năng lượng trong ngày đã đạt mức tương đối đầy đủ, việc đánh giá chất xơ và vi chất sẽ có cơ sở hơn.

c. Context-Awareness và phân loại insight

Health Insights còn có cơ chế nhận biết ngữ cảnh thời gian. Trong mã nguồn, hệ

thông xác định ngày đã đủ điều kiện đánh giá thiếu hụt hay chưa dựa trên ba yếu tố: dữ liệu thuộc ngày trong quá khứ, thời điểm hiện tại từ client đã từ 20h trở đi, hoặc người dùng đã ghi nhận đủ các bữa chính gồm sáng, trưa và tối. Chỉ khi một trong các điều kiện này thỏa mãn, các cảnh báo thiếu hụt mới được kích hoạt.

Cách xử lý này giúp hạn chế cảnh báo sai thời điểm. Nếu người dùng mới nhập một phần dữ liệu trong ngày, hệ thống không nên kết luận quá sớm rằng khẩu phần thiếu protein, thiếu chất xơ hoặc thiếu vi chất. Ngược lại, các cảnh báo về nhóm thừa như vượt calo, đường hoặc natri cao có thể được kích hoạt sớm hơn, vì chỉ cần đã vượt ngưỡng thì vẫn cần thông báo cho người dùng.

Các insight được chia thành bốn nhóm severity chính: danger, warning, water và suggestion. Nhóm danger dùng cho các vấn đề cần ưu tiên như vượt calo nhiều hoặc chỉ số lệch rõ so với mục tiêu. Nhóm warning dùng cho các lưu ý cần điều chỉnh. Nhóm water dành riêng cho nước uống. Nhóm suggestion chứa các gợi ý cải thiện nhẹ hơn. Danh sách kết quả được sắp xếp theo thứ tự ưu tiên: danger, warning, water, suggestion.

Nhóm insight

Severity

Vai trò hiển thị

Cảnh báo nguy cơ

danger

Ưu tiên hiển thị trước

Cảnh báo cần chú ý

warning

Nhắc người dùng điều chỉnh

Nước uống

water

Theo dõi lượng nước trong ngày

Gợi ý cải thiện

suggestion

Đưa ra hướng điều chỉnh nhẹ

Cấu trúc này phù hợp với Frontend, vì component hiển thị insight có thể dựa vào severity để chọn nhãn, màu sắc và thứ tự hiển thị.

d. Điểm sức khỏe trong ngày

Bên cạnh danh sách insight, hệ thống còn tính điểm sức khỏe trong ngày thông qua hàm calculateDailyHealthScore. Nếu người dùng chưa có dữ liệu ăn uống, điểm trả về là rỗng và giao diện hiển thị trạng thái chưa có dữ liệu. Khi đã có dữ liệu, điểm được tính trong khoảng 0 đến 100 dựa trên mức phù hợp giữa dữ liệu thực tế và mục tiêu cá nhân.

Điểm sức khỏe không chỉ được tính bằng một điều kiện đơn lẻ. Hệ thống trừ điểm theo các vấn đề dinh dưỡng phát hiện được trong danh sách insight, đồng thời áp dụng thêm một số cơ chế điều chỉnh. Nếu năng lượng nạp vào lệch nhiều so với mục tiêu, điểm có thể bị ảnh hưởng bởi hệ số phạt theo mức lệch calo. Nếu lượng đường cao, hệ thống có thêm cơ chế phạt liên quan đến đường. Khi năng lượng đã ở mức đủ nhưng vi chất hoặc chất xơ quá thấp, điểm cuối cùng có thể bị giới hạn bởi cơ chế hard cap.

Cách tính này giúp tránh trường hợp người dùng đạt điểm cao chỉ vì ăn gần đủ calo, trong khi khẩu phần vẫn thiếu chất xơ hoặc vi chất đáng kể. Ngược lại, nếu người dùng đạt một số mục tiêu như calo trong khoảng phù hợp, đủ protein, đủ nước hoặc đủ chất xơ, hệ thống có thể ghi nhận bằng các bonus trong kết quả điểm. Các bonus này được Frontend hiển thị như những nhãn nhỏ trong thẻ điểm sức khỏe.

e. Kết quả đầu ra và vai trò trong hệ thống

Kết quả đầu ra của Health Insights gồm hai phần chính: danh sách insight và điểm sức khỏe trong ngày. Mỗi insight chứa các trường như severity, icon, title và message. Điểm sức khỏe gồm điểm số, nhãn đánh giá, biểu tượng và danh sách bonus nếu có. Frontend sử dụng các dữ liệu này để hiển thị trên Dashboard hoặc các khu vực liên quan đến phân tích trong ngày.

Health Insights đóng vai trò chuyển dữ liệu định lượng thành thông tin diễn giải ngắn. Các con số như tổng calo, lượng protein, chất xơ hoặc nước uống sẽ có ý nghĩa hơn khi được so sánh với mục tiêu cá nhân. Nhờ đó, người dùng không chỉ xem dữ liệu đã ghi nhận mà còn biết chỉ số nào đang cần chú ý hơn trong ngày.

Gợi ý chèn hình 3.x: Sơ đồ luồng Health Insights gồm các bước: tổng hợp dữ liệu trong ngày → lấy mục tiêu từ Nutrition Core → áp dụng Triage theo mức calo → kiểm tra ngưỡng cảnh thời gian → tạo insight theo severity → tính điểm sức khỏe → trả về Dashboard.

Nhìn chung, Health Insights được triển khai như một lớp phân tích nhẹ trên dữ liệu nhật ký dinh dưỡng. Module này không thay thế tư vấn chuyên môn, mà cung cấp các nhận xét tự động dựa trên dữ liệu người dùng đã ghi nhận và mục tiêu cá nhân trong hệ thống. Việc đặt Health Insights trong tầng service giúp các quy tắc phân tích được quản lý tập trung, đồng thời tạo cơ sở để mở rộng thêm điều kiện đánh giá khi hệ thống cần bổ sung chức năng.

3.2.10. Meal Planner

Meal Planner là nhóm xử lý được xây dựng để hỗ trợ tính toán khẩu phần bữa ăn dựa trên mục tiêu năng lượng và các chất dinh dưỡng đa lượng của người dùng. Khác với các chức năng chỉ ghi nhận món ăn sau khi người dùng đã ăn, Meal Planner xử lý theo hướng dự kiến khẩu phần trước, trong đó hệ thống tính toán khối lượng nguyên liệu sao cho bữa ăn bám sát mục tiêu protein, carbohydrate và chất béo đã được phân bổ. Đây là

module có tính toán phức tạp hơn so với các chức năng nhập liệu thông thường, vì một nguyên liệu thường không chỉ chứa một loại chất dinh dưỡng duy nhất.

Trong mã nguồn Backend, Meal Planner được triển khai trong `mealPlanner.service.js`. Module này sử dụng dữ liệu từ Nutrition Core, gồm năng lượng mục tiêu trong ngày và phân bổ macro, sau đó chia mục tiêu này xuống từng bữa ăn theo cấu hình của người dùng. Các nguyên liệu được lựa chọn từ cơ sở dữ liệu thực phẩm, phân theo vai trò như nguồn carbohydrate, nguồn protein, nguồn chất béo và rau hoặc nguồn chất xơ. Kết quả của thuật toán là một cấu trúc khẩu phần gồm danh sách nguyên liệu, khối lượng tương ứng và tổng dinh dưỡng ước tính của bữa ăn.

Điểm cần chú ý là Meal Planner không chỉ chọn món theo tên, mà xử lý trực tiếp trên thành phần dinh dưỡng của nguyên liệu. Cách tiếp cận này phù hợp với mục tiêu theo dõi định lượng của hệ thống, vì khối lượng nguyên liệu có thể được tính toán rõ ràng thay vì chỉ dựa trên khẩu phần ước lượng. Tuy nhiên, kết quả vẫn cần được hiểu là gợi ý kỹ thuật dựa trên dữ liệu dinh dưỡng hiện có, không phải thực đơn bắt buộc cho mọi người dùng.

a. Dữ liệu đầu vào và phân bổ mục tiêu bữa ăn

Dữ liệu đầu vào của Meal Planner gồm mục tiêu dinh dưỡng trong ngày, cấu hình phân bổ bữa ăn, mẫu bữa ăn và danh sách thực phẩm có vai trò phù hợp. Mục tiêu dinh dưỡng trong ngày được lấy từ Nutrition Core, bao gồm `targetCalories`, protein, carbohydrate và chất béo. Cấu hình bữa ăn cho biết mỗi bữa chiếm bao nhiêu phần trăm tổng mục tiêu trong ngày. Từ đó, hệ thống tính mục tiêu năng lượng và macro riêng cho từng bữa.

Nếu người dùng chưa có cấu hình riêng, hệ thống có thể sử dụng cấu hình mặc định gồm bữa sáng 25%, bữa trưa 35%, bữa tối 30% và bữa phụ 10%. Khi cấu hình bữa ăn được cập nhật, Backend cần kiểm tra tổng phần trăm của các bữa phải bằng 100% trước khi lưu. Điều này giúp việc phân bổ năng lượng và macro theo bữa không làm lệch tổng mục tiêu trong ngày.

Ví dụ, nếu một bữa chiếm 30% tổng mục tiêu trong ngày, thì năng lượng, protein, carbohydrate và chất béo của bữa đó cũng được tính theo tỷ lệ tương ứng. Cách làm này giúp Meal Planner không tạo khẩu phần rời rạc, mà luôn bám theo mục tiêu tổng thể của người dùng.

Bảng 3.x trình bày các nhóm xử lý chính trong Meal Planner.

Nhóm xử lý

Vai trò

Meal Target Allocation

Chuyển mục tiêu dinh dưỡng trong ngày thành mục tiêu cho từng bữa

Template Matching

Chọn tổ hợp nguyên liệu theo các vai trò carbohydrate, protein, chất béo và rau

Weight Calculator

Tính khối lượng nguyên liệu để đạt mục tiêu macro của bữa ăn

Edge Case Handling

Kiểm tra nghiệm âm, khối lượng bất thường hoặc tổ hợp không khả thi

Smart Swap

Gợi ý thay thế nguyên liệu khi tổ hợp hiện tại không phù hợp

b. Lựa chọn nguyên liệu theo mẫu bữa ăn

Sau khi có mục tiêu của bữa ăn, hệ thống tiến hành lựa chọn nguyên liệu theo mẫu. Mẫu bữa ăn gồm các vị trí nguyên liệu có vai trò khác nhau, thường gồm nguồn carbohydrate, nguồn protein, nguồn chất béo và rau. Nguồn carbohydrate có thể là cơm, khoai, yến mạch hoặc các thực phẩm giàu tinh bột. Nguồn protein có thể là thịt nạc, cá, trứng hoặc các thực phẩm giàu đạm. Nguồn chất béo có thể là dầu, hạt hoặc thực phẩm giàu chất béo. Rau được sử dụng để bổ sung chất xơ và tạo cấu trúc bữa ăn hợp lý hơn.

Ở tầng API, quá trình tạo khẩu phần nhận các thông tin như mealKey, templateId và nhóm tùy chọn preferences. Trong đó, mealKey xác định bữa đang được tạo, chẳng hạn bữa sáng, trưa, tối hoặc bữa phụ; templateId xác định mẫu bữa ăn; còn preferences phản ánh các lựa chọn bổ sung như nguyên liệu được ghim hoặc nguyên liệu cần loại trừ. Ngoài ra, Backend có thể đưa danh sách dị ứng của người dùng vào danh sách loại trừ trước khi gọi service tạo khẩu phần. Cách xử lý này giúp kết quả tạo ra phù hợp hơn với dữ liệu cá nhân thay vì chỉ chọn ngẫu nhiên từ toàn bộ cơ sở dữ liệu thực phẩm.

Việc lựa chọn nguyên liệu không chỉ dựa vào tên món mà dựa trên vai trò dinh dưỡng của thực phẩm. Cách phân loại này giúp hệ thống tránh tình trạng chọn nhiều nguyên liệu có cùng vai trò, làm mất cân bằng khẩu phần. Ví dụ, nếu một bữa ăn đã có nguồn carbohydrate chính, hệ thống cần ưu tiên chọn thêm nguồn protein và chất béo thay vì tiếp tục chọn một thực phẩm giàu tinh bột khác.

Trong phạm vi thuật toán, rau hoặc nguồn chất xơ được xử lý theo hướng cố định khối lượng trước. Khối lượng rau được đặt ở mức 200g. Lượng protein, carbohydrate và chất béo có trong phần rau này được trừ khỏi mục tiêu của bữa ăn trước khi giải bài toán khối lượng cho ba nguyên liệu còn lại. Cách xử lý này giúp thuật toán giảm số biến cần giải, vì hệ phương trình chính chỉ cần tìm khối lượng cho ba nhóm carbohydrate, protein và chất béo.

c. Tính khối lượng nguyên liệu bằng hệ phương trình tuyến tính

Vấn đề chính của Meal Planner là hiện tượng chồng lấn chất dinh dưỡng đa lượng. Một nguyên liệu không chỉ chứa đúng một loại macro. Ví dụ, gạo hoặc yến mạch có thể chứa chủ yếu carbohydrate nhưng vẫn có một lượng protein nhất định; thịt hoặc trứng

là nguồn protein nhưng có thể chứa chất béo; một số nguồn chất béo cũng có thể chứa protein hoặc carbohydrate. Vì vậy, nếu chỉ lấy mục tiêu carbohydrate chia cho lượng carbohydrate của thực phẩm, kết quả sẽ không phản ánh đầy đủ phần protein và chất béo đi kèm.

Để xử lý vấn đề này, hệ thống mô hình hóa bài toán dưới dạng hệ phương trình tuyến tính. Giả sử ba nguyên liệu chính có khối lượng cần tìm là w_1 , w_2 và w_3 , tính theo đơn vị 100g. Mỗi nguyên liệu có lượng protein, carbohydrate và chất béo trên 100g. Mục tiêu của bữa ăn sau khi đã trừ phần rau là T_p , T_c và T_f . Khi đó, hệ thống cần tìm w_1 , w_2 và w_3 sao cho tổng protein, carbohydrate và chất béo từ ba nguyên liệu khớp với mục tiêu của bữa ăn.

Hệ phương trình được mô tả như sau:

$$p_1 \times w_1 + p_2 \times w_2 + p_3 \times w_3 = T_p$$

$$c_1 \times w_1 + c_2 \times w_2 + c_3 \times w_3 = T_c$$

$$f_1 \times w_1 + f_2 \times w_2 + f_3 \times w_3 = T_f$$

Trong đó, p , c và f lần lượt là lượng protein, carbohydrate và chất béo của từng nguyên liệu trên 100g. Các biến w_1 , w_2 và w_3 là khối lượng cần tìm theo đơn vị 100g. Sau khi giải hệ, hệ thống nhân kết quả với 100 để thu được khối lượng gram cần dùng cho từng nguyên liệu.

Để giải hệ phương trình, Meal Planner sử dụng phương pháp khử Gauss kết hợp chọn phần tử trụ từng phần. Ở mỗi bước, thuật toán chọn dòng có phần tử phù hợp nhất làm phần tử chéo chính nhằm hạn chế trường hợp chia cho 0 hoặc chia cho giá trị quá nhỏ. Nếu hệ phương trình suy biến, nghĩa là các nguyên liệu có tỷ lệ macro quá giống nhau khiến ma trận không giải được ổn định, thuật toán không ép tạo kết quả mà trả về trạng thái không khả thi để thử tổ hợp khác.

Cách giải bằng hệ phương trình tuyến tính giúp Meal Planner xử lý chính xác hơn so với các phương pháp chia đơn giản theo từng chất. Thuật toán không xem nguồn carbohydrate, protein và chất béo là hoàn toàn độc lập, mà tính đến phần chồng lấn giữa các chất trong từng nguyên liệu. Đây là điểm kỹ thuật quan trọng của module này.

d. Kiểm tra ngoại lệ và cơ chế Smart Swap

Kết quả toán học không phải lúc nào cũng phù hợp với thực tế sử dụng. Hệ phương trình có thể cho ra nghiệm âm, hoặc khối lượng nguyên liệu quá nhỏ hay quá lớn. Ví dụ, nếu nguồn protein được chọn có quá nhiều chất béo, thuật toán có thể không tìm được cách vừa đạt mục tiêu protein vừa giữ chất béo trong giới hạn. Trường hợp này cho thấy tổ hợp nguyên liệu hiện tại không phù hợp với mục tiêu bữa ăn.

Để xử lý các tình huống như vậy, Meal Planner có bước kiểm tra ngoại lệ sau khi giải hệ. Nếu nghiệm âm, khối lượng bất thường hoặc tổ hợp không khả thi, hệ thống không

sử dụng kết quả đó làm khẩu phần. Thay vào đó, thuật toán thử chọn tổ hợp nguyên liệu khác trong một số vòng lặp giới hạn. Theo thiết kế của thuật toán, hệ thống có thể thử tối đa 15 lần để tìm tổ hợp phù hợp hơn.

Cơ chế Smart Swap được sử dụng khi nguyên liệu hiện tại làm cho bài toán khó thỏa mãn. Thay vì tự điều chỉnh sai lệch để tạo ra một khẩu phần gần đúng nhưng thiếu tin cậy, hệ thống trả về thông tin lỗi và gợi ý thay thế nguyên liệu phù hợp hơn. Ví dụ, nếu một nguồn protein có quá nhiều chất béo, hệ thống có thể gợi ý chuyển sang nguồn đạm nạc hơn. Cách xử lý này giúp kết quả có tính giải thích: nếu không tạo được khẩu phần hợp lý, hệ thống nêu nguyên nhân và hướng thay thế thay vì trả ra một khẩu phần có số liệu không phù hợp.

Để giảm chi phí truy vấn khi cần tìm các nguồn thay thế, hệ thống có thể sử dụng cache cho danh sách nguyên liệu phù hợp, chẳng hạn danh sách nguồn protein nạc. Việc sử dụng cache giúp hạn chế truy vấn lặp lại trong các vòng thử, đặc biệt khi thuật toán phải kiểm tra nhiều tổ hợp nguyên liệu liên tiếp.

e. Kết quả đầu ra và vai trò trong hệ thống

Kết quả đầu ra của Meal Planner là một cấu trúc dữ liệu mô tả khẩu phần được đề xuất. Kết quả này thường gồm danh sách nguyên liệu, vai trò của từng nguyên liệu, khối lượng tính bằng gram, tổng năng lượng và tổng macro ước tính. Nếu không tìm được tổ hợp khả thi, kết quả có thể chứa thông tin lỗi và gợi ý thay thế để người dùng hoặc giao diện xử lý tiếp.

Meal Planner có liên hệ trực tiếp với Nutrition Core vì cần targetCalories và mục tiêu macro làm dữ liệu đầu vào. Trong phạm vi triển khai hiện tại, Meal Planner ưu tiên các ràng buộc về năng lượng, protein, carbohydrate, chất béo, vai trò nguyên liệu, tính khả thi của khối lượng và dữ liệu thực phẩm hiện có. Food Scoring có thể được xem là hướng mở rộng nếu hệ thống cần bổ sung tiêu chí về chất lượng thực phẩm trong các phiên bản sau.

Gợi ý chèn hình 3.x: Sơ đồ luồng Meal Planner gồm các bước: lấy mục tiêu từ Nutrition Core → phân bổ mục tiêu theo bữa → chọn mẫu nguyên liệu → áp dụng preferences và danh sách loại trừ → cố định phần rau 200g → trừ macro của rau khỏi mục tiêu → lập hệ phương trình 3x3 → giải bằng khử Gauss → kiểm tra nghiệm → Smart Swap nếu không khả thi → trả về khẩu phần.

Nhìn chung, Meal Planner được triển khai như một module tính toán khẩu phần dựa trên mục tiêu dinh dưỡng cá nhân. Điểm chính của module là xử lý hiện tượng chồng lấn macro bằng hệ phương trình tuyến tính, thay vì tính khối lượng từng nguyên liệu bằng các phép chia rời rạc. Việc kết hợp phân bổ mục tiêu theo bữa, lựa chọn nguyên liệu theo vai trò, giải hệ bằng khử Gauss và kiểm tra ngoại lệ giúp hệ thống tạo ra khẩu phần có cơ

sở tính toán rõ ràng. Các trường hợp không khả thi được xử lý bằng cơ chế Smart Swap, qua đó hạn chế việc trả về kết quả không phù hợp với mục tiêu dinh dưỡng.

3.3. Hybrid Nutrition Scanner

Hybrid Nutrition Scanner là chức năng hỗ trợ người dùng ghi nhận thông tin dinh dưỡng của thực phẩm đóng gói thông qua hai hướng xử lý: tra cứu bằng mã vạch và trích xuất thông tin từ ảnh nhãn dinh dưỡng. Chức năng này được xây dựng nhằm giảm thao tác nhập liệu thủ công khi người dùng cần ghi lại các chỉ số như năng lượng, protein, carbohydrate, chất béo, chất xơ, đường và natri. Nếu phải nhập từng chỉ số bằng tay, người dùng dễ mất thời gian và có thể nhập sai dữ liệu, làm ảnh hưởng đến kết quả tổng hợp trong nhật ký ăn uống.

Trong hệ thống NutriTrack, scanner không chỉ là một chức năng quét đơn lẻ mà được triển khai theo hướng kết hợp nhiều nguồn dữ liệu. Với các sản phẩm phổ biến, mã vạch giúp hệ thống tra cứu nhanh thông tin đã có trong cơ sở dữ liệu hoặc nguồn bên ngoài. Với các sản phẩm chưa có dữ liệu, người dùng có thể chụp ảnh bảng thông tin dinh dưỡng để AI Vision trích xuất số liệu. Cách kết hợp này giúp hệ thống xử lý được cả sản phẩm đã biết và sản phẩm mới, đồng thời tạo cơ chế bổ sung dữ liệu vào cơ sở dữ liệu sản phẩm nội bộ.

a. Luồng tra cứu mã vạch nhiều tầng

Khi người dùng nhập hoặc quét mã vạch, Backend gọi service scanner để xử lý. Trước hết, mã vạch được chuẩn hóa để tránh sai lệch do khoảng trắng hoặc định dạng nhập không thống nhất. Sau đó, hệ thống thực hiện tra cứu theo nhiều tầng ưu tiên. Cách thiết kế này giúp tận dụng dữ liệu cục bộ trước, chỉ gọi nguồn bên ngoài khi dữ liệu nội bộ chưa có kết quả phù hợp.

Tầng đầu tiên là cơ sở dữ liệu cục bộ với các sản phẩm đã được xác thực. Đây là nguồn được ưu tiên cao nhất vì có tốc độ phản hồi nhanh và dữ liệu đã có mức tin cậy tốt hơn. Nếu tìm thấy sản phẩm có trạng thái verified và confidenceScore đạt ngưỡng phù hợp, hệ thống có thể trả dữ liệu ngay cho người dùng mà không cần gọi API bên ngoài.

Nếu không tìm thấy sản phẩm đã xác thực, hệ thống tiếp tục kiểm tra các sản phẩm cục bộ chưa được xác thực hoàn toàn. Đây có thể là dữ liệu do người dùng hoặc cộng đồng đóng góp trước đó. Dữ liệu ở tầng này vẫn có giá trị tham khảo, nhưng cần hiển thị kèm trạng thái và điểm tin cậy để người dùng biết rằng sản phẩm chưa được kiểm chứng đầy đủ.

Nếu cơ sở dữ liệu cục bộ không có kết quả phù hợp, Backend gọi Open Food Facts API để tra cứu theo mã vạch. Khi nguồn này trả về dữ liệu hợp lệ, hệ thống lưu lại sản phẩm vào cơ sở dữ liệu cục bộ với nguồn dữ liệu tương ứng. Nhờ vậy, các lần tra cứu sau có thể lấy từ cơ sở dữ liệu nội bộ, giảm số lần gọi API bên ngoài.

Nếu cả cơ sở dữ liệu cục bộ và Open Food Facts đều không tìm thấy dữ liệu, hệ thống trả về trạng thái không tìm thấy. Khi đó, Frontend có thể chuyển người dùng sang luồng AI Vision để chụp ảnh bảng dinh dưỡng. Luồng này đóng vai trò dự phòng cho những sản phẩm chưa có trong cơ sở dữ liệu mã vạch.

Bảng 3.x. Luồng tra cứu mã vạch trong Hybrid Nutrition Scanner

Tầng xử lý

Nguồn dữ liệu

Điều kiện sử dụng

Kết quả

Local Verified

Cơ sở dữ liệu cục bộ

Sản phẩm đã xác thực, điểm tin cậy đạt ngưỡng

Trả sản phẩm ngay

Local Unverified

Dữ liệu cộng đồng

Sản phẩm có dữ liệu nhưng chưa xác thực đầy đủ

Trả sản phẩm kèm trạng thái tin cậy

Open Food Facts API

API bên ngoài

Local DB không có kết quả phù hợp

Lấy dữ liệu và lưu lại vào Local DB

AI Vision Fallback

Ảnh nhãn dinh dưỡng

Không tìm thấy bằng mã vạch

Trích xuất dữ liệu từ ảnh

b. Xử lý ảnh bằng AI Vision

Trong trường hợp mã vạch không tìm thấy sản phẩm, người dùng có thể chụp ảnh bảng thông tin dinh dưỡng. Ảnh được gửi lên Backend dưới dạng dữ liệu base64. Ở tầng ứng dụng, Express được cấu hình giới hạn kích thước JSON lớn hơn thông thường để có thể nhận ảnh từ chức năng AI Vision. Sau khi nhận ảnh, controller gọi service Gemini Vision để trích xuất dữ liệu dinh dưỡng từ ảnh.

Kết quả AI Vision thường gồm tên sản phẩm nếu đọc được, năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri và một số vi chất nếu có. Dữ liệu này cần được chuẩn hóa về cùng dạng trường mà hệ thống sử dụng. Ví dụ, đơn vị có thể được xác định là 100g hoặc 100ml tùy theo nhãn sản phẩm. Việc chuẩn hóa giúp dữ liệu từ AI có thể dùng chung với dữ liệu từ mã vạch và cơ sở dữ liệu thực phẩm.

Một điểm quan trọng là Backend không mặc định xem dữ liệu AI là chính xác tuyệt đối. Ảnh có thể bị mờ, thiếu sáng, lệch góc hoặc bảng dinh dưỡng có cách trình bày khác nhau. Vì vậy, sau khi AI Vision trả về kết quả, Frontend hiển thị biểu mẫu để người dùng kiểm tra và xác nhận lại các chỉ số đã trích xuất. Chỉ khi người dùng xác nhận, dữ liệu mới được gửi đến API đóng góp để Backend kiểm tra và lưu vào cơ sở dữ liệu. Cách xử lý này tạo thêm một lớp kiểm soát giữa kết quả AI và dữ liệu được lưu chính thức.

c. Physics Validation và kiểm tra tính hợp lý

Physics Validation là bước kiểm tra nhằm phát hiện các dữ liệu dinh dưỡng bất thường. Mục tiêu của bước này không phải là chứng minh dữ liệu chắc chắn đúng, mà là loại bỏ hoặc cảnh báo những kết quả có khả năng sai rõ ràng. Điều này đặc biệt cần thiết đối với dữ liệu do AI trích xuất, vì AI có thể đọc nhầm số, nhầm đơn vị hoặc nhầm cột trên bao bì.

Nhóm kiểm tra đầu tiên liên quan đến khối lượng. Với dữ liệu tính trên 100g hoặc 100ml, tổng protein, carbohydrate và chất béo không được vượt quá 100g. Nếu tổng ba thành phần này vượt quá giới hạn vật lý của khẩu phần, dữ liệu có dấu hiệu bất thường và không nên được chấp nhận trực tiếp. Ngoài ra, hệ thống còn kiểm tra một số trường hợp cần cảnh báo như chất xơ lớn hơn carbohydrate hoặc đường lớn hơn carbohydrate, vì một số nhãn dinh dưỡng có thể trình bày carbohydrate theo cách khác nhau.

Nhóm kiểm tra thứ hai liên quan đến năng lượng. Với dữ liệu tính trên 100g, năng lượng không được vượt quá 900 kcal, vì chất béo là nhóm sinh năng lượng cao nhất với khoảng 9 kcal/g. Bên cạnh đó, hệ thống kiểm tra chéo năng lượng bằng công thức Atwater:

$$\text{caloriesFromMacro} = \text{protein} \times 4 + \text{carbohydrate} \times 4 + \text{fat} \times 9$$

Nếu năng lượng đọc được lệch quá nhiều so với năng lượng tính từ protein, carbohydrate và chất béo, hệ thống đánh dấu lỗi hoặc cảnh báo để người dùng kiểm tra lại. Trường hợp năng lượng đọc được lớn hơn năng lượng ước tính khoảng 4 lần có thể gợi ý khả năng nhãn đang ghi theo đơn vị kJ thay vì kcal. Nhờ đó, người dùng có cơ sở sửa lại dữ liệu trước khi xác nhận.

Việc đưa Physics Validation vào luồng scanner giúp giảm nguy cơ cơ sở dữ liệu bị nhiễu bởi các kết quả AI sai. Nếu dữ liệu không hợp lệ, hệ thống có thể trả về kết quả kiểm tra để giao diện thông báo cho người dùng. Nếu dữ liệu vượt qua kiểm tra, hệ thống mới tiếp tục xử lý lưu sản phẩm và ghi nhận đóng góp.

d. Lưu đóng góp và cập nhật điểm tin cậy

Sau khi dữ liệu AI được người dùng xác nhận và vượt qua kiểm tra hợp lý, Backend tạo hoặc cập nhật bản ghi sản phẩm trong bảng sản phẩm quét. Với sản phẩm mới do AI Vision và người dùng xác nhận, dữ liệu được lưu ở trạng thái unverified với

confidenceScore ban đầu khoảng 0.3. Đồng thời, hệ thống ghi nhận một bản ghi trong ProductContribution để lưu lại dữ liệu mà người dùng đã xác nhận.

Cơ chế đóng góp giúp hệ thống không chỉ phục vụ một lượt quét đơn lẻ, mà còn cải thiện dữ liệu sản phẩm theo thời gian. Khi nhiều người dùng cùng đóng góp dữ liệu cho một mã vạch, hệ thống có thể so sánh mức độ tương đồng giữa các lượt đóng góp. Nếu dữ liệu giữa các lượt không lệch quá nhiều, confidenceScore có thể được tăng lên. Khi đạt mức tin cậy phù hợp, sản phẩm có thể được chuyển sang trạng thái verified để được ưu tiên ở tầng Local Verified trong các lần tra cứu sau.

Ngược lại, nếu người dùng báo cáo sản phẩm sai hoặc các lượt đóng góp có sự khác biệt lớn, hệ thống có thể hạ mức tin cậy hoặc gắn trạng thái tranh chấp. Cách thiết kế này tạo ra một vòng cải thiện dữ liệu: sản phẩm chưa có trong cơ sở dữ liệu có thể được thêm qua AI Vision; các lượt đóng góp tiếp theo giúp kiểm tra lại dữ liệu; sản phẩm đủ tin cậy được ưu tiên trả về ở tầng tra cứu cục bộ.

e. Kết quả trả về và vai trò trong hệ thống

Kết quả trả về của scanner thường gồm trạng thái tìm thấy, nguồn dữ liệu, thông tin sản phẩm và điểm tin cậy. Thông tin sản phẩm được chuẩn hóa trước khi trả về API, gồm mã vạch, tên sản phẩm, năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri, vi chất nếu có, ảnh sản phẩm, đơn vị, trạng thái và nguồn dữ liệu. Frontend sử dụng kết quả này để hiển thị cho người dùng kiểm tra trước khi thêm vào nhật ký ăn uống.

Hybrid Nutrition Scanner có vai trò kết nối giữa dữ liệu sản phẩm đóng gói và các module dinh dưỡng khác. Sau khi sản phẩm được xác nhận và thêm vào nhật ký, dữ liệu năng lượng và chất dinh dưỡng sẽ ảnh hưởng đến Dashboard, Health Insights, PDF Report và các thống kê theo ngày. Vì vậy, việc kiểm tra dữ liệu scanner không chỉ quan trọng cho riêng chức năng quét, mà còn ảnh hưởng đến độ chính xác của toàn bộ hệ thống theo dõi dinh dưỡng.

Gợi ý chèn hình 3.x: Sơ đồ luồng Hybrid Nutrition Scanner gồm các bước: quét mã vạch → tìm Local Verified → tìm Local Unverified → gọi Open Food Facts và lưu lại nếu có → nếu không tìm thấy thì chuyển AI Vision → trích xuất dữ liệu từ ảnh → người dùng kiểm tra và xác nhận → Physics Validation → lưu ProductContribution → cập nhật confidenceScore → thêm sản phẩm vào nhật ký.

Nhìn chung, Hybrid Nutrition Scanner được triển khai như một luồng xử lý kết hợp giữa tra cứu mã vạch, nguồn dữ liệu bên ngoài, AI Vision và cơ chế kiểm tra độ tin cậy. Thiết kế nhiều tầng giúp hệ thống ưu tiên dữ liệu nhanh và tin cậy trước, đồng thời vẫn có phương án xử lý khi gặp sản phẩm mới. Physics Validation, bước xác nhận của người dùng và cơ chế cập nhật confidenceScore giúp hạn chế việc lưu dữ liệu bất thường, từ đó

làm cho chức năng scanner phù hợp hơn với mục tiêu theo dõi dinh dưỡng cá nhân của NutriTrack.

3.4. Bảo mật và xử lý lỗi

Bảo mật và xử lý lỗi là phần hỗ trợ quan trọng trong quá trình vận hành Backend của hệ thống NutriTrack. Do hệ thống lưu trữ dữ liệu cá nhân liên quan đến hồ sơ người dùng, nhật ký ăn uống, cân nặng và mục tiêu dinh dưỡng, các API cần có cơ chế xác thực người dùng, giới hạn truy cập và trả lỗi nhất quán. Trong phạm vi triển khai, hệ thống tập trung vào một số nhóm xử lý chính: xác thực bằng token, bảo vệ mật khẩu, giới hạn request, chuẩn hóa phản hồi API và xử lý lỗi tập trung.

Đối với xác thực, Backend sử dụng cơ chế Access Token và Refresh Token. Khi người dùng đăng nhập thành công, hệ thống tạo Access Token để Frontend gửi kèm trong header Authorization khi gọi các API cần xác thực. Refresh Token có thời gian sống dài hơn và được lưu trong HttpOnly Cookie. Việc đặt Refresh Token trong HttpOnly Cookie giúp hạn chế việc JavaScript phía Frontend truy cập trực tiếp vào token này. Ngoài ra, cookie refresh token được cấu hình đường dẫn riêng cho endpoint làm mới token, nhờ đó giảm phạm vi gửi cookie trong các request không cần thiết.

Các nhóm API nghiệp vụ như Dashboard, Diary, Weight, Water, Exercise, Meal Planner, Profile, Report, Adaptive TDEE và Scanner được bảo vệ bằng middleware requireAuthApi. Middleware này kiểm tra Access Token trong header trước khi cho phép controller xử lý request. Khi Access Token hết hạn, Frontend có thể gọi endpoint refresh token để lấy Access Token mới dựa trên Refresh Token trong cookie. Cách tổ chức này giúp tách rõ token dùng để gọi API hàng ngày và token dùng để duy trì phiên đăng nhập.

Mật khẩu người dùng không được lưu trực tiếp dưới dạng văn bản gốc. Khi đăng ký, Backend sử dụng bcrypt để băm mật khẩu trước khi lưu vào cơ sở dữ liệu. Khi đăng nhập, mật khẩu người dùng nhập vào được so sánh với giá trị đã băm. Nếu email không tồn tại hoặc mật khẩu không khớp, hệ thống trả về thông báo chung như “Email hoặc mật khẩu không đúng” thay vì nêu rõ sai email hay sai mật khẩu. Cách xử lý này giúp giảm khả năng suy đoán tài khoản hợp lệ từ phản hồi của hệ thống.

Bên cạnh xác thực, hệ thống áp dụng giới hạn request cho các route đăng nhập và đăng ký. Trong mã nguồn, nhóm route này được giới hạn 20 request trong 15 phút cho mỗi IP. Nếu vượt quá số lần cho phép, server trả về lỗi yêu cầu thử lại sau. Ngoài ra, chức năng AI Vision trong Scanner cũng có giới hạn request riêng theo người dùng, nhằm hạn chế việc gửi quá nhiều ảnh trong thời gian ngắn.

Bảng 3.x. Một số cơ chế bảo mật và xử lý lỗi trong Backend

Cơ chế

Vai trò

Cách triển khai

Access Token

Xác thực request cần đăng nhập

Gửi trong header Authorization

Refresh Token

Làm mới phiên đăng nhập

Lưu trong HttpOnly Cookie

requireAuthApi

Bảo vệ API nghiệp vụ

Kiểm tra token trước khi vào controller

bcrypt

Bảo vệ mật khẩu người dùng

Băm mật khẩu trước khi lưu

Rate limit

Hạn chế request bất thường

Giới hạn login/register theo IP

AI Vision rate limit

Hạn chế gửi ảnh quá nhiều

Giới hạn request scanner theo người dùng

Response helper

Chuẩn hóa phản hồi API

Dùng res.success, res.error, res.validationError

Error handler

Xử lý lỗi chưa bắt

Ghi log và trả lỗi máy chủ nội bộ

Đối với xử lý lỗi, Backend sử dụng response helper cho toàn bộ các request bắt đầu bằng /api. Khi xử lý thành công, response có trường success: true, kèm dữ liệu và thông báo. Khi xảy ra lỗi, response có trường success: false và thông báo lỗi. Với lỗi kiểm tra dữ liệu đầu vào, hệ thống có thể trả thêm danh sách errors để Frontend hiển thị lỗi theo từng trường. Cách tổ chức này giúp các controller không phải tự viết cấu trúc phản hồi lặp lại và giúp Frontend xử lý kết quả API thống nhất hơn.

Hệ thống cũng phân biệt các nhóm lỗi HTTP thường gặp. Lỗi dữ liệu đầu vào có thể trả về mã 400 hoặc 422; lỗi xác thực trả về 401; lỗi xung đột dữ liệu như email đã tồn tại trả về 409; lỗi vượt quá giới hạn request trả về 429; lỗi máy chủ trả về 500. Ngoài ra, Backend có cơ chế xử lý riêng cho các API không tồn tại. Mọi request bắt đầu bằng /api

nhưng không khớp route nào sẽ nhận phản hồi JSON báo endpoint không tồn tại, tránh trường hợp Frontend gọi sai API nhưng lại nhận về file HTML của React SPA.

Ở cuối luồng xử lý, Backend có error handler tổng quát để bắt các lỗi chưa được xử lý trong route hoặc controller. Error handler ghi log lỗi ở phía server và trả về thông báo lỗi máy chủ nội bộ cho client. Việc không trả trực tiếp stack trace về Frontend giúp hạn chế lộ thông tin triển khai. Nhìn chung, các cơ chế bảo mật và xử lý lỗi trong NutriTrack được triển khai ở mức phù hợp với một hệ thống quản lý dinh dưỡng cá nhân, giúp bảo vệ phiên đăng nhập, hạn chế request bất thường và duy trì cấu trúc phản hồi API nhất quán.

Gợi ý chèn hình 3.x: Có thể chèn sơ đồ ngắn về luồng xác thực: đăng nhập → kiểm tra mật khẩu → tạo Access Token và Refresh Token → gửi Access Token cho Frontend → lưu Refresh Token trong HttpOnly Cookie → làm mới Access Token khi cần.

3.5. Kết luận chương

Chương 3 đã trình bày quá trình xây dựng và triển khai hệ thống NutriTrack từ kiến trúc tổng thể đến các module xử lý cụ thể. Về phía Backend, hệ thống được tổ chức theo hướng phân lớp, trong đó route định nghĩa tài nguyên API, controller tiếp nhận và trả phản hồi HTTP, service xử lý nghiệp vụ, còn model làm việc với cơ sở dữ liệu MySQL thông qua Sequelize. Cách tổ chức này giúp tách biệt giữa xử lý giao tiếp và xử lý nghiệp vụ, đặc biệt phù hợp với hệ thống có nhiều chức năng tính toán dinh dưỡng.

Về phía Frontend, hệ thống sử dụng React để xây dựng giao diện dạng SPA. Các thành phần như React Router, Auth Context, Axios và TanStack Query được sử dụng để quản lý điều hướng, xác thực, gọi API và đồng bộ dữ liệu. Các màn hình như Dashboard, Diary, Profile, Meal Planner và Scanner được triển khai theo hướng component hóa, giúp giao diện được tổ chức rõ ràng và thuận tiện hơn trong quá trình bảo trì.

Chương này cũng đã trình bày các module xử lý cốt lõi của hệ thống, gồm Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner. Các module này không hoạt động rời rạc mà liên kết với nhau thông qua dữ liệu mục tiêu cá nhân, nhật ký ăn uống, cân nặng, nước uống và thông tin thực phẩm. Bên cạnh đó, Hybrid Nutrition Scanner hỗ trợ thu thập dữ liệu sản phẩm đóng gói thông qua mã vạch và AI Vision, kết hợp với cơ chế kiểm tra hợp lý dữ liệu trước khi lưu. Chức năng xuất báo cáo PDF cũng được triển khai nhằm tổng hợp dữ liệu theo khoảng thời gian phục vụ theo dõi và đánh giá.

Ngoài các chức năng nghiệp vụ, hệ thống còn có các cơ chế bảo mật và xử lý lỗi như xác thực bằng Access Token và Refresh Token, băm mật khẩu, bảo vệ API nghiệp vụ, giới hạn request ở các luồng nhạy cảm, chuẩn hóa phản hồi API, xử lý API 404 và error handler tổng quát. Những thành phần này giúp hệ thống duy trì cấu trúc phản hồi nhất

quán, hạn chế lỗi trong quá trình sử dụng và bảo vệ các API cần xác thực.

Nhìn chung, Chương 3 đã làm rõ cách các thiết kế ở Chương 2 được hiện thực hóa trong mã nguồn. Đây là cơ sở để Chương 4 tiếp tục trình bày quá trình kiểm thử, thực nghiệm và đánh giá kết quả hoạt động của hệ thống.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Trên cơ sở các nội dung phân tích, thiết kế và xây dựng hệ thống đã trình bày ở các chương trước, chương này trình bày quá trình thực nghiệm và đánh giá hệ thống NutriTrack. Mục tiêu của chương là kiểm tra khả năng hoạt động của các chức năng chính, đánh giá kết quả xử lý của một số thuật toán được sử dụng trong hệ thống và xác định các hạn chế còn tồn tại sau quá trình triển khai.

Nội dung thực nghiệm được thực hiện trên môi trường cục bộ, bao gồm phía backend, frontend và cơ sở dữ liệu. Các nhóm chức năng được kiểm thử tập trung vào những nghiệp vụ chính của hệ thống, gồm quản lý tài khoản, ghi nhận nhật ký dinh dưỡng, theo dõi chỉ số cơ thể, thống kê dashboard, đánh giá chất lượng món ăn, đề xuất bữa ăn, quét thông tin dinh dưỡng và sinh báo cáo. Bên cạnh kiểm thử chức năng, chương này cũng trình bày kết quả thực nghiệm đối với các thành phần xử lý cốt lõi như Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner.

Việc đánh giá được thực hiện dựa trên ba nhóm tiêu chí chính. Thứ nhất, hệ thống cần phản hồi đúng với dữ liệu đầu vào và trả về kết quả phù hợp với từng trường hợp kiểm thử. Thứ hai, giao diện người dùng cần hiển thị đúng dữ liệu nhận được từ máy chủ và cập nhật trạng thái sau các thao tác thêm, sửa, xóa. Thứ ba, các thuật toán tính toán cần cho kết quả nhất quán với dữ liệu đầu vào, đồng thời phù hợp với mục tiêu theo dõi và quản lý dinh dưỡng cá nhân.

Kết quả của chương này là cơ sở để nhận xét mức độ hoàn thành của hệ thống, chỉ ra các hạn chế còn tồn tại và đề xuất hướng phát triển trong các phiên bản tiếp theo.

4.1. Môi trường thực nghiệm

Mục tiêu của phần thực nghiệm là kiểm tra khả năng vận hành của hệ thống NutriTrack trong điều kiện triển khai cục bộ, bao gồm phía máy chủ, giao diện người dùng và cơ sở dữ liệu. Môi trường thực nghiệm được thiết lập theo mô hình client-server. Trong đó, backend cung cấp các API phục vụ xử lý nghiệp vụ, xác thực người dùng, quản lý nhật ký dinh dưỡng, tính toán chỉ số sức khỏe và sinh báo cáo. Frontend được sử dụng để thao tác trực tiếp với các chức năng của hệ thống thông qua trình duyệt web. Cơ sở dữ liệu MySQL được dùng để lưu trữ thông tin người dùng, món ăn, nhật ký ăn uống, cân nặng, nước uống, luyện tập và các kết quả tính toán liên quan.

Hệ thống được kiểm thử trên môi trường phát triển cục bộ nhằm kiểm tra khả năng hoạt động của các chức năng chính trước khi đánh giá kết quả thực nghiệm. Backend được khởi chạy bằng Node.js, sử dụng Express để định nghĩa API và Sequelize ORM để thao tác với cơ sở dữ liệu MySQL. Frontend được xây dựng bằng React và Vite, sử dụng Axios để gửi yêu cầu HTTP đến backend, TanStack Query để quản lý dữ liệu lấy từ máy chủ và React Router để điều hướng giữa các trang chức năng. Các chức năng yêu cầu

đăng nhập được kiểm thử thông qua cơ chế xác thực bằng access token và refresh token.

Bảng 4.1. Môi trường phần mềm và công cụ thực nghiệm

Thành phần

Công nghệ/Công cụ sử dụng

Vai trò trong thực nghiệm

Hệ điều hành

Windows 10/11

Môi trường chạy ứng dụng và công cụ phát triển

Trình soạn thảo

Visual Studio Code

Chỉnh sửa và quản lý mã nguồn

Backend

Node.js, Express

Cung cấp API và xử lý nghiệp vụ phía máy chủ

Cơ sở dữ liệu

MySQL, Sequelize ORM

Lưu trữ và truy vấn dữ liệu hệ thống

Frontend

React, Vite

Xây dựng giao diện và điều hướng chức năng

Giao tiếp API

Axios, JSON

Trao đổi dữ liệu giữa frontend và backend

Quản lý dữ liệu giao diện

TanStack Query

Lấy dữ liệu, cập nhật cache và đồng bộ trạng thái

Xác thực

JWT, refresh token, cookie HttpOnly

Kiểm tra các chức năng yêu cầu đăng nhập

Sinh báo cáo

PDFKit

Kiểm tra chức năng tạo báo cáo PDF

Kiểm thử API

Postman/Developer Tools

Gửi request và quan sát phản hồi API

Kiểm thử giao diện

Chrome/Edge

Thao tác trực tiếp với các chức năng hệ thống

Dữ liệu thực nghiệm được chuẩn bị theo các nhóm chức năng chính của NutriTrack. Nhóm dữ liệu người dùng gồm giới tính, chiều cao, cân nặng, tuổi, mức độ vận động và mục tiêu cá nhân. Nhóm dữ liệu dinh dưỡng gồm các món ăn có thông tin năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri và một số vi chất. Nhóm dữ liệu nhật ký gồm bản ghi ăn uống theo ngày, lượng nước uống, cân nặng và hoạt động luyện tập. Các dữ liệu này được sử dụng để kiểm tra luồng tính toán chỉ số dinh dưỡng, tổng hợp dashboard, đánh giá chất lượng món ăn, đề xuất bữa ăn và sinh báo cáo PDF.

Trong quá trình thực nghiệm, hệ thống được đánh giá chủ yếu ở ba khía cạnh. Thứ nhất, các API cần trả về đúng cấu trúc dữ liệu và mã trạng thái tương ứng với từng trường hợp kiểm thử. Thứ hai, giao diện cần hiển thị đúng dữ liệu nhận được từ máy chủ và cập nhật lại sau khi người dùng thêm, sửa hoặc xóa bản ghi. Thứ ba, các thuật toán tính toán cần cho kết quả nhất quán với dữ liệu đầu vào, đặc biệt đối với các chức năng như Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner. Các kết quả kiểm thử chi tiết được trình bày trong các mục tiếp theo của chương này.

4.2. Kiểm thử chức năng

Kiểm thử chức năng được thực hiện nhằm đánh giá khả năng đáp ứng của hệ thống NutriTrack đối với các nghiệp vụ chính đã xây dựng. Phạm vi kiểm thử tập trung vào những chức năng có sự tương tác trực tiếp giữa người dùng, giao diện frontend, API backend và cơ sở dữ liệu. Các trường hợp kiểm thử được lựa chọn theo luồng sử dụng thực tế của hệ thống, bao gồm xác thực tài khoản, thiết lập hồ sơ cá nhân, ghi nhận nhật ký dinh dưỡng, theo dõi nước uống, cân nặng, luyện tập, thống kê dashboard, lập kế hoạch bữa ăn, quét thông tin sản phẩm và xuất báo cáo.

Quá trình kiểm thử được thực hiện trên môi trường cục bộ. Backend được khởi chạy bằng Node.js và cung cấp các API theo tiền tố /api/v1. Các nhóm route nghiệp vụ như /dashboard, /diary, /weight, /water, /exercise, /meal-planner, /profile, /report, /adaptive-tdee và /scanner yêu cầu người dùng đăng nhập trước khi truy cập. Riêng nhóm chức năng xác thực tại /auth cho phép người dùng đăng ký, đăng nhập, đăng xuất, làm mới access token và lấy thông tin tài khoản hiện tại. Cấu trúc phản hồi của API được kiểm tra theo dạng JSON thống nhất, trong đó phản hồi thành công có trường success: true, còn phản hồi lỗi có trường success: false kèm thông báo tương ứng.

Phương pháp kiểm thử được thực hiện theo hai hướng. Thứ nhất, các API được kiểm tra trực tiếp bằng công cụ gửi request để xác nhận mã trạng thái, dữ liệu trả về và khả năng xử lý lỗi đầu vào. Thứ hai, các chức năng được kiểm tra thông qua giao diện người dùng để đối chiếu trạng thái hiển thị sau các thao tác thêm, sửa hoặc xóa dữ liệu. Trạng

thái “Đạt” trong bảng kiểm thử được ghi nhận khi chức năng trả về phản hồi phù hợp với kết quả mong đợi trong phạm vi dữ liệu thực nghiệm đã chuẩn bị.

Bảng 4.2. Kết quả kiểm thử chức năng chính của hệ thống

Nhóm chức năng

Trường hợp kiểm thử

Kết quả cần đạt

Trạng thái

Xác thực tài khoản

Đăng ký và đăng nhập bằng thông tin hợp lệ

Hệ thống trả về thông tin người dùng, access token và thiết lập phiên đăng nhập

Đạt

Xác thực tài khoản

Đăng nhập bằng email hoặc mật khẩu sai

Hệ thống từ chối đăng nhập và trả về thông báo lỗi

Đạt

Bảo vệ API

Gọi API yêu cầu đăng nhập nhưng không gửi access token hợp lệ

Hệ thống không cho phép truy cập tài nguyên được bảo vệ

Đạt

Hồ sơ người dùng

Thiết lập giới tính, ngày sinh, chiều cao, cân nặng, mức độ vận động và mục tiêu

Hồ sơ được lưu, trạng thái onboarding được cập nhật

Đạt

Dashboard

Lấy dữ liệu tổng quan theo ngày

Hệ thống trả về tổng năng lượng, chất dinh dưỡng chính, nước uống, luyện tập và chỉ số liên quan

Đạt

Nhật ký dinh dưỡng

Thêm hoặc xóa món ăn theo ngày, bữa ăn và khẩu phần

Bản ghi được cập nhật, tổng dinh dưỡng trong ngày được tính lại

Đạt

Thực phẩm cá nhân

Tạo, sửa hoặc xóa món ăn do người dùng tự nhập

Hệ thống chỉ xử lý dữ liệu thuộc người dùng hiện tại

Đạt

Nước uống và luyện tập

Ghi nhận lượng nước uống, mục tiêu nước và hoạt động luyện tập

Dữ liệu được lưu, tổng lượng nước hoặc năng lượng tiêu hao được cập nhật

Đạt

Cân nặng

Ghi nhận cân nặng hợp lệ, dữ liệu ngoài khoảng cho phép hoặc ngày trong tương lai

Dữ liệu hợp lệ được lưu; dữ liệu không hợp lệ bị từ chối

Đạt

Meal Planner

Sinh gợi ý bữa ăn và thay thế nguyên liệu

Hệ thống trả về danh sách nguyên liệu, khẩu phần và giá trị dinh dưỡng ước tính

Đạt

Scanner

Tra cứu barcode hoặc xử lý ảnh thông tin dinh dưỡng

Hệ thống trả về thông tin sản phẩm hoặc yêu cầu xác nhận khi dữ liệu chưa đủ chắc chắn

Đạt

Báo cáo PDF

Xuất báo cáo theo khoảng thời gian tuần hoặc tháng

File PDF được sinh từ dữ liệu người dùng và trả về cho trình duyệt

Đạt

Xử lý lỗi

Gửi dữ liệu sai, thiếu token, gọi sai endpoint hoặc gửi request vượt giới hạn

Hệ thống trả về phản hồi lỗi dạng JSON và không ghi dữ liệu không hợp lệ

Đạt

Đối với nhóm chức năng xác thực, hệ thống được kiểm thử với các tình huống đăng ký tài khoản mới, đăng nhập tài khoản đã tồn tại, đăng nhập sai thông tin và làm mới access token. Khi đăng nhập thành công, backend trả về access token trong phần thân phản hồi và thiết lập refresh token dưới dạng HttpOnly Cookie. Frontend sử dụng access token để gọi các API yêu cầu đăng nhập. Khi token không hợp lệ hoặc không được gửi kèm request, các route được bảo vệ không trả về dữ liệu nghiệp vụ. Ngoài ra, các trường hợp lỗi như đăng nhập sai, thiếu dữ liệu đầu vào hoặc gọi endpoint không tồn tại đều được phản hồi bằng cấu trúc JSON thống nhất.

Đối với nhóm hồ sơ người dùng, kiểm thử tập trung vào luồng onboarding và cập nhật thông tin cá nhân. Người dùng cần cung cấp các thông tin cơ bản như giới tính, ngày sinh, chiều cao, cân nặng, mức độ vận động và mục tiêu. Đây là các dữ liệu đầu

vào cần thiết để hệ thống tính BMI, BMR, TDEE, mục tiêu năng lượng và mục tiêu nước. Khi dữ liệu hợp lệ, thông tin được lưu vào cơ sở dữ liệu và trạng thái hoàn thành onboarding được chuyển sang true. Khi dữ liệu nằm ngoài khoảng cho phép, hệ thống trả về thông báo lỗi và không cập nhật bản ghi.

Đối với nhóm nhật ký dinh dưỡng và dashboard, kiểm thử được thực hiện bằng cách thêm món ăn vào từng bữa trong ngày, thay đổi khẩu phần và xóa bản ghi đã thêm. Khi một món ăn được ghi nhận, hệ thống lưu khẩu phần, bữa ăn, ngày sử dụng và các giá trị dinh dưỡng tại thời điểm thêm. Cách lưu này giúp dữ liệu nhật ký giữ nguyên giá trị đã ghi nhận, kể cả khi thông tin món ăn trong cơ sở dữ liệu được chỉnh sửa sau đó. Sau khi nhật ký thay đổi, dashboard cần cập nhật lại tổng năng lượng, protein, carbohydrate, chất béo, lượng nước, năng lượng tiêu hao từ luyện tập và các chỉ số cơ thể liên quan. Kết quả kiểm thử cho thấy dữ liệu hiển thị trên dashboard thay đổi tương ứng với các bản ghi đã thêm hoặc xóa trong nhật ký.

Gợi ý chèn **Hình 4.2. Màn hình nhật ký dinh dưỡng sau khi thêm món ăn vào bữa trong ngày.**

Đối với nhóm cân nặng, nước uống và luyện tập, kiểm thử được thực hiện theo các thao tác ghi nhận dữ liệu hàng ngày. Chức năng cân nặng cho phép thêm hoặc cập nhật cân nặng theo ngày, đồng thời đồng bộ cân nặng hiện tại của người dùng theo bản ghi mới nhất. Các trường hợp nhập cân nặng ngoài khoảng cho phép, chọn ngày trong tương lai hoặc nhập giá trị chênh lệch lớn so với xu hướng gần nhất được xử lý bằng thông báo lỗi hoặc yêu cầu xác nhận. Chức năng nước uống cho phép ghi nhận lượng nước theo đơn vị ml và cập nhật mục tiêu nước/ngày. Chức năng luyện tập cho phép chọn môn thể thao, nhập thời lượng và tính năng lượng tiêu hao dựa trên cân nặng người dùng. Sau mỗi thao tác hợp lệ, dữ liệu tổng hợp trong ngày được cập nhật để giao diện hiển thị lại trạng thái mới.

Đối với nhóm Meal Planner, kiểm thử tập trung vào các thao tác lấy cấu hình bữa ăn, cập nhật cấu hình, sinh gợi ý bữa ăn và thay thế nguyên liệu. Kết quả trả về cần có cấu trúc rõ ràng để frontend hiển thị, bao gồm danh sách nguyên liệu, khối lượng hoặc khẩu phần ước tính, tổng năng lượng và các chất dinh dưỡng chính. Trường hợp thay thế nguyên liệu được kiểm tra nhằm bảo đảm hệ thống vẫn trả về một phương án bữa ăn có thể hiển thị sau khi người dùng thay đổi thành phần trong gợi ý ban đầu.

Gợi ý chèn **Hình 4.3. Màn hình gợi ý bữa ăn từ chức năng Meal Planner.**

Đối với nhóm Hybrid Nutrition Scanner, các trường hợp kiểm thử gồm tra cứu sản phẩm bằng barcode, giải mã barcode từ ảnh, trích xuất thông tin dinh dưỡng từ ảnh, xác nhận đóng góp dữ liệu sản phẩm, tải ảnh sản phẩm và báo cáo dữ liệu sản phẩm sai. Khi dữ liệu ảnh hoặc barcode không đủ điều kiện xử lý, hệ thống trả về thông báo lỗi thay vì

ghi nhận dữ liệu chưa được xác nhận. Đối với dữ liệu sản phẩm mới, hệ thống yêu cầu người dùng xác nhận trước khi lưu đóng góp vào cơ sở dữ liệu. Cách xử lý này giúp hạn chế việc lưu các bản ghi sản phẩm thiếu thông tin hoặc chưa đủ độ tin cậy.

Đối với chức năng xuất báo cáo, hệ thống được kiểm thử với dữ liệu mẫu trong phạm vi 7 ngày và 30 ngày. Báo cáo PDF cần tổng hợp được thông tin người dùng, chỉ số cơ thể, năng lượng, chất dinh dưỡng chính, nước uống, luyện tập, biến động cân nặng và dữ liệu Adaptive TDEE nếu có. Kết quả kiểm thử cho thấy báo cáo có thể được sinh theo khoảng thời gian được chọn và trả về dưới dạng file để người dùng xem hoặc tải xuống từ trình duyệt.

Gợi ý chèn **Hình 4.4. Màn hình báo cáo hoặc file PDF được sinh từ dữ liệu người dùng.**

Nhìn chung, các nhóm chức năng chính của hệ thống đã đáp ứng được các luồng kiểm thử cơ bản trong phạm vi môi trường cục bộ. Các route yêu cầu đăng nhập chỉ được xử lý khi request có access token hợp lệ. Các thao tác thêm, sửa, xóa dữ liệu được kiểm tra theo phạm vi người dùng hiện tại, qua đó hạn chế truy cập dữ liệu của tài khoản khác. Các lỗi thường gặp như thiếu dữ liệu đầu vào, dữ liệu ngoài khoảng cho phép, token không hợp lệ, endpoint không tồn tại và gửi request vượt giới hạn đều được phản hồi bằng cấu trúc JSON thống nhất. Kết quả kiểm thử chức năng là cơ sở để tiếp tục đánh giá các thuật toán xử lý chính trong mục 4.3.

4.3. Thực nghiệm các thuật toán chính

Bên cạnh kiểm thử chức năng, hệ thống NutriTrack còn được thực nghiệm trên các thuật toán xử lý chính nhằm đánh giá khả năng tính toán và phản hồi theo dữ liệu đầu vào. Phạm vi thực nghiệm tập trung vào các thành phần có ảnh hưởng trực tiếp đến kết quả hiển thị cho người dùng, gồm Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights, Meal Planner và Hybrid Nutrition Scanner. Các thuật toán này không được đánh giá như mô hình học máy, mà được kiểm tra theo hướng thuật toán nghiệp vụ. Theo đó, dữ liệu đầu vào cần được xử lý đúng quy tắc, kết quả trả về cần nhất quán và các trường hợp ngoại lệ cần được kiểm soát.

Dữ liệu thực nghiệm được chuẩn bị theo các tình huống thường gặp trong quá trình sử dụng hệ thống. Với Nutrition Core, dữ liệu đầu vào gồm giới tính, tuổi, chiều cao, cân nặng, mức độ vận động và mục tiêu cá nhân. Với Adaptive TDEE, dữ liệu đầu vào gồm nhật ký năng lượng và cân nặng theo tuần. Với Food Scoring, dữ liệu thực phẩm được kiểm tra theo các chỉ số năng lượng, protein, carbohydrate, chất béo, chất xơ, đường và natri. Với Health Insights, dữ liệu nhật ký trong ngày được dùng để sinh cảnh báo hoặc gợi ý. Với Meal Planner, hệ thống được thử với các mục tiêu năng lượng và macro khác nhau. Với Hybrid Nutrition Scanner, dữ liệu thực nghiệm gồm barcode và ảnh bảng

thành phần dinh dưỡng.

Bảng 4.3. Kết quả thực nghiệm các thuật toán chính

Thuật toán

Dữ liệu đầu vào

Kết quả cần kiểm tra

Kết quả thực nghiệm

Nutrition Core

Hồ sơ người dùng, mức độ vận động, mục tiêu

Tính BMI, BMR, TDEE, macro và mục tiêu nước

Trả về đủ các chỉ số nền tảng cho Dashboard, Profile, Meal Planner và PDF Report

Adaptive TDEE

Năng lượng nạp vào và cân nặng theo tuần

Ước lượng TDEE từ dữ liệu thực tế

Tính được TDEE theo tuần và có giới hạn khi dữ liệu vượt ngưỡng cho phép

Food Scoring

Năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri

Chấm điểm chất lượng món ăn

Điểm thay đổi theo mật độ protein, chất xơ, đường và natri

Health Insights

Tổng năng lượng, macro, nước uống, luyện tập

Sinh cảnh báo hoặc gợi ý theo ngưỡng

Tạo được cảnh báo thiếu nước, thiếu protein, dư năng lượng hoặc mất cân đối macro

Meal Planner

Mục tiêu năng lượng, macro và cấu hình bữa ăn

Sinh khẩu phần phù hợp với mục tiêu

Trả về nguyên liệu, khẩu phần ước tính và tổng dinh dưỡng của bữa ăn

Hybrid Nutrition Scanner

Barcode hoặc ảnh nhãn dinh dưỡng

Trích xuất và chuẩn hóa dữ liệu sản phẩm

Trả về sản phẩm hợp lệ hoặc yêu cầu xác nhận khi dữ liệu chưa đủ điều kiện

Đối với Nutrition Core, kết quả thực nghiệm cho thấy các chỉ số nền tảng được tính và tái sử dụng ở nhiều chức năng khác nhau. Khi hồ sơ cá nhân được cập nhật, hệ thống tính BMI, BMR, TDEE, năng lượng mục tiêu, tỷ lệ protein, carbohydrate, chất béo và mục tiêu nước. Các giá trị này được sử dụng trong Dashboard để so sánh với dữ liệu ăn uống thực tế, trong Meal Planner để xác định mục tiêu bữa ăn và trong PDF Report để

tổng hợp kết quả theo khoảng thời gian. Việc dùng chung một lớp xử lý giúp hạn chế sai khác giữa các màn hình chức năng.

Đối với Adaptive TDEE, thực nghiệm được thực hiện với dữ liệu cân nặng và năng lượng trong nhiều ngày. Một trường hợp kiểm thử sử dụng dữ liệu 7 ngày, trong đó cân nặng giảm từ 80,2 kg xuống 79,5 kg và năng lượng trung bình đạt khoảng 1970 kcal/ngày. Hệ thống ghi nhận TDEE tính là 2500 kcal/ngày, Adaptive TDEE là 2200 kcal/ngày và mức chênh lệch là 300 kcal/ngày, tương ứng 12%. Kết quả này cho thấy thuật toán có thể phản ánh sự khác biệt giữa công thức tính và dữ liệu thực tế. Ngoài ra, thuật toán có cơ chế giới hạn kết quả trong khoảng cho phép so với TDEE tính nhằm giảm ảnh hưởng của dữ liệu nhập sai.

Đối với Food Scoring và Health Insights, kết quả thực nghiệm cho thấy hệ thống có thể chuyển dữ liệu dinh dưỡng thành điểm số, cảnh báo và gợi ý ngắn gọn. Trong phạm vi dữ liệu thử nghiệm, các món có mật độ protein hoặc chất xơ cao, đồng thời có lượng đường và natri thấp, nhận điểm tốt hơn. Ngược lại, món có nhiều đường, natri hoặc năng lượng cao nhưng ít chất dinh dưỡng có xu hướng bị giảm điểm. Với Health Insights, khi dữ liệu nhật ký trong ngày thay đổi, danh sách cảnh báo cũng thay đổi theo. Các cảnh báo như thiếu nước, thiếu protein, dư năng lượng hoặc mất cân đối macro được sinh ra dựa trên các ngưỡng điều kiện đã định nghĩa trong backend.

Đối với Meal Planner, thực nghiệm tập trung vào khả năng tạo khẩu phần theo mục tiêu năng lượng và macro của bữa ăn. Hệ thống trả về danh sách nguyên liệu, khối lượng hoặc khẩu phần ước tính, tổng năng lượng và các chất dinh dưỡng chính. Khi tổ hợp nguyên liệu không tạo được kết quả hợp lệ, hệ thống không ghi nhận khẩu phần sai lệch mà trả về thông báo để người dùng thay đổi lựa chọn. Đối với Hybrid Nutrition Scanner, hai luồng chính được kiểm tra gồm tra cứu barcode và nhận dạng ảnh nhãn dinh dưỡng. Khi barcode khớp với dữ liệu đã có, hệ thống trả về thông tin sản phẩm tương ứng. Khi dữ liệu được trích xuất từ ảnh còn thiếu hoặc không đạt điều kiện kiểm tra hợp lệ, hệ thống yêu cầu xác nhận trước khi lưu vào cơ sở dữ liệu.

Gợi ý chèn **Hình 4.5. Kết quả thực nghiệm Meal Planner hoặc Adaptive TDEE trên giao diện hệ thống.**

Nhìn chung, các thuật toán chính đã xử lý được các tình huống dữ liệu cơ bản và trả về kết quả phù hợp với mục tiêu của từng chức năng. Nutrition Core đóng vai trò tính toán nền tảng, Adaptive TDEE hỗ trợ điều chỉnh theo dữ liệu thực tế, Food Scoring đánh giá chất lượng món ăn, Health Insights chuyển dữ liệu thành cảnh báo, Meal Planner tạo khẩu phần theo mục tiêu và Scanner hỗ trợ thu thập dữ liệu thực phẩm. Kết quả thực nghiệm cũng cho thấy chất lượng dữ liệu đầu vào vẫn là yếu tố ảnh hưởng trực tiếp đến kết quả xử lý.

4.4. Đánh giá kết quả

Trong phạm vi kiểm thử cục bộ, hệ thống NutriTrack đã đáp ứng được các yêu cầu cơ bản của một hệ thống quản lý dinh dưỡng cá nhân. Các chức năng chính như xác thực tài khoản, quản lý hồ sơ, ghi nhận nhật ký ăn uống, theo dõi cân nặng, nước uống, luyện tập, thống kê dashboard, lập kế hoạch bữa ăn, quét thông tin sản phẩm và xuất báo cáo đều có thể thực hiện theo luồng nghiệp vụ đã thiết kế. Dữ liệu sau khi được thêm, sửa hoặc xóa được cập nhật lại trên các màn hình liên quan, qua đó bảo đảm sự liên kết giữa giao diện, API và cơ sở dữ liệu.

Bảng 4.4. Tổng hợp đánh giá kết quả thực nghiệm

Tiêu chí đánh giá

Kết quả ghi nhận

Nhận xét

Tính đầy đủ chức năng

Các nhóm chức năng chính đã được triển khai và kiểm thử

Đáp ứng phạm vi chức năng của đề án

Tính nhất quán dữ liệu

Dữ liệu nhật ký, dashboard, thuật toán và báo cáo có liên kết với nhau

Hạn chế sai khác giữa các màn hình chức năng

Phản hồi API

API trả về dữ liệu và lỗi theo cấu trúc JSON thống nhất

Thuận lợi cho frontend xử lý và hiển thị

Kết quả thuật toán

Các thuật toán chính trả về kết quả theo dữ liệu đầu vào

Phù hợp với mục tiêu hỗ trợ theo dõi dinh dưỡng cá nhân

Xuất báo cáo

Báo cáo PDF tổng hợp được dữ liệu trong phạm vi 7 ngày và 30 ngày

Phù hợp cho chức năng xem lại quá trình theo dõi

Kết quả thực nghiệm cho thấy điểm chính của hệ thống nằm ở khả năng liên kết nhiều nhóm dữ liệu trong cùng một luồng xử lý. Thông tin hồ sơ người dùng được sử dụng để tính các chỉ số nền tảng; dữ liệu nhật ký được dùng để cập nhật dashboard; dữ liệu nhiều ngày tiếp tục được sử dụng cho Adaptive TDEE và báo cáo PDF. Tuy nhiên, kết quả đánh giá mới dừng ở môi trường cục bộ và dữ liệu kiểm thử có kiểm soát, chưa bao gồm đánh giá trên dữ liệu lớn, nhiều người dùng đồng thời hoặc môi trường triển khai thực tế.

4.5. Hạn chế và hướng phát triển

Mặc dù hệ thống NutriTrack đã triển khai được các chức năng chính phục vụ quản lý dinh dưỡng cá nhân, kết quả thực nghiệm vẫn còn một số giới hạn cần tiếp tục cải thiện.

Trước hết, quá trình kiểm thử mới được thực hiện chủ yếu trên môi trường cục bộ và dữ liệu có kiểm soát, do đó chưa phản ánh đầy đủ tình huống nhiều người dùng truy cập đồng thời trong thời gian dài. Cơ sở dữ liệu thực phẩm đã có các trường dinh dưỡng cơ bản, tuy nhiên số lượng món ăn và sản phẩm đóng gói vẫn phụ thuộc vào dữ liệu nhập sẵn hoặc dữ liệu do người dùng đóng góp. Điều này có thể ảnh hưởng đến kết quả tìm kiếm, chấm điểm món ăn và lập kế hoạch bữa ăn.

Đối với các thuật toán xử lý, Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights và Meal Planner đều phụ thuộc vào độ đầy đủ của dữ liệu đầu vào. Nếu người dùng ghi nhật ký ăn uống không thường xuyên, nhập sai cân nặng hoặc bỏ qua dữ liệu luyện tập, kết quả tổng hợp và các gợi ý có thể bị sai lệch. Riêng Meal Planner đã hỗ trợ loại trừ thực phẩm dị ứng và ghim nguyên liệu khi sinh gợi ý bữa ăn, nhưng kết quả vẫn phụ thuộc vào độ phong phú của dữ liệu nguyên liệu, mẫu bữa ăn và cấu hình bữa ăn. Chức năng Hybrid Nutrition Scanner cũng chịu ảnh hưởng bởi chất lượng ảnh, góc chụp, độ rõ của nhãn sản phẩm và cách trình bày thông tin dinh dưỡng trên bao bì. Vì vậy, kết quả trích xuất từ ảnh cần tiếp tục được kiểm soát bằng bước xác nhận của người dùng.

Bảng 4.5. Hạn chế và hướng phát triển của hệ thống

Hạn chế hiện tại

Hướng phát triển

Dữ liệu thực nghiệm chủ yếu được kiểm tra trên môi trường cục bộ

Triển khai thử nghiệm trên môi trường máy chủ và kiểm tra với nhiều tài khoản người dùng

Cơ sở dữ liệu thực phẩm và sản phẩm đóng gói còn phụ thuộc vào dữ liệu nhập sẵn

Mở rộng nguồn dữ liệu thực phẩm và bổ sung cơ chế kiểm duyệt dữ liệu do người dùng đóng góp

Kết quả Adaptive TDEE phụ thuộc vào nhật ký cân nặng và năng lượng nhiều ngày

Bổ sung cơ chế nhắc nhập dữ liệu và đánh giá độ tin cậy của từng giai đoạn theo dõi

Meal Planner đã có loại trừ dị ứng và ghim nguyên liệu, nhưng còn phụ thuộc vào dữ liệu nguyên liệu và mẫu bữa ăn

Mở rộng mẫu bữa ăn, bổ sung ràng buộc về khẩu vị, ngân sách, thời gian chuẩn bị và mục tiêu theo từng giai đoạn

Scanner phụ thuộc vào chất lượng ảnh, barcode và thông tin trên nhãn sản phẩm

Cải thiện bước kiểm tra dữ liệu sau nhận dạng và quy trình xác nhận trước khi lưu

Báo cáo PDF mới tập trung vào dữ liệu theo tuần và tháng

Bổ sung biểu đồ xu hướng dài hạn và so sánh giữa các giai đoạn theo dõi

Trong các phiên bản tiếp theo, hệ thống có thể được phát triển theo ba hướng chính. Thứ nhất, cần mở rộng dữ liệu thực phẩm và tăng khả năng kiểm soát chất lượng dữ liệu,

đặc biệt đối với dữ liệu sản phẩm do người dùng đóng góp. Thứ hai, cần đánh giá hệ thống trong điều kiện sử dụng dài hạn để kiểm tra khả năng vận hành của dashboard, Adaptive TDEE và báo cáo PDF. Thứ ba, các chức năng gợi ý như Health Insights và Meal Planner có thể được mở rộng theo hướng cá nhân hóa sâu hơn, trong đó tập trung vào khẩu vị, ngân sách, thời gian chuẩn bị và mục tiêu dinh dưỡng theo từng giai đoạn. Các hướng phát triển này giúp hệ thống phù hợp hơn với điều kiện sử dụng thực tế, đồng thời vẫn giữ trọng tâm là theo dõi dữ liệu dinh dưỡng cá nhân.

4.6. Kết luận chương

Chương 4 đã trình bày quá trình thực nghiệm và đánh giá hệ thống NutriTrack sau giai đoạn triển khai. Nội dung chương tập trung vào môi trường thực nghiệm, kiểm thử chức năng, thực nghiệm các thuật toán chính, đánh giá kết quả, đồng thời nêu ra một số hạn chế và hướng phát triển tiếp theo. Trong phạm vi kiểm thử cục bộ, các nhóm chức năng chính của hệ thống đã hoạt động theo luồng nghiệp vụ đã thiết kế, bao gồm xác thực tài khoản, quản lý hồ sơ, ghi nhận nhật ký dinh dưỡng, theo dõi nước uống, cân nặng, luyện tập, thống kê dashboard, lập kế hoạch bữa ăn, quét thông tin sản phẩm và xuất báo cáo PDF.

Bảng 4.6. Tổng hợp nội dung đánh giá trong Chương 4

Nội dung đánh giá

Kết quả tổng quát

Môi trường thực nghiệm

Hệ thống được kiểm thử trên môi trường cục bộ với backend, frontend và cơ sở dữ liệu

Kiểm thử chức năng

Các chức năng chính có thể thực hiện theo luồng người dùng đã xây dựng

Thực nghiệm thuật toán

Các thuật toán chính trả về kết quả theo dữ liệu đầu vào

Xử lý lỗi

API phản hồi lỗi theo cấu trúc JSON thống nhất

Hạn chế

Kết quả còn phụ thuộc vào dữ liệu đầu vào, phạm vi kiểm thử và chất lượng dữ liệu thực phẩm

Thông qua thực nghiệm, hệ thống cho thấy khả năng liên kết giữa dữ liệu người dùng, nhật ký dinh dưỡng và các thành phần tính toán. Dữ liệu hồ sơ được dùng để xác định các chỉ số nền tảng; dữ liệu nhật ký được dùng để cập nhật dashboard, báo cáo và Adaptive TDEE; dữ liệu thực phẩm được dùng trong chấm điểm món ăn và lập kế hoạch bữa ăn. Tuy nhiên, kết quả trong chương này mới phản ánh phạm vi thử nghiệm của đề án, chưa

bao quát đầy đủ điều kiện sử dụng thực tế với nhiều người dùng và dữ liệu dài hạn. Đây là cơ sở để hệ thống tiếp tục được phát triển theo các hướng như mở rộng dữ liệu thực phẩm, kiểm thử triển khai, cải thiện nhận dạng nhãn dinh dưỡng và mở rộng thêm các ràng buộc cá nhân khác trong chức năng gợi ý bữa ăn.

KẾT LUẬN

Đồ án đã trình bày quá trình nghiên cứu, phân tích, thiết kế, xây dựng và đánh giá hệ thống NutriTrack, một hệ thống hỗ trợ quản lý dinh dưỡng cá nhân. Xuất phát từ nhu cầu theo dõi năng lượng, thành phần dinh dưỡng và chỉ số cơ thể theo dữ liệu hằng ngày, hệ thống được xây dựng theo mô hình ứng dụng web với backend xử lý nghiệp vụ, frontend cung cấp giao diện tương tác và cơ sở dữ liệu lưu trữ thông tin người dùng, thực phẩm, nhật ký ăn uống, cân nặng, nước uống, luyện tập và báo cáo.

Trong quá trình thực hiện, đồ án đã làm rõ cơ sở lý thuyết liên quan đến quản lý dinh dưỡng cá nhân, bao gồm các khái niệm về năng lượng, protein, carbohydrate, chất béo, chất xơ, đường, natri, BMI, BMR, TDEE và mục tiêu dinh dưỡng. Trên cơ sở đó, hệ thống được phân tích theo các nhóm chức năng chính như quản lý tài khoản, thiết lập hồ sơ người dùng, ghi nhận nhật ký ăn uống, theo dõi chỉ số cơ thể, thống kê dữ liệu dinh dưỡng, đề xuất bữa ăn, quét thông tin sản phẩm và sinh báo cáo. Các chức năng này được tổ chức theo kiến trúc tách biệt giữa tầng giao diện, tầng xử lý nghiệp vụ và tầng dữ liệu.

Về mặt triển khai, hệ thống đã xây dựng được các chức năng cốt lõi phục vụ quá trình theo dõi dinh dưỡng cá nhân. Người dùng có thể đăng ký, đăng nhập, cập nhật hồ sơ, thêm món ăn vào nhật ký, ghi nhận cân nặng, nước uống và hoạt động luyện tập. Dựa trên các dữ liệu này, hệ thống có thể tổng hợp dashboard, tính toán chỉ số nền tảng, đánh giá chất lượng món ăn, đưa ra cảnh báo sức khỏe, lập kế hoạch bữa ăn và xuất báo cáo theo khoảng thời gian. Các thành phần như Nutrition Core, Adaptive TDEE, Food Scoring, Health Insights, Meal Planner và Hybrid Nutrition Scanner được triển khai nhằm hỗ trợ xử lý dữ liệu dinh dưỡng thay vì chỉ lưu trữ thông tin thô.

Trong phạm vi kiểm thử cục bộ, kết quả thực nghiệm cho thấy các nhóm chức năng chính đã hoạt động theo luồng nghiệp vụ đã thiết kế. Các API phản hồi dữ liệu theo cấu trúc JSON thống nhất, các chức năng yêu cầu đăng nhập được bảo vệ bằng cơ chế xác thực, các thao tác thêm, sửa, xóa dữ liệu được kiểm tra theo phạm vi người dùng hiện tại. Bên cạnh đó, các thuật toán xử lý chính cho kết quả phù hợp với dữ liệu đầu vào trong phạm vi kiểm thử. Adaptive TDEE phản ánh sự khác biệt giữa công thức tính tĩnh và dữ liệu thực tế; Food Scoring hỗ trợ đánh giá món ăn theo mật độ dinh dưỡng; Health Insights chuyển dữ liệu nhật ký thành cảnh báo hoặc gợi ý; Meal Planner tạo khẩu phần theo mục tiêu năng lượng và thành phần dinh dưỡng; Hybrid Nutrition Scanner hỗ trợ thu thập thông tin sản phẩm từ barcode hoặc ảnh nhãn dinh dưỡng.

Mặc dù vậy, hệ thống vẫn còn một số hạn chế. Phạm vi kiểm thử chủ yếu được thực hiện trên môi trường cục bộ và dữ liệu có kiểm soát, chưa đánh giá đầy đủ trong điều kiện nhiều người dùng truy cập đồng thời. Cơ sở dữ liệu thực phẩm và sản phẩm đóng gói còn phụ thuộc vào dữ liệu nhập sẵn hoặc dữ liệu do người dùng đóng góp. Kết quả

của các chức năng như Adaptive TDEE, Health Insights và Meal Planner phụ thuộc đáng kể vào mức độ đầy đủ và chính xác của nhật ký người dùng. Ngoài ra, chức năng quét thông tin dinh dưỡng từ ảnh vẫn có thể bị ảnh hưởng bởi chất lượng ảnh, góc chụp và cách trình bày nhãn sản phẩm.

Trong hướng phát triển tiếp theo, hệ thống có thể được mở rộng theo một số hướng chính. Thứ nhất, cần bổ sung và kiểm duyệt dữ liệu thực phẩm để tăng độ bao phủ của cơ sở dữ liệu dinh dưỡng. Thứ hai, hệ thống cần được triển khai thử nghiệm trên môi trường máy chủ để đánh giá khả năng vận hành, thời gian phản hồi và mức độ phù hợp khi có nhiều người dùng. Thứ ba, chức năng gợi ý bữa ăn có thể tiếp tục được mở rộng trên cơ sở các ràng buộc đã có như loại trừ thực phẩm dị ứng và ghim nguyên liệu, trong đó tập trung thêm vào khẩu vị, ngân sách, thời gian chuẩn bị, chế độ ăn đặc thù và mục tiêu theo từng giai đoạn. Thứ tư, chức năng báo cáo có thể được bổ sung biểu đồ xu hướng dài hạn, hỗ trợ theo dõi sự thay đổi của cân nặng, năng lượng, macro và thói quen ăn uống qua nhiều tuần hoặc nhiều tháng.

Tổng thể, đề án đã hoàn thành mục tiêu xây dựng một hệ thống quản lý dinh dưỡng cá nhân có khả năng ghi nhận, tính toán, tổng hợp và đánh giá dữ liệu dựa trên thông tin người dùng cung cấp. Kết quả đạt được cho thấy hệ thống có thể đáp ứng các nghiệp vụ chính của bài toán, đồng thời tạo nền tảng để tiếp tục phát triển các chức năng phân tích và gợi ý dinh dưỡng trong các phiên bản sau.

TÀI LIỆU THAM KHẢO

Tiếng Việt

[1] Viện Dinh dưỡng Quốc gia ...

Tiếng Anh

[2] Cloudinary ...

[3] Drewnowski ...

[4] EFSA ...

[5] FAO/WHO/UNU ...

[6] Google ...

[7] Hall ...

[8] Hyndman ...

[9] Jones ...

[10] Mifflin ...

[11] Open Food Facts ...

[12] React ...

[13] Sequelize ...

[14] World Health Organization ...

...